

Complejidad Computacional

Santiago Figueira

Departamento de Computación - FCEN - UBA

1er cuatrimestre 2025

(versión sin transiciones)

Contenidos

Clase 1

- Preliminares
- Representaciones en binario
- Problemas de decisión
- Máquinas de Turing
- Configuraciones y cálculos

Clase 2

- Funciones computables
- Tiempo de cómputo
- Codificación de máquinas
- Variantes de máquinas

Clase 3

- Halt y máquina universal
- La clase **P**
- La clase **NP**

Clase 4

- Máquinas no-determinísticas
- Reducibilidad polinomial
- Lógica proposicional

Clase 5

- Mini-configuraciones
- Teorema de Cook-Levin
- Problemas **NP-completos**
- La clase **coNP**
- Las clases **EXPTIME** y **NEXPTIME**

Clase 6

- La jerarquía de tiempos determinísticos
- La jerarquía de tiempos no-determinísticos
- Teorema de Ladner

Clase 7

- Espacio usado por un cómputo
- Espacio polinomial: **PSPACE** y **NPSPACE**
- Teorema de Savitch

Clase 8

- Espacio logarítmico: **L** y **NL**
- Teorema de Immerman-Szelepcsényi

Clase 9

- La jerarquía polinomial
- Problemas **Σ_i^P -completos**

Clase 10

- Máquinas con oráculo
- Teorema de Baker, Gill, Solovay
- La jerarquía polinomial y **NP** con oráculos

Clase 11

- Circuitos booleanos
- La clase **P/poly**
- Tamaño y profundidad de circuitos

Clase 12

- Las clases **NC_{nu}** y **AC_{nu}**
- Uniformidad: las clases **NC** y **AC**
- P-completitud**

Clase 1

Clase 1

Preliminares
Representaciones en binario
Problemas de decisión
Máquinas de Turing
Configuraciones y cálculos

Clase 2

Funciones computables
Tiempo de cómputo
Codificación de máquinas
Variantes de máquinas

Clase 3

Halt y máquina universal
La clase P
La clase NP

Clase 4

Máquinas no-determinísticas
Reducibilidad polinomial
Lógica proposicional

Clase 5

Mini-configuraciones
Teorema de Cook-Levin
Problemas NP -completos
La clase $coNP$
Las clases $EXPTIME$ y $NEXPTIME$

Clase 6

La jerarquía de tiempos determinísticos
La jerarquía de tiempos no-determinísticos
Teorema de Ladner

Clase 7

Espacio usado por un cómputo
Espacio polinomial: $PSPACE$ y $NPSPACE$
Teorema de Savitch

Clase 8

Espacio logarítmico: L y NL
Teorema de Immerman-Szelepcsényi

Clase 9

La jerarquía polinomial
Problemas Σ_i^P -completos

Clase 10

Máquinas con oráculo
Teorema de Baker, Gill, Solovay
La jerarquía polinomial y NP con oráculos

Clase 11

Circuitos booleanos
La clase P_{poly}
Tamaño y profundidad de circuitos

Clase 12

Las clases NC_{nu} y AC_{nu}
Uniformidad: las clases NC y AC
 P -completitud

1-1

Organización

- Docentes
 - Santiago Figueira (Prof.)
 - Santiago Cifuentes (JTP)
 - Ayelén Dinkel (AY1)
 - Joaquín Laks (AY2)
 - Belén Loleo Saigos (AY2)
 - Martín Santesteban (AY2)
 - Tomás Spognardi (AY2)
- Miércoles de 17 a 22. Aula 1309 Pab. 0 + ∞ .
- Parciales: 14/5 y 25/6; Recus: 2/7 y 16/7
- Bibliografía:
 - Sanjeev Arora y Boaz Barak. Computational complexity: a modern approach, Cambridge University Press, 2009, <https://theory.cs.princeton.edu/complexity/book.pdf>
 - apunte (en el campus)
 - estas slides

1-2

Preliminares

Clase 1

Preliminares

Representaciones en binario

Problemas de decisión

Máquinas de Turing

Configuraciones y cálculos

1-3

Notación

Dado un alfabeto Γ :

Notación: Conjunto de palabras sobre un alfabeto

Γ^* es el conjunto con todas las palabras formadas sobre Γ

Ejemplo

Si $\Gamma = \{0, 1\}$, entonces $\Gamma^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, 001, \dots\}$

Notación: Palabra vacía

ϵ representa la palabra vacía.

Notación: Longitud de una palabra

Si $\sigma \in \Gamma^*$ entonces $|\sigma|$ es la longitud de σ , es decir la cantidad de letras de σ .

Ejemplo

Si $\Gamma = \{0, 1\}$, $\sigma = 101$, $\tau = \epsilon$ entonces $|\sigma| = 3$, $|\tau| = 0$.

1-4

Notación

Notación: Conjunto de palabras de largo k

Γ^k es el conjunto con todas las palabras formadas sobre Γ de longitud k . Equivalentemente, a veces las vamos a tomar como tuplas de dimensión k .

Ejemplo

Si $\Gamma = \{0, 1\}$, entonces $\Gamma^2 = \{00, 01, 10, 11\}$.

También puede ser $\Gamma^2 = \{(0, 0), (0, 1), (1, 0), (1, 1)\}$.

Va a quedar claro del contexto cuál usamos.

1-5

Notación

Notación: Concatenación de palabras

Si $\sigma, \tau \in \Gamma^*$, $\sigma\tau$ representa la concatenación de σ con τ .

Notación: i -ésimo elemento de una palabra

Si $\sigma \in \Gamma^*$, $i \in \{0, \dots, |\sigma| - 1\}$, entonces $\sigma(i)$ es la $(i + 1)$ -ésima letra de σ .

Ejemplo

Si $\Gamma = \{0, 1\}$, y $\sigma = 101$ entonces $\sigma(0) = 1$, $\sigma(1) = 0$, $\sigma(2) = 1$.

1-6

Representaciones en binario

Clase 1

Preliminares

Representaciones en binario

Problemas de decisión

Máquinas de Turing

Configuraciones y cálculos

1-7

Representaciones de números naturales

Buscamos

- fijar un alfabeto; elegimos el binario: $\{0, 1\}$
- representar objetos (números, grafos, etc.) en binario
- trabajar con métodos efectivos que transforman unos objetos en otros

Definición

Si $n \in \mathbb{N}$, notamos $[n]$ a la representación de n en binario y notamos $|n|$ al tamaño de $[n]$, es decir $|n| = \lceil \log n \rceil$.

Ejemplos

- $[0] = 0$; $|0| = 1$
- $[5] = 101$; $|5| = 3$

1-8

Representaciones autodelimitantes

Definición (codificación autodelimitante de cadenas)

Para $\sigma = b_1 b_2 \dots b_n$ ($b_i \in \{0, 1\}$), definimos

$$\langle \sigma \rangle = b_1 b_1 \ b_2 b_2 \ \dots \ b_n b_n \ 10$$

$\langle \sigma \rangle$ es autodelimitante (el 10 marca el final).

Ejemplos

- $\langle 101 \rangle = 11 \ 00 \ 11 \ 10$
- $\langle \epsilon \rangle = 10$

(los espacios no forman parte de la codificación)

1-9

Representaciones de listas, tuplas, conjuntos

Definición

Codificamos la lista $L = \sigma_1, \dots, \sigma_n$ ($\sigma_i \in \{0, 1\}^*$) como

$$\langle L \rangle = \langle \sigma_1 \rangle \ \langle \sigma_2 \rangle \ \dots \ \langle \sigma_n \rangle \ 01$$

(01 marca fin de lista). Codificamos tuplas y conjuntos como listas.

Ejemplos

- $\langle 11, 100, 0001 \rangle = \langle (11, 100, 0001) \rangle = \langle \{11, 100, 0001\} \rangle = 111110 \ 11000010 \ 0000001110 \ 01$
- $\langle \emptyset \rangle = 01$

(los espacios no forman parte de la codificación)

1-10

Representaciones de otros objetos

Usaremos misma notación $\langle X \rangle$ para otros objetos matemáticos X a veces sin especificar formalmente su representación.

Ejemplo

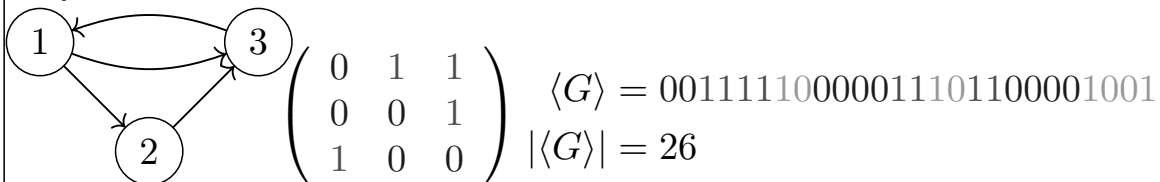
$$A = \begin{pmatrix} b_{11} & b_{12} & \dots & b_{1k} \\ b_{21} & b_{22} & \dots & b_{2k} \\ & & \dots & \\ b_{\ell 1} & b_{\ell 2} & \dots & b_{\ell k} \end{pmatrix} \quad (b_{ij} \in \{0, 1\}^*)$$

$$\langle A \rangle = \langle \langle b_{11}, \dots, b_{1k} \rangle \langle b_{21}, \dots, b_{2k} \rangle \dots \langle b_{\ell 1}, \dots, b_{\ell k} \rangle \rangle$$

$$|\langle A \rangle| = 2 \cdot \ell \cdot (k + 1) + 2$$

Ejemplo

Si $G = (E, V)$ es un digrafo, podemos representar G con la matriz de adyacencia



1-11

Representaciones de otros objetos

Notación: codificación de tuplas de objetos

Siempre supondremos que cada objeto O (lista, grafo, número, etc.) tiene una codificación razonable $\langle O \rangle$ en binario.

Muchas veces, vamos a notar la codificación de tuplas $(O_1, \dots, O_n)$ de objetos simplemente como

$$\langle O_1, \dots, O_n \rangle$$

en vez de

$$\langle \langle O_1 \rangle, \dots, \langle O_n \rangle \rangle$$

Del mismo modo, usaremos $|O|$ en vez de $|\langle O \rangle|$ para referirnos al tamaño de la codificación de O .

Todo lo que dijimos hasta acá sobre notación binaria, vale para otros alfabetos finitos Γ .

- a veces vamos a usar alfabetos con más de dos símbolos
- a veces vamos usaremos notación unaria

1-12

Problemas de decisión

Clase 1

Preliminares

Representaciones en binario

Problemas de decisión

Máquinas de Turing

Configuraciones y cálculos

1-13

Problemas de decisión

Nos restringimos a **problemas de decisión**, que se formalizan como funciones booleanas:

$$f : \{0, 1\}^* \rightarrow \{0, 1\}$$

Cada función booleana f representa el lenguaje (conjunto de palabras en $\{0, 1\}^*$)

$$\mathcal{L}(f) = \{x : f(x) = 1\} \subseteq \{0, 1\}^*$$

Y viceversa: cualquier lenguaje $\mathcal{L} \subseteq \{0, 1\}^*$ se puede representar por la función booleana

$$\chi_{\mathcal{L}}(x) = \begin{cases} 1 & \text{si } x \in \mathcal{L} \\ 0 & \text{si no} \end{cases}$$

problemas de decisión = funciones booleanas = lenguajes

1-14

Ejemplos de problemas de decisión

Ejemplo

El problema de decidir si un número x es par se formaliza con

$$f : \{0, 1\}^* \rightarrow \{0, 1\}$$
$$f([x]) = \begin{cases} 1 & \text{si } x \text{ es par} \\ 0 & \text{si no} \end{cases} \quad \text{o bien con } \mathcal{L} = \{[x] : x \text{ es par}\}$$

Ejemplo

El problema de decidir si un grafo G tiene un k -coloreo

$$f : \{0, 1\}^* \rightarrow \{0, 1\}$$

$$f(\langle\langle G \rangle, [k] \rangle) = \begin{cases} 1 & \text{si } G \text{ tiene un } k\text{-coloreo} \\ 0 & \text{si no} \end{cases}$$

o bien con

$$\mathcal{L} = \{\langle\langle G \rangle, [k] \rangle : G \text{ tiene un } k\text{-coloreo}\}$$

1-15

Notación: $\mathcal{L}$

A partir de ahora $\mathcal{L}$ va a ser un lenguaje en $\{0, 1\}^*$.

1-16

Máquinas de Turing

Clase 1

Preliminares

Representaciones en binario

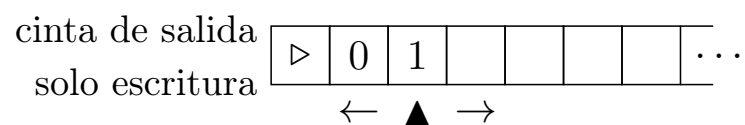
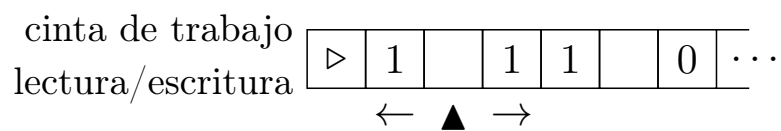
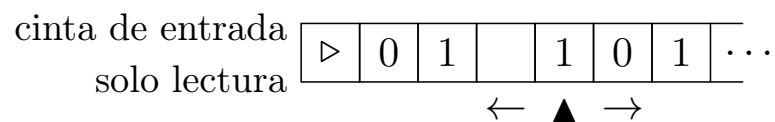
Problemas de decisión

Máquinas de Turing

Configuraciones y cálculos

1-17

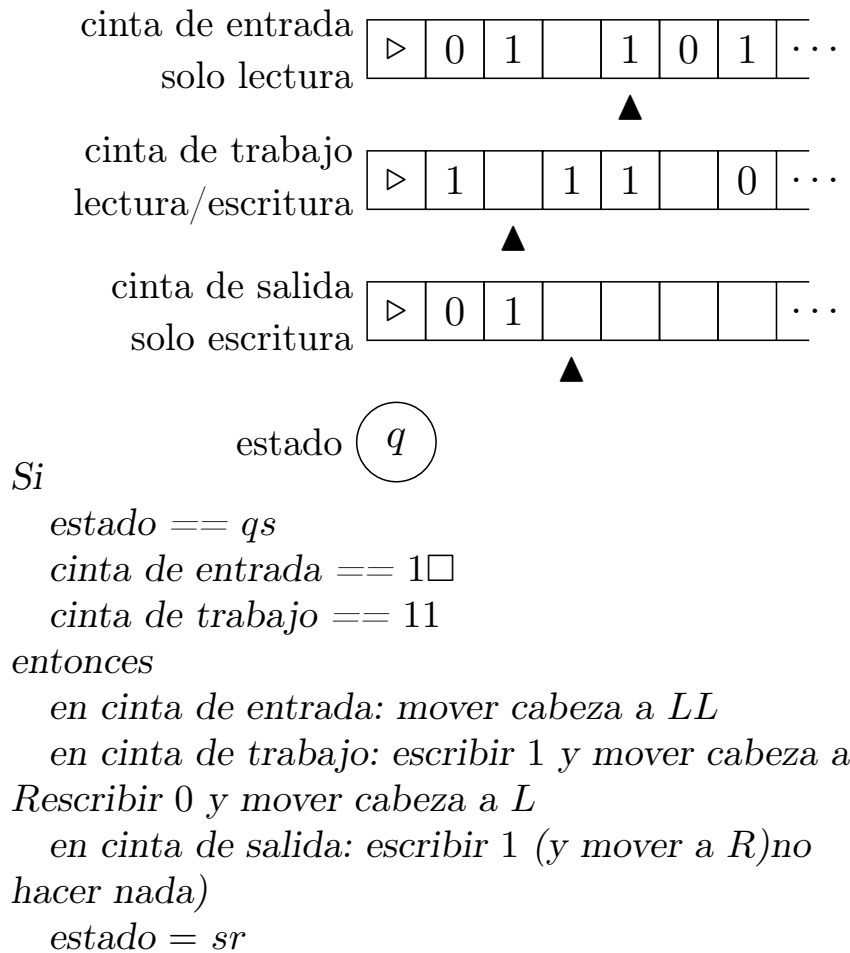
Máquina de Turing con 3 cintas



estado q

1-18

Máquina de Turing con 3 cintas



1-19

Máquina de Turing con 3 cintas

Definición

Una **máquina de Turing con 3 cintas** (o simplemente **máquina con 3 cintas**) es una tripla (Σ, Q, δ) , donde

- Q : es el conjunto finito de **estados**
 - $q_0 \in Q$: estado inicial
 - $q_f \in Q$: estado final
 - Σ : es el **alfabeto** (conjunto finito de símbolos)
 - $\triangleright \in \Sigma$: comienzo de cinta
 - $\square \in \Sigma$: blanco
- $\Sigma = \{0, 1, \triangleright, \square\}$ es el alfabeto **estándar**.
- $\delta : Q \times \Sigma \times \Sigma \rightarrow \{L, R, S\} \times \Sigma \times \{L, R, S\} \times (\Sigma \cup \{S\}) \times Q$ es la **función de transición**
 - L : izquierda (*Left*)
 - R : derecha (*Right*)
 - S : quedarse en el lugar (*Stay*)

1-20

La función de transición

INSTRUCCIÓN

si *estado* == $q \in Q$
 cinta de entrada == $a \in \Sigma$
 cinta de trabajo == $b \in \Sigma$
entonces
 en cinta de entrada: mover cabeza a $L/R/S$
 en cinta de trabajo: escribir $c \in \Sigma$ y
 mover cabeza a $L/R/S$
 en cinta de salida: escribir $d \in \Sigma$ (y mover
 a R) / no hacer nada
 estado = $r \in Q$

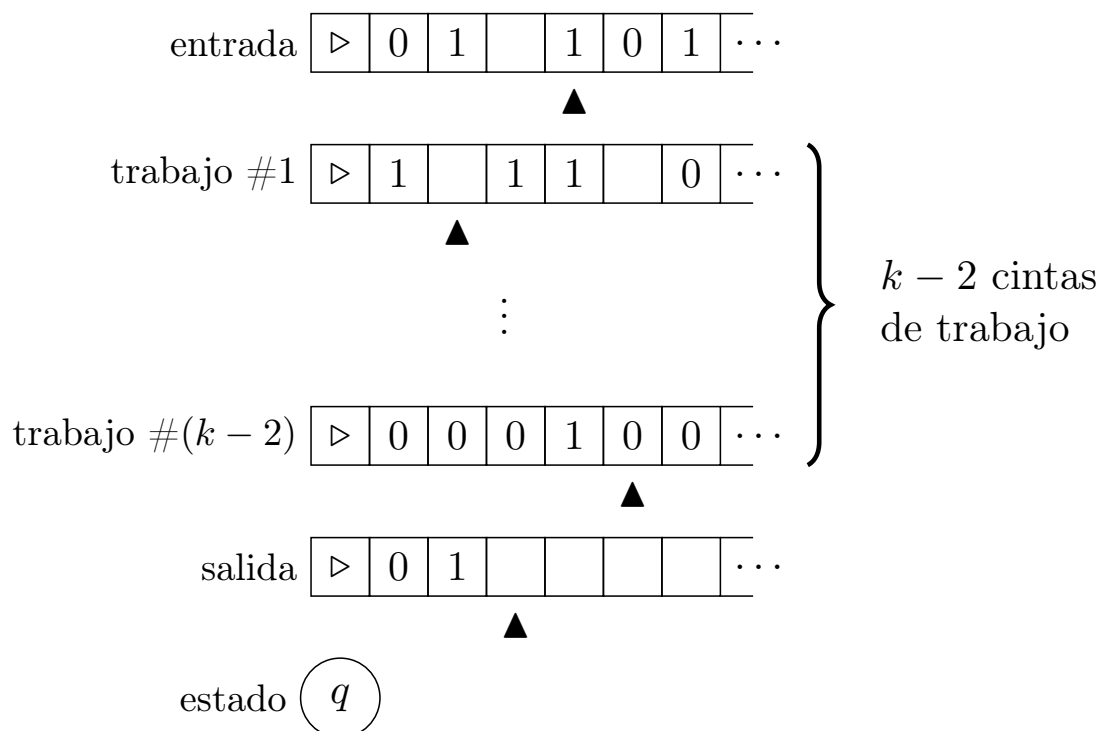
$$\delta : Q \times \Sigma \times \Sigma \rightarrow \{L, R, S\} \times \Sigma \times \{L, R, S\} \times (\Sigma \cup \{S\}) \times Q$$

Hay restricciones sobre δ :

- $\delta(*, \triangleright, *)$ no puede ser de la forma $(L, *, *, *, *)$
- $\delta(*, *, \triangleright)$ no puede ser de la forma $(*, *, L, *, *)$
- $\delta(q_f, a, b)$ solo puede ser de la forma (S, b, S, S, q_f)
 - el estado q_f termina el movimiento

1-21

Máquina de Turing con k cintas



1-22

Máquina de Turing (general)

Definición

Una **máquina de Turing** (o simplemente **máquina**) es una tripla (Σ, Q, δ) , donde

- Q : es el conjunto finito de **estados**
 - $q_0 \in Q$: estado inicial
 - $q_f \in Q$: estado final
- Σ : es el **alfabeto** (conjunto finito de símbolos)
 - $\triangleright \in \Sigma$: comienzo de cinta
 - $\square \in \Sigma$: blanco

Si no aclaramos, el alfabeto es el **estándar** $\Sigma = \{0, 1, \triangleright, \square\}$

- La **función de transición** es

$$\delta : Q \times \Sigma^{k-1} \rightarrow \underbrace{\{L, R, S\}}_{\text{entrada}} \times \underbrace{\Sigma^{k-2} \times \{L, R, S\}^{k-2}}_{k-2 \text{ trabajos}} \times \underbrace{(\Sigma \cup \{S\})}_{\text{salida}} \times Q$$

donde $k \geq 3$.

1-23

Configuraciones y cálculos

Clase 1

Preliminares

Representaciones en binario

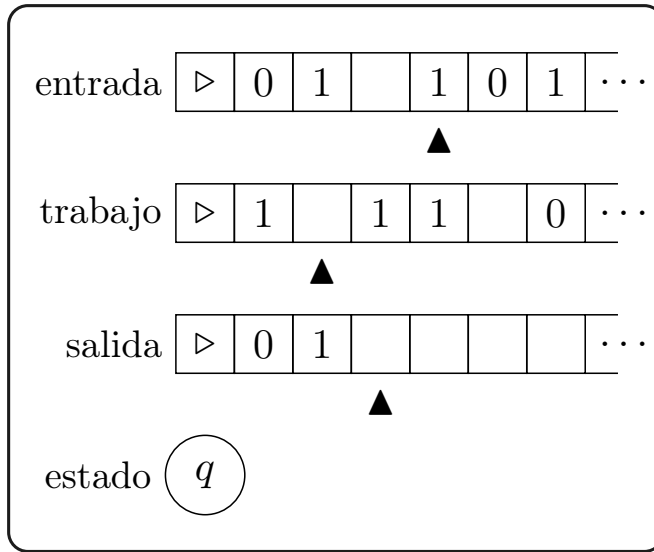
Problemas de decisión

Máquinas de Turing

Configuraciones y cálculos

Configuración

Una **configuración** es la fotografía de la máquina en un determinado momento, por ejemplo



Tiene toda la información sobre:

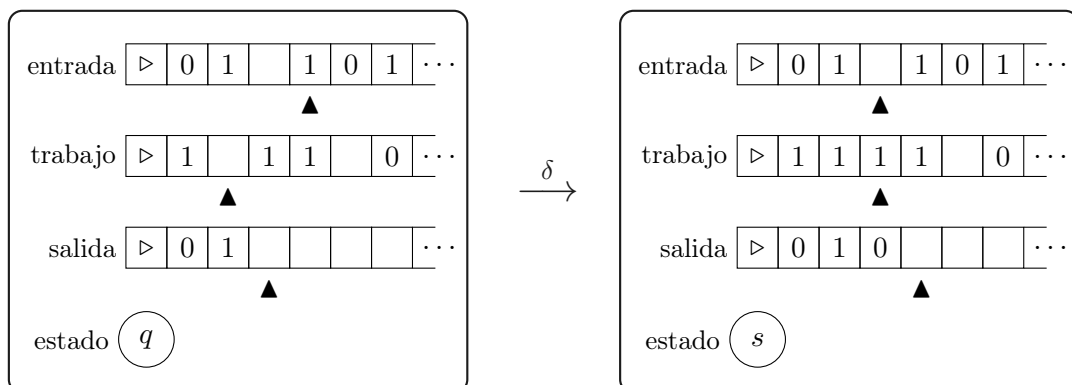
- contenido de todas las cintas
- posición de cada cabeza
- estado

Alcanza con considerar cintas que solo tienen finitos 0s y 1s (es decir, a partir de una posición i , todas las cintas tienen blancos a la derecha de i).

1-25

Evolución de un cómputo

La función de transición δ hace **evolucionar** el cómputo **en un paso** a partir de una dada configuración C :



Si

$\text{estado} == q$

$\text{cinta de entrada} == 1$

$\text{cinta de trabajo} == \square$

entonces

en cinta de entrada: mover cabeza a L

en cinta de trabajo: escribir 1 y mover cabeza a R

en cinta de salida: escribir 1 (y mover a R)

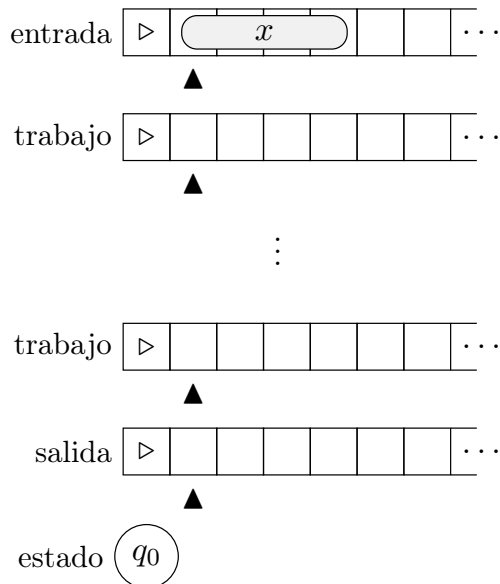
$\text{estado} = s$

$$\delta(q, 1, \square) = (L, 1, R, 0, s)$$

1-26

Configuración inicial

La **configuración inicial** de la máquina $M = (\Sigma, Q, \delta)$ con k cintas para $x \in \Sigma^*$ es:

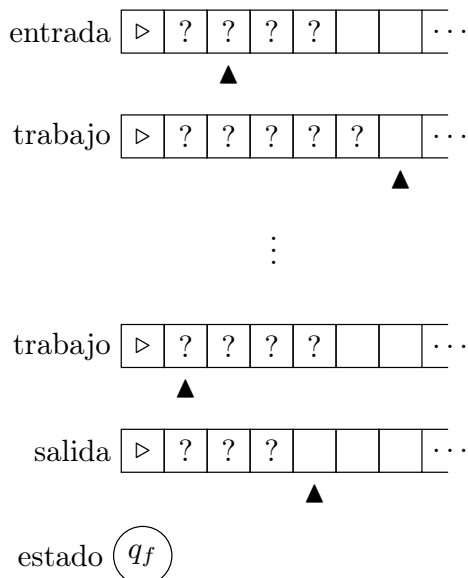


- la cinta de entrada contiene x y el resto en blanco
- todas las cintas de trabajo en blanco
- la cinta de salida en blanco
- todas las cabezas en la segunda celda
- el estado es q_0 (estado inicial)

1-27

Configuración final

Una **configuración final** de la máquina $M = (\Sigma, Q, \delta)$ con k cintas es cualquiera de esta forma:



- el estado es q_f (se congela)
- cualquier cosa escrita en las cintas de trabajo o de salida
- cualquier posición de las cabezas
- decimos que una máquina en una configuración final **paró o terminó**
- si en la cinta de salida hay escrito un $\triangleright 1$ seguido de blancos decimos que la configuración final es **aceptadora**

1-28

Notación

Notación:

Fijemos un alfabeto finito Γ tal que $\triangleright \notin \Gamma$ y definamos $\Gamma' = \Gamma \cup \{\triangleright, \square\}$.

- las máquinas van a trabajar con el alfabeto Γ' , que es lo mismo que Γ pero con el agregado de $\triangleright$ y $\square$.
- las entradas y salidas de las máquinas van a ser palabras sobre Γ

1-29

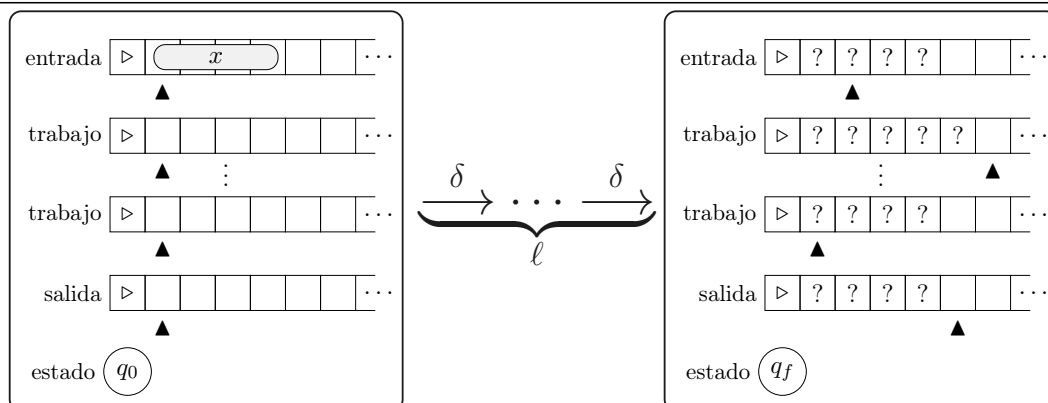
Cómputo

Definición

Un **cómputo** de $M = (\Gamma', Q, \delta)$ a partir de $x \in \Gamma^*$ es una secuencia $C_0, \dots, C_\ell$ de configuraciones tal que

- C_0 es inicial para x
- C_ℓ es final y
- C_{i+1} es la evolución de C_i en un paso.

Llamamos **longitud** del cómputo a ℓ . Decimos que M con entrada x **termina** si existe un cómputo de M a partir de x .



1-30

Clase 2

Clase 1

Preliminares
Representaciones en binario
Problemas de decisión
Máquinas de Turing
Configuraciones y cálculos

Clase 2

Funciones computables
Tiempo de cómputo
Codificación de máquinas
Variantes de máquinas

Clase 3

Halt y máquina universal
La clase **P**
La clase **NP**

Clase 4

Máquinas no-determinísticas
Reducibilidad polinomial
Lógica proposicional

Clase 5

Mini-configuraciones
Teorema de Cook-Levin
Problemas **NP-completos**
La clase **coNP**
Las clases **EXPTIME** y **NEXPTIME**

Clase 6

La jerarquía de tiempos determinísticos
La jerarquía de tiempos no-determinísticos
Teorema de Ladner

Clase 7

Espacio usado por un cómputo
Espacio polinomial: **PSPACE** y **NPSPACE**
Teorema de Savitch

Clase 8

Espacio logarítmico: **L** y **NL**
Teorema de Immerman-Szelepcsényi

Clase 9

La jerarquía polinomial
Problemas Σ_i^P -completos

Clase 10

Máquinas con oráculo
Teorema de Baker, Gill, Solovay
La jerarquía polinomial y **NP** con oráculos

Clase 11

Circuitos booleanos
La clase **P**/**poly**
Tamaño y profundidad de circuitos

Clase 12

Las clases **NC_{nu}** y **AC_{nu}**
Uniformidad: las clases **NC** y **AC**
P-completitud

2-1

Funciones computables

Clase 2

Funciones computables
Tiempo de cómputo
Codificación de máquinas
Variantes de máquinas

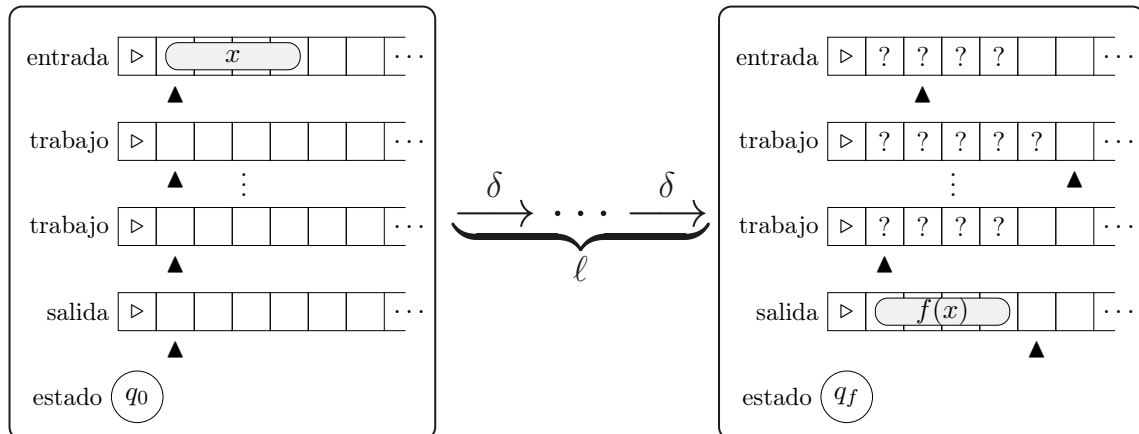
2-2

Función computada por una máquina

Definición

Sean $f : \Gamma^* \rightarrow \Gamma^*$ y $M = (\Gamma', Q, \delta)$ una máquina. Decimos que M **computa** f si para todo $x \in \Gamma^*$, hay un cómputo $C_0, \dots, C_\ell$ de M a partir de x y en C_ℓ la cinta de salida tiene escrito $f(x)$ seguido de blancos.

En este caso, notamos $M(x)$ para representar $f(x)$. Decimos que f es **computable** si existe una máquina que la computa.



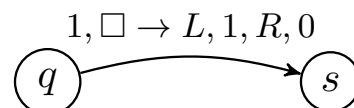
2-3

Máquinas representadas como autómatas

Podemos representar las máquinas como autómatas.

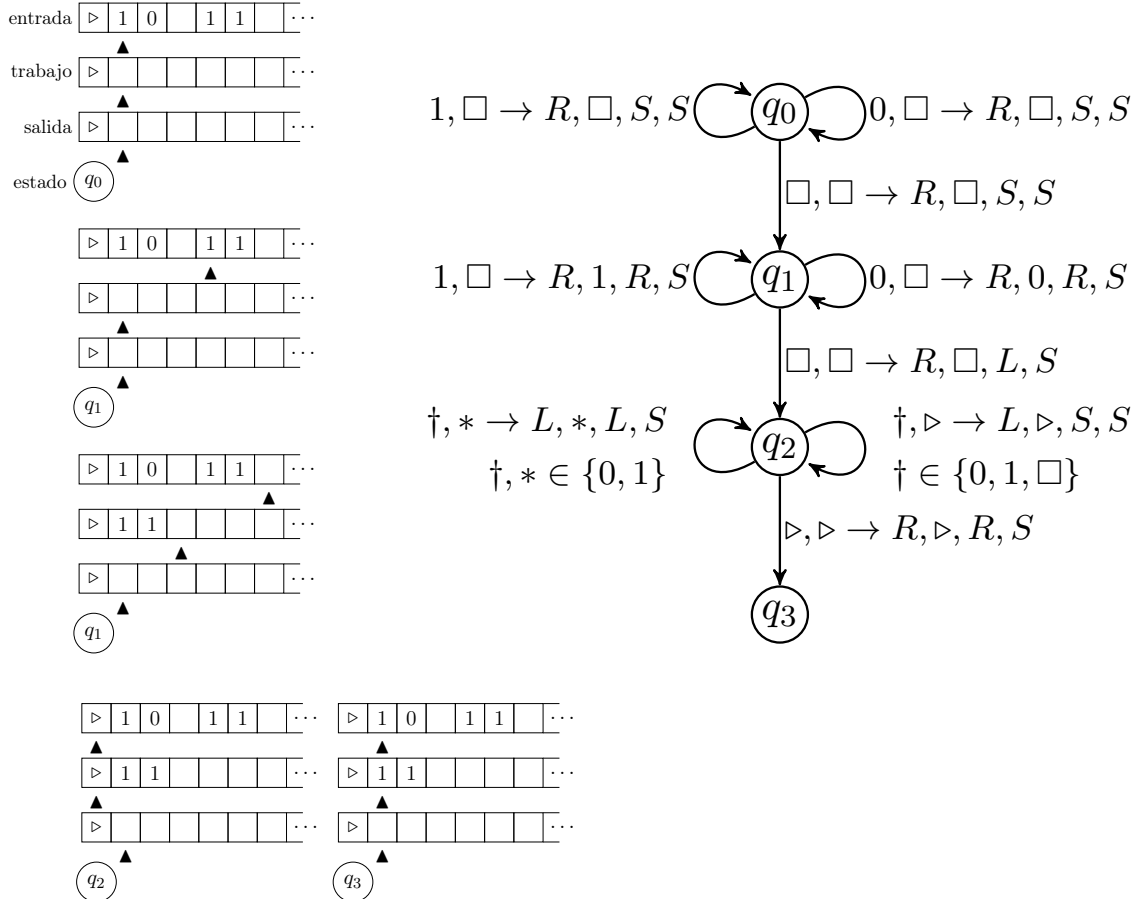
- los nodos del autómata son los estados
- las transiciones representan δ así:

$$\delta(q, 1, \square) = (L, 1, R, 0, s) =$$

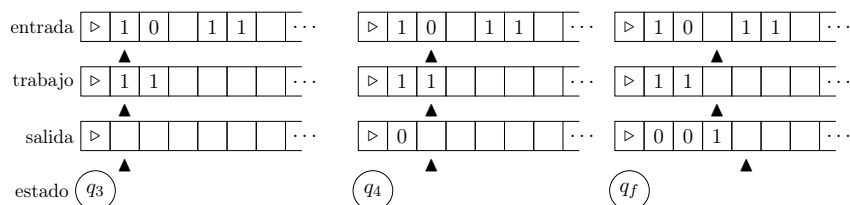
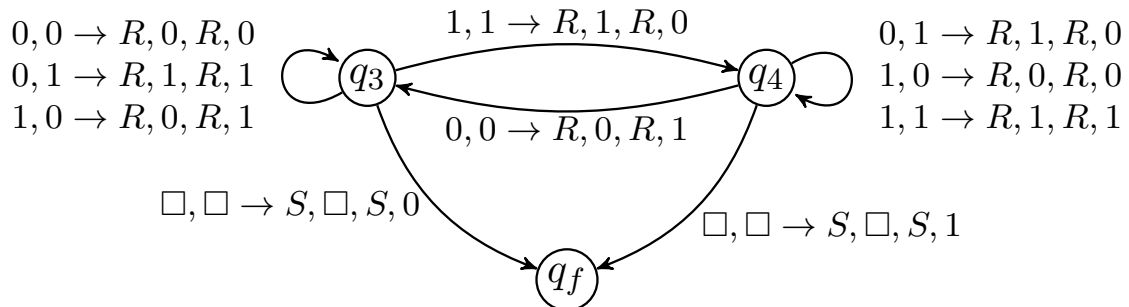


2-4

Ejemplo



2-5



Dados $x, y \in \{0, 1\}^n$, con entrada $x \square y$, devuelve $x + y$, si representamos los números en binario con el dígito menos significativo a la izquierda.

2-6

Función parcial

Definición

Una **función parcial** es una función que puede indefinirse en algunos puntos. Notamos

- $f(x) \uparrow$ cuando f está indefinida en x
- $f(x) \downarrow$ cuando f está definida en x

Llamamos **dominio de f** al conjunto

$$\text{dom}f = \{x : f(x) \downarrow\}.$$

Notación: función parcial

A las funciones parciales f las vamos a notar $f : \subseteq \Gamma \rightarrow \Gamma$ para marcar que el dominio de f no necesariamente es Γ .

En general, como las máquinas pueden **colgarse** (es decir, no terminar), van a computar funciones *parciales*.

2-7

Función *parcial* computada por una máquina

Definición

Sean $f : \subseteq \Gamma^* \rightarrow \Gamma^*$ una función parcial y $M = (\Gamma', Q, \delta)$ una máquina. Decimos que M **computa** f si para todo $x \in \Gamma^*$:

- si $f(x) \downarrow$ entonces hay un cómputo $C_0, \dots, C_\ell$ de M a partir de x y en C_ℓ la cinta de salida tiene escrito $f(x)$ seguido de blancos. Decimos que $M(x) \downarrow$.
- si $f(x) \uparrow$ entonces hay una secuencia infinita de configuraciones $C_0, C_1, \dots$ tal que C_0 es inicial C_{i+1} es la evolución de C_i en un paso y ningún C_i es final
 - es decir no hay cómputo de M a partir de x
 - M ‘se cuelga’ con entrada x

Decimos que f es **parcial computable** si existe una máquina que la computa.

2-8

Funciones *totales* vs funciones *parciales*

- las máquinas pueden ‘colgarse’ a partir de cierta entrada, es decir no llegar nunca a una configuración final
- en general las máquinas computan funciones *parciales*
- pero en la Teoría de la Complejidad estamos interesados en funciones (estándar), no en funciones *parciales*

Notación: función parcial

- ‘función’ es un mapeo (estándar) de *toda* entrada a una salida
 - las notamos como siempre: $f : \Gamma^* \rightarrow \Gamma^*$
- ‘función parcial’ es un mapeo posiblemente incompleto
 - también las notamos $f : \subseteq \Gamma^* \rightarrow \Gamma^*$ para reforzar que el dominio no necesariamente es Γ^*
- a veces, para reforzar, llamamos ‘función total’ a las primeras (las estándar)

2-9

Infinitas máquinas para la misma función

Para una función fija $f : \Gamma^* \rightarrow \Gamma^*$ computable, existen infinitas máquinas que la computan

- si existe M tal que computa f , podemos definir otra máquina que agrega estados a los de M y transiciones espurias de modo que no modifique la función que computa

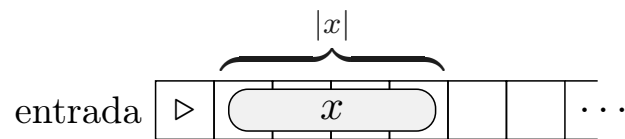
2-10

Tamaño de la entrada

Sea $x \in \Gamma^*$.

Vamos a prestar especial atención a la cantidad de celdas que se necesitan para representar x en la cinta de entrada.

Recordar que el **tamaño** de x , notado $|x|$, es la cantidad de símbolos de x .



2-11

Tiempo de cómputo

Clase 2

Funciones computables

Tiempo de cómputo

Codificación de máquinas

Variantes de máquinas

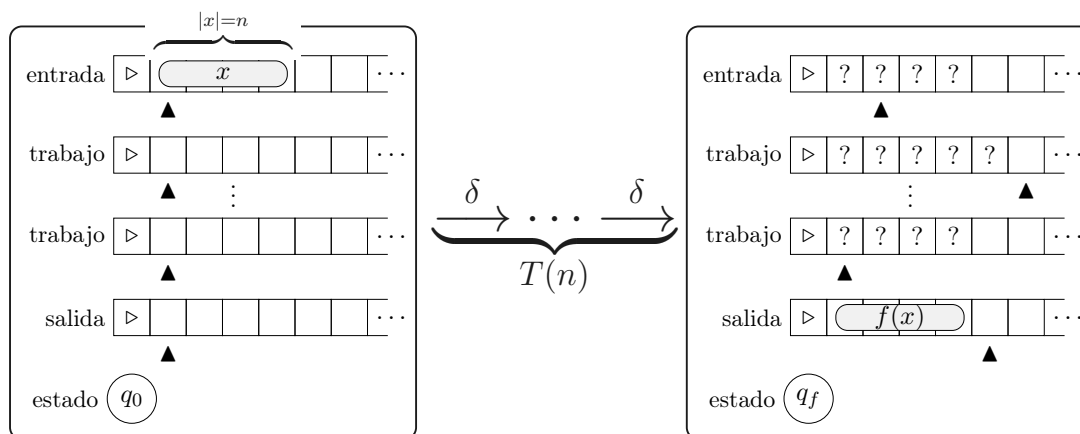
2-12

Tiempo de cómputo

Definición

Sean $f : \Gamma^* \rightarrow \Gamma^*$, $T : \mathbb{N} \rightarrow \mathbb{N}$ y $M = (\Gamma', Q, \delta)$ una máquina.

- **M corre en tiempo $T(n)$** si para todo $x \in \Gamma^*$ hay un cómputo de M a partir de x de longitud $\leq T(|x|)$ (en particular, M no se cuelga nunca).
- **M computa f en tiempo $T(n)$** si M corre en tiempo $T(n)$ y M computa f .



2-13

Notación de ‘O grande’

Definición

Sean $f, g : \mathbb{N} \rightarrow \mathbb{N}$. Decimos que $f = O(g)$ si existe c tal que para todo n suficientemente grande tenemos

$$f(n) \leq c \cdot g(n).$$

Ejemplos

- $4(n + 2) = O(n)$
- $1000000n + 1000000 = O(n)$
- $n \log n = O(n^2)$
- $5n^2 + 3n + 1 = O(n^2)$
- $2^n \neq O(n^k)$ para ningún k

Siempre que sea necesario, usaremos $\log n$ como abreviatura de $\lceil \log n \rceil$.

2-14

Cómputo en tiempo $O(T(n))$ y tiempo polinomial

Definición

M **corre en tiempo $O(T(n))$** si existe una constante c tal que para todo $x \in \Gamma^*$, salvo finitos, hay un cómputo de M a partir de x de longitud a lo sumo $c \cdot T(|x|)$.

Definición

M **corre en tiempo polinomial** si existe un polinomio p tal que M corre en tiempo $O(p)$:

Es equivalente a decir:

existe constante c tal que M corre en tiempo $O(n^c)$

2-15

Funciones computables en tiempo $O(T(n))$ / polinomial

Definición

Una función $f : \Gamma^* \rightarrow \Gamma^*$ es **computable en tiempo $T(n)$ [en tiempo $O(T(n))$]** si existe una máquina $M = (\Gamma', Q, \delta)$ tal que

- M computa f y
- M corre en tiempo $T(n)$ [en tiempo $O(T(n))$].

Definición

Una función $f : \Gamma^* \rightarrow \Gamma^*$ es **computable en tiempo polinomial** si existe una máquina $M = (\Gamma', Q, \delta)$ tal que

- M computa f y
- M corre en tiempo polinomial.

Definición

Decimos que un lenguaje $\mathcal{L}$ es **decidible** en tiempo $T(n)$, $O(T(n))$, polinomial, si $\chi_{\mathcal{L}}$ es computable en tiempo $T(n)$, $O(T)$, polinomial.

2-16

Múltiples parámetros

Buscamos computar funciones de varios parámetros, por ejemplo

$$f : \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{0, 1\}^*$$

No podemos codificar la entrada (σ, τ) como $\sigma\tau$ porque no se entiende dónde termina σ y dónde empieza τ

- podemos usar 3 símbolos: usar los blancos de la cinta de entrada y representar (σ, τ) como $\triangleright\sigma\square\tau\square\square\square\dots$, o sea



- podemos usar solo 2 símbolos (0 y 1) y usar codificaciones autodelimitantes sobre $\{0, 1\}^*$



Elegimos la segunda por comodidad de notación.

2-17

Funciones construibles en tiempo

Definición

Una función $T : \mathbb{N} \rightarrow \mathbb{N}$ es **construible en tiempo** si

- $T(n) \geq n$
- la función $1^n \mapsto [T(n)]$ es computable en tiempo $O(T(n))$

Ejemplos de funciones construibles en tiempo

- $n \log n$
- n
- n^2
- 2^n

2-18

Notación: f, T

A partir de ahora

- f siempre va a ser una función $\{0, 1\}^* \rightarrow \{0, 1\}^*$
- $T : \mathbb{N} \rightarrow \mathbb{N}$ siempre va a ser una función construible en tiempo

(salvo que se diga lo contrario)

2-19

Codificación de máquinas

Clase 2

Funciones computables

Tiempo de cómputo

Codificación de máquinas

Variantes de máquinas

2-20

Codificación de máquinas

Recordar que una máquina M con k cintas ($k \geq 3$) es una tripla (Σ, Q, δ) , donde

- Q es un conjunto finito de estados
- Σ es el alfabeto (siempre finito)
- la función de transición es

$$\delta : Q \times \Sigma^{k-1} \rightarrow \underbrace{\{L, R, S\}}_{\text{entrada}} \times \underbrace{\Sigma^{k-2} \times \{L, R, S\}^{k-2}}_{\text{trabajo}} \times \underbrace{(\Sigma \cup \{S\})}_{\text{salida}} \times Q$$

Queremos representar máquinas con palabras en $\{0, 1\}^*$.

- nos restringimos al alfabeto estándar $\Sigma = \{0, 1, \triangleright, \square\}$ pero se puede hacer con cualquier alfabeto.

2-21

Codificación de máquinas

Codificación de estados, movimientos y símbolos

Numeramos los estados de Q desde 0 hasta $|Q| - 1$ y representamos cada estado n con $[n]$

- reservamos el 0 para q_0
- reservamos el 1 para q_f

Codificamos cada símbolo de $\Sigma \cup \{L, R, S\}$ con

- | | | |
|---------------|----------------------------|---------------|
| • $[0] = 000$ | • $[\square] = 010$ | • $[L] = 100$ |
| • $[1] = 001$ | • $[\triangleright] = 011$ | • $[R] = 101$ |
| | | • $[S] = 110$ |

2-22

Codificación de máquinas

Codificación de δ

Codificamos una tupla

$$\vec{v} = (E, t_1, \dots, t_{k-2}, T_1, \dots, T_{k-2}, Z, q) \in$$
$$\underbrace{\{L, R, S\}}_{\text{entrada}} \times \underbrace{\Sigma^{k-2} \times \{L, R, S\}^{k-2}}_{\text{trabajo}} \times \underbrace{(\Sigma \cup \{S\})}_{\text{salida}} \times Q$$

con

$$\langle \vec{v} \rangle = \langle [E], [t_1], \dots, [t_{k-2}], [T_1], \dots, [T_{k-2}], [Z], [q] \rangle \in \{0, 1\}^*$$

Codificamos

$$\delta : Q \times \Sigma^{k-1} \rightarrow \underbrace{\{L, R, S\}}_{\text{entrada}} \times \underbrace{\Sigma^{k-2} \times \{L, R, S\}^{k-2}}_{\text{trabajo}} \times \underbrace{(\Sigma \cup \{S\})}_{\text{salida}} \times Q$$

con $\langle \delta \rangle =$

$$\langle \{ \langle [q], [a_1], \dots, [a_{k-1}], \langle \delta(q, a_1, \dots, a_{k-1}) \rangle \rangle : q \in Q, a_1, \dots, a_{k-1} \in \Sigma \} \rangle$$
$$\in \{0, 1\}^*$$

2-23

Máquinas $\longleftrightarrow$ palabras en binario

Codificamos $M = (\Sigma, Q, \delta)$ con k cintas ($k \geq 3$) con

$$\langle M \rangle = \langle [|Q|], [k], \langle \delta \rangle \rangle \in \{0, 1\}^*$$

- toda palabra $x \in \{0, 1\}^*$ representa alguna máquina
 - si x no respeta el patrón $\langle M \rangle$ para alguna $M = (\Sigma, Q, \delta)$, entonces suponemos que representa una máquina trivial fija cualquiera (por ejemplo una que termina ni bien empieza y devuelve la palabra vacía)
- identificamos máquinas con palabras $x \in \{0, 1\}^*$; hablamos de ‘la máquina x ’ o ‘la x -ésima máquina’ para referirnos a la única máquina M tal que $\langle M \rangle = x$
- toda máquina representa una función parcial
- para toda función parcial existen infinitas máquinas que la computan
- toda función parcial se codifica con infinitas palabras

2-24

Numerables máquinas, numerables funciones computables

- hay una cantidad numerable de máquinas
- hay una cantidad numerable de funciones $\{0, 1\}^* \rightarrow \{0, 1\}$ computables
- hay una cantidad no numerable de funciones $\{0, 1\}^* \rightarrow \{0, 1\}$
- por lo tanto, debe haber funciones $\{0, 1\}^* \rightarrow \{0, 1\}$ no computables
- ¿qué ejemplo concreto conocemos?

2-25

Variantes de máquinas

Clase 2

Funciones computables

Tiempo de cómputo

Codificación de máquinas

Variantes de máquinas

2-26

Máquinas sobre alfabetos no estándar

Proposición

Sea Γ un alfabeto. Si f es computable en tiempo $T(n)$ por una máquina $M = (\Gamma, Q, \delta)$, entonces f es computable en tiempo $O(\log |\Gamma| \cdot T(n))$ por una máquina $M' = (\Sigma, Q', \delta')$ donde $\Sigma = \{0, 1, \triangleright, \square\}$.

Idea de la prueba.

Podemos codificar cada símbolo de Γ en binario usando $\lceil \log |\Gamma| \rceil$ celdas. □

2-27

Máquinas de cinta única

Máquinas de cinta única: tienen una sola cinta con una cabeza de lectura y escritura. Para estas máquinas, la función de transición es

$$\delta : Q \times \Sigma \rightarrow \Sigma \times \{L, R, S\} \times Q$$

(con restricciones para no pasarse del comienzo de cinta).

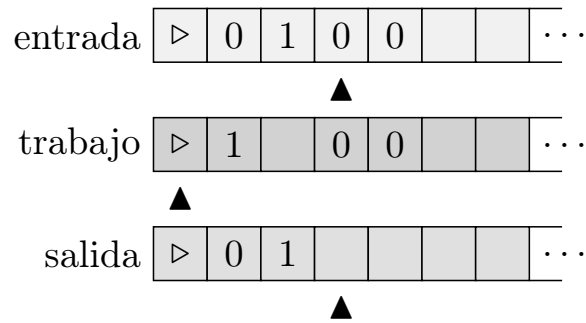
Proposición

Si f es computable en tiempo $T(n)$ por una máquina estándar de $k \geq 3$ cintas (entrada, salida y $k - 2$ cintas de trabajo), entonces f es computable en tiempo $O(T(n)^2)$ por una máquina de cinta única.

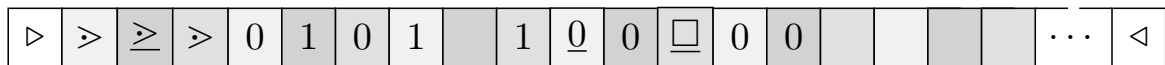
2-28

Idea de la prueba.

Codificamos las cintas de la máquina M que computa f (alfabeto estándar $\Sigma = \{0, 1, \triangleright, \square\}$)



con k cintas en una máquina M' de cinta única con alfabeto $\Gamma = \{\triangleright, 0, 1, \square, \geq, \underline{0}, \underline{1}, \underline{\square}, \triangleleft\}$.



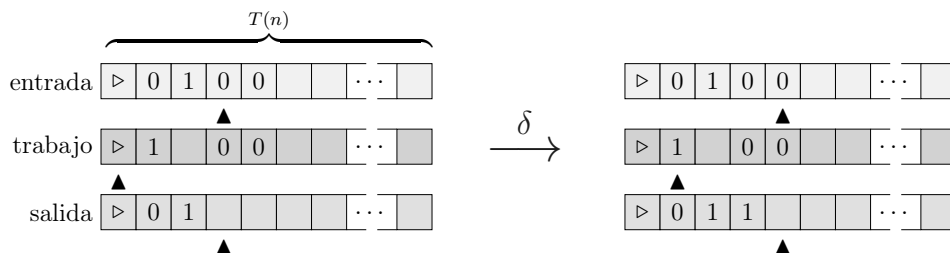
Símbolo subrayado indica la posición de la cabeza.

M recorre en tiempo $T(n)$, entonces a lo sumo usa $T(n)$ celdas de cada una de sus cintas.

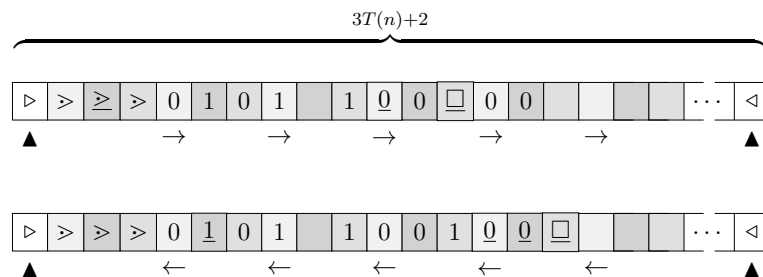
M' codifica toda la información en las primeras $k \cdot T(n)$ celdas. No usa más que eso.

2-29

Cada vez que M hace esto:



M' barre de izquierda a derecha y de derecha a izquierda.



□

2-30

M' con entrada x hace esto:

- Obtiene $|x| = n$ y calcula $T(n)$. Mueve la cabeza hasta la posición $k \cdot T(n) + 1$ y escribe $\triangleleft$. Esto le va a servir para marcar la parte de la cinta única que va a usar. Luego vuelve a la primera posición.
- Codifica la configuración inicial de M con entrada x en su cinta (esto incluye a x) usando exactamente $k \cdot T(n)$ celdas.
- Maneja el estado de M internamente, sin usar la cinta
- Repite lo siguiente hasta que M llegue a el estado final:
 - Barre la cinta de izquierda a derecha obteniendo los símbolos leídos por cada cabeza (subrayados)
 - Dependiendo de qué hace la función de transición de M con esa información, elige qué hacer con cada cabeza.
 - Barre la cinta de derecha a izquierda volcando esa nueva información (cambia a lo sumo 1 símbolo por cada cinta, las posiciones de las cabezas y el estado).
- Borra todo lo que hay en su cinta salvo la información de la salida de $M(x)$

M' corre en tiempo $O(T(n)^2)$ y calcula f . Luego simular M' con otra máquina sobre el alfabeto estándar. $\square$

2-31

Máquinas *oblivious*

Oblivious = que no se da cuenta, ajena, inconsciente

Definición

Una máquina M es ***oblivious*** si para cada entrada x y para cada $i \in \mathbb{N}$,

- la posición de las cabezas de las cintas de entrada y trabajo en el i -ésimo paso del cómputo de M con entrada x solo depende de i y de $|x|$ (pero no de x), y
- las funciones que computan esas posiciones a partir de $i, |x|$ son computables en tiempo polinomial.

Proposición

Si f es computable en tiempo $T(n)$ por una máquina estándar entonces hay una máquina *oblivious* que computa f en tiempo $O(T(n)^2)$.

Demostración.

La idea de la prueba anterior: M' es *oblivious*. $\square$

2-32

Máquinas con cintas bi-infinitas

Máquinas con cintas bi-infinitas: tienen cintas infinitas en ambas direcciones en lugar de ser infinitas a derecha.

Proposición

Si f es computable por una máquina con cintas bi-infinitas en tiempo $T(n)$, entonces f es computable por una máquina estándar en tiempo $O(T(n))$.

Idea de la prueba.

Podemos ‘plegar’ cada cinta: convertir

...

		0		0	1	1	1		0		0	0	...
--	--	---	--	---	---	---	---	--	---	--	---	---	-----

en

▷	1	1	1		0	0		0	0		1	...
---	---	---	---	--	---	---	--	---	---	--	---	-----

sobre un alfabeto $\{\triangleright\} \cup \{0, 1, \square\}^2$. Según qué porción de la cinta bi-infinita esté procesando, usar primer o segundo símbolo de 00, 01, 10, 11. Luego, traducir al alfabeto estándar. □

2-33

Clase 3

Clase 1

- Preliminares
- Representaciones en binario
- Problemas de decisión
- Máquinas de Turing
- Configuraciones y cálculos

Clase 2

- Funciones computables
- Tiempo de cómputo
- Codificación de máquinas
- Variantes de máquinas

Clase 3

- Halt y máquina universal
- La clase **P**
- La clase **NP**

Clase 4

- Máquinas no-determinísticas
- Reducibilidad polinomial
- Lógica proposicional

Clase 5

- Mini-configuraciones
- Teorema de Cook-Levin
- Problemas **NP-completos**
- La clase **coNP**
- Las clases **EXPTIME** y **NEXPTIME**

Clase 6

- La jerarquía de tiempos determinísticos
- La jerarquía de tiempos no-determinísticos
- Teorema de Ladner

Clase 7

- Espacio usado por un cómputo
- Espacio polinomial: **PSPACE** y **NPSPACE**
- Teorema de Savitch

Clase 8

- Espacio logarítmico: **L** y **NL**
- Teorema de Immerman-Szelepcsényi

Clase 9

- La jerarquía polinomial
- Problemas Σ_i^P -completos

Clase 10

- Máquinas con oráculo
- Teorema de Baker, Gill, Solovay
- La jerarquía polinomial y **NP** con oráculos

Clase 11

- Circuitos booleanos
- La clase **P/poly**
- Tamaño y profundidad de circuitos

Clase 12

- Las clases **NC_{nu}** y **AC_{nu}**
- Uniformidad: las clases **NC** y **AC**
- P-completitud**

Halt y máquina universal

Clase 3

Halt y máquina universal

La classe P

La clase NP

3-2

El problema de la detención (*halting problem*)

Definimos $halt : \{0, 1\}^* \rightarrow \{0, 1\}$ como

$$halt(x) = \begin{cases} 1 & \text{si la máquina } x \text{ con entrada } x \text{ termina} \\ 0 & \text{si no} \end{cases}$$

Teorema (Turing 1936)

halt no es computable.

Demostración.

Supongamos que *halt* es computable. Definimos una máquina M tal que $M(x)$ termina sii $halt(x) = 0$. Sea $y = \langle M \rangle$.

$M(y)$ termina sii la máquina y con entrada y no termina
 sii $M(y)$ no termina

Absurdo.

☐

La máquina universal

Llamemos M_i a la máquina tal que $\langle M \rangle = i$. Definimos

$$u : \subseteq \{0, 1\}^* \rightarrow \{0, 1\}^*$$

$$u(\langle i, x \rangle) = M_i(x)$$

La función u es parcial. En particular $u(\langle i, x \rangle) \downarrow$ sii $M_i(x) \downarrow$.

Teorema

Existe una máquina U que computa la función $u(\langle i, x \rangle) = M_i(x)$. Más aún, si M_i con entrada x termina en t pasos, entonces U con entrada $\langle i, x \rangle$ termina en $c \cdot t \cdot \log t$ pasos, donde c depende solo de i .

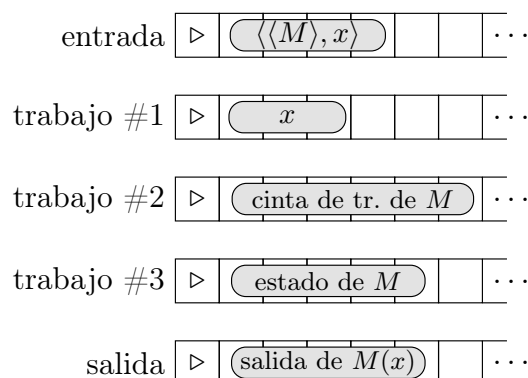
3-4

Idea de la prueba (un caso simple).

U tiene 3 cintas de trabajo.

Supongamos $M = (\Sigma, Q, \delta)$ con *una sola* cinta de trabajo.

- U recibe como entrada $\langle \langle M \rangle, x \rangle$. Objetivo: copiar $M(x)$
- $\langle M \rangle$ tiene la información sobre el comportamiento de M
- U copia x en la cinta de trabajo #1
- U escribe $[q_0]$ en la cinta de trabajo #3
- mientras que lo que esté escrito en en la cinta #3 no sea $[q_f]$, busca en la cinta de entrada la información sobre δ que necesita y actualiza las cintas #2 y #3 de manera acorde



3-5

Si $M = (\Sigma, Q, \delta)$ con entrada x termina en t pasos, esta simulación usa $c \cdot t$ pasos, donde c depende de M pero no de x

- U tiene que buscar la definición de δ en la cinta de entrada por cada instrucción que simula de M

Pero esto es una simplificación del caso general:

- U está simulando una M con 1 sola cinta de trabajo
- si M tiene más cintas de trabajo, se puede transformar en una M' que usa *una sola* cinta con el mismo comportamiento de M
- si M llegaba al resultado en tiempo t , M' lo hace en tiempo $O(t^2)$. Esto es demasiado para la cota $O(t \log t)$ del teorema.
- la demostración del caso general es más técnica

□

3-6

La máquina universal con tiempo acotado

Llamemos M_i a la máquina tal que $\langle M \rangle = i$.

$$\tilde{u} : \{0, 1\}^* \rightarrow \{0, 1\}^*$$

$$\tilde{u}(\langle i, t, x \rangle) = \begin{cases} 1 & M_i(x) \text{ termina en } \leq t \text{ pasos} \\ 0 & \text{si no} \end{cases}$$

La función $\tilde{u}$ es total.

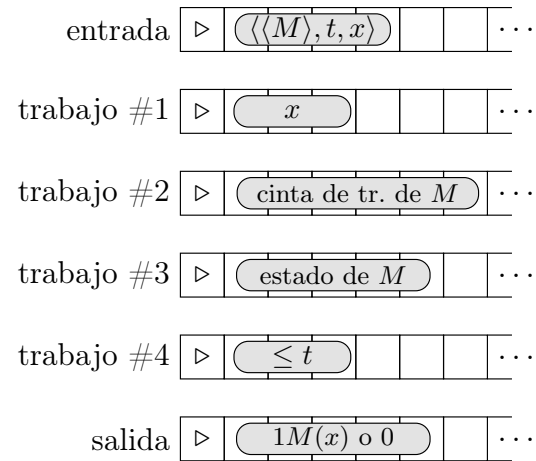
Teorema

Existe una máquina $\tilde{U}$ que computa la función $\tilde{u}(\langle i, t, x \rangle)$ en tiempo $c \cdot t \cdot \log t$, donde c depende solo de i .

3-7

Idea de la prueba.

Como antes, pero ahora $\tilde{U}$ tiene una cinta de trabajo más, que usa para llevar cuenta de la cantidad de pasos en la simulación



3-8

La clase **P**

Clase 3

Halt y máquina universal

La clase **P**

La clase **NP**

3-9

Clases de complejidad y la clase $\mathbf{DTIME}(T(n))$

Sea $\mathcal{L} \subseteq \{0, 1\}^*$ un lenguaje (o problema) y sea Σ el alfabeto estándar.

Definición

Una máquina $M = (Q, \Sigma, \delta)$ **decide** $\mathcal{L}$ [en tiempo $T(n)$] si M computa $\chi_{\mathcal{L}}$ [en tiempo $T(n)$]. Decimos que M **acepta** x cuando $M(x) = 1$ y que M **rechaza** x cuando $M(x) = 0$.

Notación: $\mathcal{L}(M)$

El lenguaje decidido por M es $\mathcal{L}(M)$.

Una **clase de complejidad** es un conjunto de problemas que son decidibles por máquinas con ciertos recursos acotados. Por ahora, nos centramos en el recurso *tiempo de cómputo*.

Clase de complejidad: $\mathbf{DTIME}(T(n))$

$\mathbf{DTIME}(T(n))$ es la clase de lenguajes $\mathcal{L}$ tal que existe una máquina M que decide $\mathcal{L}$ en tiempo $O(T(n))$.

3-10

La clase $\mathbf{P}$

Clase de complejidad: $\mathbf{P}$

$$\mathbf{P} = \bigcup_{c>0} \mathbf{DTIME}(n^c)$$

Son los problemas que se resuelven por una máquina en tiempo polinomial respecto al tamaño de su entrada.

Observación

Por ahora, una ‘máquina’ $M = (\Sigma, Q, \delta)$ es un dispositivo *determinístico*. De ahí la ‘D’ en $\mathbf{DTIME}$. Más adelante veremos máquinas *no-determinísticas*.

Decimos “el problema X está en $\mathbf{P}$ ” o “el problema X se resuelve en tiempo polinomial” para referirnos a que el lenguaje de decisión que corresponde a X está en $\mathbf{P}$.

$\mathbf{P}$ es considerada la clase de problemas “factibles”.

3-11

Ejemplo de problema en $\mathbf{P}$

Un grafo $G = (V, E)$ es *conexo* si todo par de nodos de G está unido por un camino.

Problema: Conectividad de un grafo

$$\text{CON} = \{\langle G \rangle : G \text{ es un grafo conexo}\}$$

- Representamos $G = (V, E)$ con su matriz de adyacencia
- Suponemos $V = \{0, \dots, k-1\}$
- Hay una máquina M que hace esto:

Explora G usando depth first search desde u y marca los nodos visitados. Si quedó un nodo sin visitar, escribe 0 en la salida; si no, escribe un 1. Luego termina (que quiere decir pasar a q_f).

- Si $n = |\langle G \rangle|$ (siempre n va a representar el tamaño de la entrada), M llega al resultado en tiempo $O(n^2)$, entonces $\text{CON} \in \mathbf{P}$.

3-12

Simplificación de máquinas para problemas de decisión

Para calcular funciones valuadas en $\{0, 1\}$, no hace falta una cinta de salida (porque solo hay que devolver un 0 o un 1). Podemos trabajar con máquinas que solo tienen una cinta de entrada, una trabajo y

- dejar la salida (0 o 1) en la celda en la que termina la cabeza de trabajo cuando entra a q_f , o bien
- reemplazar q_f por un $q_{\text{sí}}$ (que significa que devuelve 1) y un q_{no} (que significa que devuelve 0); el estado de la configuración final es ahora $q_{\text{sí}}$ o bien q_{no} .

Llamemos a cualquiera de estas variantes una **máquina sin cinta de salida**.

Proposición

Si $\mathcal{L}$ es decidible en tiempo $T(n)$ por una máquina estándar de 3 cintas (entrada, trabajo y salida), entonces $\mathcal{L}$ es decidible en tiempo $O(T(n))$ por una máquina sin cinta de salida.

3-13

La clase **NP**

Clase 3

Halt y máquina universal

La clase **P**

La clase **NP**

3-14

La clase **NP**

Clase de complejidad: **NP**

NP es la clase de lenguajes $\mathcal{L}$ tal que existe un polinomio $p : \mathbb{N} \rightarrow \mathbb{N}$ y una máquina M tal que

- M corre en tiempo polinomial
- para todo x :

$$x \in \mathcal{L} \quad \text{sii} \quad \text{existe } u \in \{0, 1\}^{p(|x|)} \text{ tal que } M(\langle x, u \rangle) = 1$$

M se llama el **verificador para $\mathcal{L}$** ; u se llama **certificado para x** .

Notar que la segunda condición es equivalente a

$$x \in \mathcal{L} \quad \text{sii} \quad \text{existe } u \in \{0, 1\}^{p(|x|)} \text{ tal que } M(xu) = 1$$

porque $M(y)$ puede primero contar la cantidad m de celdas en la cinta de entrada hasta el primer blanco y luego buscar el primer n tal que $m = n + p(n)$. Tenemos que

$$y(0), \dots, y(n-1) = x \quad \text{y} \quad y(n), \dots, y(n+p(n)-1) = u$$

3-15

Teorema

$\mathbf{P} \subseteq \mathbf{NP}$.

Demostración.

Supongamos que $\mathcal{L} \in \mathbf{P}$. Supongamos una máquina M tal que M decide $\mathcal{L}$ en tiempo polinomial. Tomamos $p(n) = 0$.

Definimos la máquina M' que, dado $\langle x, \epsilon \rangle$ como entrada, simula M con entrada x . Como M corre en tiempo polinomial, M' también.

- $x \in \mathcal{L}$ sii $M(x) = 1$
- sii $M'(\langle x, \epsilon \rangle) = 1$
- sii existe $u \in \{0, 1\}^0$ tal que $M'(\langle x, u \rangle) = 1$

□

3-16

Ejemplo de problema en $\mathbf{NP}$

Un conjunto X de nodos de un grafo $G = (V, E)$ es **independiente** si no existen $u, v \in X$ tal que $(u, v) \in E$.

Problema: Conjunto independiente

$$\text{INDSET} = \{ \langle G, k \rangle \mid \begin{array}{l} G \text{ tiene un conjunto inde-} \\ \text{pendiente de } \geq k \text{ vértices} \end{array} \}$$

- suponemos que $V = \{[0], [1], \dots, [|V| - 1]\}$
- certificado: lista de k nodos distintos de V que forman un conjunto independiente
- podemos codificar ese certificado en una palabra u de tamaño $O(k \lceil \log |V| \rceil)$. Sea $n = |\langle G, k \rangle|$. Como $k \leq |V|$, entonces $|u| = O(n \log n)$ (usar codificación de listas)
- $|u| = p(n)$ para un polinomio cuadrático p (completar con 0s al final)
- M recibe como entrada x . Si x no es de la forma $\langle \langle G, k \rangle, u \rangle$ con $|u| = p(n)$, escribe 0 en la salida. Si no: si u codifica un conjunto independiente de G de k nodos, escribe 1 en la salida; si no, escribe 0.
- M corre en tiempo polinomial.
- $x \in \text{INDSET}$ sii existe $u \in \{0, 1\}^{p(|x|)}$ tal que $M(\langle x, u \rangle) = 1$.

3-17

Clase 4

Clase 1

Preliminares
Representaciones en binario
Problemas de decisión
Máquinas de Turing
Configuraciones y cálculos

Clase 2

Funciones computables
Tiempo de cómputo
Codificación de máquinas
Variantes de máquinas

Clase 3

Halt y máquina universal
La clase **P**
La clase **NP**

Clase 4

Máquinas no-determinísticas
Reducibilidad polinomial
Lógica proposicional

Clase 5

Mini-configuraciones
Teorema de Cook-Levin
Problemas **NP-completos**
La clase **coNP**
Las clases **EXPTIME** y **NEXPTIME**

Clase 6

La jerarquía de tiempos determinísticos
La jerarquía de tiempos no-determinísticos
Teorema de Ladner

Clase 7

Espacio usado por un cómputo
Espacio polinomial: **PSPACE** y **NPSPACE**
Teorema de Savitch

Clase 8

Espacio logarítmico: **L** y **NL**
Teorema de Immerman-Szelepcsényi

Clase 9

La jerarquía polinomial
Problemas **Σ_i^P -completos**

Clase 10

Máquinas con oráculo
Teorema de Baker, Gill, Solovay
La jerarquía polinomial y **NP** con oráculos

Clase 11

Circuitos booleanos
La clase **P/poly**
Tamaño y profundidad de circuitos

Clase 12

Las clases **NC_{nu}** y **AC_{nu}**
Uniformidad: las clases **NC** y **AC**
P-completitud

4-1

Máquinas no-determinísticas

Clase 4

Máquinas no-determinísticas
Reducibilidad polinomial
Lógica proposicional

4-2

Máquinas determinísticas

Las máquinas que vimos hasta ahora son *determinísticas*: cada configuración evoluciona de una única forma a partir de la configuración inicial.

Un cómputo de $M = (\Sigma, Q, \delta)$ a partir de x es

$$C_0 \xrightarrow{\delta} C_1 \xrightarrow{\delta} C_2 \xrightarrow{\delta} \dots \xrightarrow{\delta} C_\ell$$

tal que C_ℓ es final.

A partir de ahora, las llamamos también **máquinas determinísticas**.

Notación: Máquina determinística
Máquina = máquina determinística

4-3

Máquinas no-determinísticas

Las máquinas **no-determinísticas** son triplas (Σ, Q, δ) , como antes, pero con dos diferencias respecto a las determinísticas:

1. la función de transición es

$\delta : Q \times \Sigma^{k-1} \rightarrow \left(\underbrace{\{L, R, S\}}_{\text{entrada}} \times \underbrace{\Sigma^{k-2} \times \{L, R, S\}^{k-2}}_{\text{trabajo}} \times \underbrace{(\Sigma \cup \{S\})}_{\text{salida}} \times Q \right)^2$

Ahora δ especifica una o dos posibles **evoluciones en un paso** a partir de una configuración dada.

2. Q tiene 3 estados distinguidos:

q_0 , $q_{\text{sí}}$, q_{no}

4-4

Cómputo de una máquina no-determinística

Definición

Una **configuración final** de una máquina no-determinística $N = (\Sigma, Q, \delta)$ es una en la que el estado es $q_{\text{sí}}$ o q_{no} . Un **cómputo** de N a partir de $x \in \{0, 1\}^*$ es una secuencia $C_0, \dots, C_\ell$ de configuraciones tal que

1. C_0 es inicial a partir de x
2. C_{i+1} es la evolución de C_i en un paso dado por alguna de las 2 tuplas de δ
3. C_ℓ está en estado $q_{\text{sí}}$ o q_{no}

Un **cómputo aceptador** es uno en el que C_ℓ está en estado $q_{\text{sí}}$, y en ese caso a C_ℓ lo llamamos configuración **aceptadora**. Llamamos **longitud** del cómputo a ℓ .

A diferencia de las máquinas determinísticas, hay posiblemente muchos cómputos a partir de una configuración dada.

4-5

Lenguaje aceptado por una máquina no-determinística

Definición

Decimos que la máquina no-determinística N **acepta** x si existe un cómputo aceptador $C_0, \dots, C_\ell$ de N a partir de x . En caso contrario decimos que N **rechaza** x .

Decimos que N **decide** el lenguaje $\mathcal{L}$ si para todo $x \in \{0, 1\}^*$,

$$x \in \mathcal{L} \quad \text{sii} \quad N \text{ acepta } x$$

Observación

- Las máquinas no-determinísticas no computan funciones; solo aceptan lenguajes (que es una forma de computar funciones valuadas en $\{0, 1\}$).
- En las máquinas no-determinísticas la cinta de salida es irrelevante (porque no participa de ninguna definición).

4-6

Lenguaje aceptado por una máquina no-determinística

Notación: $N(x), \mathcal{L}(N)$

Sea N una máquina no-determinística. $N(x) = 1$ si N acepta x ; $N(x) = 0$ si N rechaza x .

El lenguaje decidido por N es $\mathcal{L}(N)$.

4-7

Tiempo de cómputo de una máquina no-determinística

Definición

La máquina no-determinística N **corre en tiempo $T(n)$** si para todo $x \in \{0, 1\}^*$, *todo* cómputo de N a partir de x tiene longitud $\leq T(|x|)$.

Definición

La máquina no-determinística N **corre en tiempo $O(T(n))$** si existe una constante c tal que para todo $x \in \{0, 1\}^*$, salvo finitos, *todo* cómputo de N a partir de x tiene longitud $\leq c \cdot T(|x|)$.

4-8

La clase $\mathbf{NDTIME}(T(n))$

Clase de complejidad: $\mathbf{NDTIME}(T(n))$

$\mathbf{NDTIME}(T(n))$ es la clase de lenguajes $\mathcal{L}$ tal que existe una máquina no-determinística N tal que

- N decide $\mathcal{L}$
- N corre en tiempo $O(T(n))$

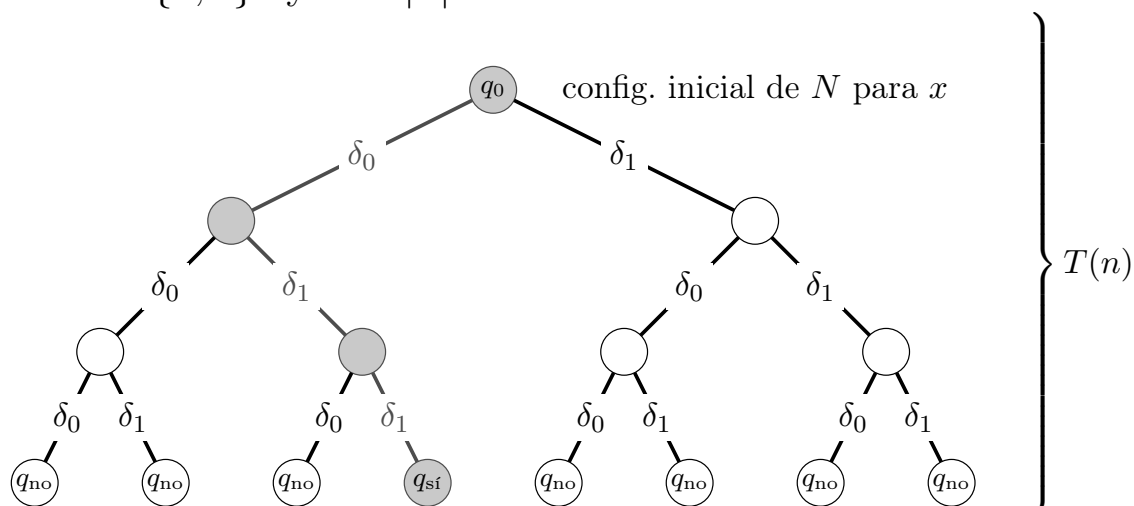
4-9

Cómpuotos en máquinas no-determinísticas

Supongamos la máquina no-determinística $N = (\Sigma, Q, \delta)$ que corre en tiempo $T(n)$. Dada $\delta(q, s) = (a, b)$ definamos

$$\delta_0(q, s) = a \quad \text{y} \quad \delta_1(q, s) = b.$$

Sea $x \in \{0, 1\}^*$ y $n = |x|$



N acepta x porque existe un cómputo aceptador

Codificamos cómputos con secuencias $\{0, 1\}^{T(n)}$.

011 es un cómputo aceptador de $N(x)$.

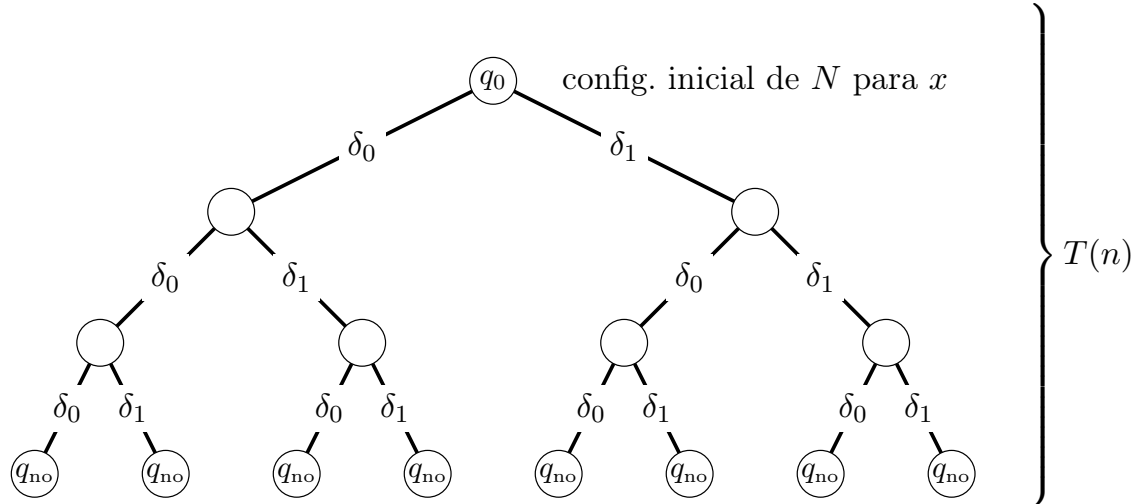
4-10

Cómputos en máquinas no-determinísticas

Supongamos la máquina no-determinística $N = (\Sigma, Q, \delta)$ que corre en tiempo $T(n)$. Dada $\delta(q, s) = (a, b)$ definamos

$$\delta_0(q, s) = a \quad \text{y} \quad \delta_1(q, s) = b.$$

Sea $x \in \{0, 1\}^*$ y $n = |x|$



N rechaza x porque no existe un cómputo aceptador

Codificamos cómputos con secuencias $\{0, 1\}^{T(n)}$.

011 no es un cómputo aceptador de $N(x)$.

4-11

Simulación de una máquina no-determinística

- Sea $N = (\Sigma, Q, \delta)$ una máquina no-determinística
- Podemos simular determinísticamente *todos* los cómputos de longitud a lo sumo t de N a partir de cualquier x y determinar cuáles de esos son aceptadores.
- A cada cómputo lo codificamos con una palabra de $u \in \{0, 1\}^t$
- Hay 2^t tales cómputos
- Para u , simulamos N como si fuera determinística siguiendo el cómputo u (toma tiempo $O(t \cdot 2^t)$)
- Si N corre en tiempo $T(n)$, podemos decidir si N acepta o rechaza x con una máquina determinística que corre en tiempo $O(2^{T(n)^2})$

Definición alternativa de NP

Teorema

$$\mathbf{NP} = \bigcup_{c \in \mathbb{N}} \mathbf{NDTIME}(n^c).$$

Existe un polinomio $p : \mathbb{N} \rightarrow \mathbb{N}$ y una máquina determinística M que corre en tiempo polinomial tal que para todo x ,

$$x \in \mathcal{L}$$

sii

existe $u \in \{0, 1\}^{p(|x|)}$
tal que $M(\langle x, u \rangle) = 1$

$\iff$

Existe una máquina no-determinística N que corre en tiempo polinomial tal que

$$x \in \mathcal{L}$$

sii

existe un

cómputo aceptador
de N a partir de x

4-13

Prueba de $\mathbf{NP} \supseteq \bigcup_{c \in \mathbb{N}} \mathbf{NDTIME}(n^c)$.

Supongamos una máquina no-determinística $N = (\Sigma, Q, \delta)$ y un polinomio p tal que N corre en tiempo p y N decide $\mathcal{L}$.

$x \in \mathcal{L}$ sii existe un cómputo aceptador $C_0, \dots, C_{p(|x|)}$
de N a partir de x .

Ya vimos que podemos codificar cómputos con cadenas binarias. Consideremos $u \in \{0, 1\}^{p(|x|)}$ tal que $u(i) = 0$ si se usa la primera componente de δ para pasar de C_i a C_{i+1} y $u(i) = 1$ en caso contrario.

Existe una máquina determinística M (verificador) tal que M con entrada $\langle x, u \rangle$ verifica que u (certificado) sea la codificación de un cómputo aceptador de N a partir de x . Si lo es, escribe 1 en la salida y si no escribe 0.

$x \in \mathcal{L}$ sii existe $u \in \{0, 1\}^{p(|x|)}$ tal que $M(\langle x, u \rangle) = 1$.

M corre en tiempo polinomial, por lo tanto $\mathcal{L} \in \mathbf{NP}$. □

4-14

Prueba de $\mathbf{NP} \subseteq \bigcup_{c \in \mathbb{N}} \mathbf{NDTIME}(n^c)$.

Supongamos una máquina determinística M que corre en tiempo polinomial y un polinomio $p : \mathbb{N} \rightarrow \mathbb{N}$ tal que para todo x ,

$$x \in \mathcal{L} \quad \text{sii} \quad \text{existe } u \in \{0, 1\}^{p(|x|)} \text{ tal que } M(\langle x, u \rangle) = 1$$

Definimos una máquina no-determinística N que con entrada x hace esto:

“inventa” una palabra $u \in \{0, 1\}^{p(|x|)}$ en su cinta de trabajo; lo logra con el no-determinismo de δ , que escribe 0 o 1. Esto le toma tiempo $p(|x|)$.

simula M con entrada $\langle x, u \rangle$. Esto le toma tiempo polinomial.

si $M(\langle x, u \rangle) = 1$ entonces entra al estado $q_{\text{sí}}$; si no entra al estado q_{no}

Tenemos

$$x \in \mathcal{L} \quad \text{sii} \quad \text{existe un cómputo aceptador de } N \text{ a partir de } x.$$

N corre en tiempo polinomial, entonces $\mathcal{L} \in \bigcup_{c \in \mathbb{N}} \mathbf{NDTIME}(n^c)$. $\square$

4-15

Máquinas no-determinísticas $\longleftrightarrow$ palabras en binario

- análogamente a lo que pasa para las máquinas determinísticas, hay una codificación de máquinas no-determinísticas con palabras en $\{0, 1\}^*$
- toda palabra $x \in \{0, 1\}^*$ representa alguna máquina no-determinística
- identificamos máquinas no-determinísticas con palabras $x \in \{0, 1\}^*$; hablamos de ‘la máquina no-determinística x ’ o ‘la x -ésima máquina no-determinística’ para referirnos a la única máquina no-determinística N tal que $\langle N \rangle = x$
- hay una cantidad numerable de máquinas no-determinísticas

4-16

La máquina no-determinística universal

Llamemos N_i a la máquina no-determinística tal que $\langle N \rangle = i$.

Teorema

Existe una máquina no-determinística NU que tal que NU acepta $(\langle i, x \rangle)$ sii N_i acepta x y si N_i corre en tiempo $T(n)$ entonces $NU(\langle i, x \rangle)$ decide si N_i acepta o rechaza x en $c \cdot T(|x|)$ pasos, donde c depende solo de i .

4-17

Reducibilidad polinomial

Clase 4

Máquinas no-determinísticas

Reducibilidad polinomial

Lógica proposicional

4-18

Reducción polinomial

Definición

$\mathcal{L} \subseteq \{0, 1\}^*$ es **Karp reducible polinomialmente** o simplemente **reducible polinomialmente** a $\mathcal{L}' \subseteq \{0, 1\}^*$, notado $\mathcal{L} \leq_p \mathcal{L}'$, si existe una función computable en tiempo polinomial $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ tal que para todo $x \in \{0, 1\}^*$

$$x \in \mathcal{L} \quad \text{sii} \quad f(x) \in \mathcal{L}'.$$

f se llama **reducción polinomial** de $\mathcal{L}$ a $\mathcal{L}'$. En este caso decimos que $\mathcal{L} \leq_p \mathcal{L}'$ vía f .

Clase de complejidad: NP-hard, NP-completo

$\mathcal{L}$ es **NP-hard** si $\mathcal{L}' \leq_p \mathcal{L}$ para todo $\mathcal{L}' \in \text{NP}$

$\mathcal{L}$ es **NP-completo** si $\mathcal{L} \in \text{NP}$ y $\mathcal{L}$ es NP-hard.

Ejercicio

Si $\mathcal{L} \leq_p \mathcal{L}'$ y $\mathcal{L}' \in \text{P}$, entonces $\mathcal{L} \in \text{P}$.

4-19

Teorema

La relación $\leq_p$ es transitiva.

Demostración.

Supongamos que $\mathcal{L} \leq_p \mathcal{L}'$ via f y $\mathcal{L}' \leq_p \mathcal{L}''$ via g .

$$x \in \mathcal{L} \quad \text{sii} \quad f(x) \in \mathcal{L}' \quad \text{sii} \quad g(f(x)) \in \mathcal{L}''$$

Veamos que $g \circ f$ es computable en tiempo polinomial.

Supongamos M_f (det.) que computa f en tiempo $O(n^c)$

Supongamos M_g (det.) que computa g en tiempo $O(n^d)$

Definimos M (det.) así: simula M_f con entrada x ; si obtiene $M_f(x) = f(x) = y$, entonces simula $M_g(y)$ y escribe el resultado en la cinta de salida.

- $M(x) = g(f(x))$, o sea M computa $g \circ f$
- $|y| = O(|x|^c)$
- M corre en tiempo $O(n^{cd})$

Entonces $\mathcal{L} \leq_p \mathcal{L}''$ vía $g \circ f$. □

4-20

Teorema

Si $\mathbf{NP-hard} \cap \mathbf{P} \neq \emptyset$, entonces $\mathbf{P} = \mathbf{NP}$.

Demostración.

Sea $\mathcal{L} \in \mathbf{NP-hard} \cap \mathbf{P}$.

Como $\mathcal{L} \in \mathbf{NP-hard}$, tenemos $\mathcal{L}' \leq_p \mathcal{L}$ para todo $\mathcal{L}' \in \mathbf{NP}$.

Fijemos $\mathcal{L}' \in \mathbf{NP}$ cualquiera y veamos que $\mathcal{L}' \in \mathbf{P}$.

Existe una función f computable en tiempo polinomial tal que para todo x

$$x \in \mathcal{L}' \quad \text{sii} \quad f(x) \in \mathcal{L}.$$

Como $\mathcal{L} \in \mathbf{P}$, $\chi_{\mathcal{L}}$ es computable en tiempo polinomial.

Entonces

$$x \in \mathcal{L}' \quad \text{sii} \quad \chi_{\mathcal{L}}(f(x)) = 1$$

y $\chi_{\mathcal{L}} \circ f$ es computable en tiempo polinomial.

Luego $\chi_{\mathcal{L}'}$ es computable en tiempo polinomial, o sea $\mathcal{L}' \in \mathbf{P}$. □

4-21

Teorema

Si $\mathcal{L} \in \mathbf{NP-completo}$, entonces $\mathcal{L} \in \mathbf{P}$ sii $\mathbf{P} = \mathbf{NP}$.

Demostración.

Supongamos $\mathcal{L} \in \mathbf{NP-completo}$

$\Rightarrow$ Supongamos $\mathcal{L} \in \mathbf{P}$.

Por hipótesis tenemos $\mathcal{L} \in \mathbf{NP-completo}$.

Luego $\mathcal{L} \in \mathbf{NP-completo} \cap \mathbf{P}$.

Entonces $\mathcal{L} \in \mathbf{NP-hard} \cap \mathbf{P}$.

Por teorema anterior, $\mathbf{P} = \mathbf{NP}$.

$\Leftarrow$ Supongamos $\mathbf{P} = \mathbf{NP}$

Por hipótesis $\mathcal{L} \in \mathbf{NP}$, de modo que $\mathcal{L} \in \mathbf{P}$. □

4-22

Un ejemplo de problema **NP-completo**

Llamemos M_y a la máquina determinística tal que $\langle M \rangle = y$.

Problema: TMSAT

$\text{TMSAT} = \{\langle y, x, 1^n, 1^t \rangle : \exists u \in \{0, 1\}^n \ M_y(xu) = 1 \text{ en } \leq t \text{ pasos}\}$

Proposición

TMSAT $\in$ NP-completo.

Demostración.

Es claro que $\text{TMSAT} \in \mathbf{NP}$. Tomemos $\mathcal{L} \in \mathbf{NP}$ y veamos que $\mathcal{L} \leq_p \text{TMSAT}$. Existen polinomios p, q y una máquina determinística M tal que $x \in \mathcal{L}$ sii $\exists u \in \{0, 1\}^{\leq p(|x|)} \ M(xu) = 1$ y M corre en tiempo $q(n)$. Definimos $f(x) = \langle \langle M \rangle, x, 1^{p(|x|)}, 1^{q(|x|+p(|x|))} \rangle$. f es computable en tiempo polinomial.

$f(x) \in \text{TMSAT}$ sii $\langle \langle M \rangle, x, 1^{p(|x|)}, 1^{q(|x|+p(|x|))} \rangle \in \text{TMSAT}$
 sii $\exists u \in \{0, 1\}^{p(|x|)} \ M(xu) = 1$
 en $\leq q(|xu|) = q(|x| + p(|x|))$ pasos
 sii $x \in \mathcal{L}$ □

4-23

Lógica proposicional

Clase 4

Máquinas no-determinísticas

Reducibilidad polinomial

Lógica proposicional

Lógica proposicional

Una **fórmula booleana** se forma con la gramática

$$\varphi ::= p \mid \neg\varphi \mid (\varphi \wedge \varphi) \mid (\varphi \vee \varphi)$$

donde $p \in \text{PROP}$. El **tamaño** de φ es la cantidad de símbolos que contiene.

Una **valuación** es una función $v : \text{PROP} \rightarrow \{0, 1\}$. Definimos $v \models \varphi$ por inducción en φ :

- si $p \in \text{PROP}$, $v \models p$ si $v(p) = 1$
- $v \models \neg\varphi$ si no $v \models \varphi$
- $v \models \varphi \wedge \psi$ si $v \models \varphi$ y $v \models \psi$
- $v \models \varphi \vee \psi$ si $v \models \varphi$ o $v \models \psi$

Decimos que

- φ es **verdadera para** v o que v **satisface** φ si $v \models \varphi$.
- φ es **falsa para** v o que v **no satisface** φ si $v \models \varphi$ es falso, notado $v \not\models \varphi$.
- φ es **satisfacible** si existe una valuación v tal que $v \models \varphi$.
- φ es una **tautología** si $v \models \varphi$ para toda valuación v .

4-25

Valuaciones

Notación: $\varphi(p_1, \dots, p_n)$

Notamos $\varphi(p_1, \dots, p_n)$ para marcar que las variables proposicionales que aparecen en φ están entre $p_1, \dots, p_n$.

Proposición

Sea $\varphi(p_1, \dots, p_n)$ una fórmula. Si v y v' son dos valuaciones tales que $v(p_i) = v'(p_i)$ para $i \in \{1, \dots, n\}$, entonces $v \models \varphi(p_1, \dots, p_n)$ sii $v' \models \varphi(p_1, \dots, p_n)$.

Para saber si $v \models \varphi(p_1, \dots, p_n)$ alcanza con conocer $v \upharpoonright \{p_1, \dots, p_n\}$. Usamos $v \in \{0, 1\}^n$ para referirnos a $\{p_i \mapsto v(i)\}$.

Ejemplo

La valuación parcial $\{p_1 \mapsto 0, p_2 \mapsto 0, p_3 \mapsto 1\}$ se representa como $(0, 0, 1)$ o como 001.

- $(0, 0, 1) \models \neg p_1 \wedge \neg p_2 \wedge p_3$ o $001 \models \neg p_1 \wedge \neg p_2 \wedge p_3$
- $(0, 0, 1) \not\models p_1 \vee p_2$ o $001 \not\models p_1 \vee p_2$

4-26

Forma normal conjuntiva

Una fórmula está en **forma normal conjuntiva (CNF)** si es de la forma

$$\underbrace{(a_{11} \vee \cdots \vee a_{1n_1})}_{\text{cláusula}} \wedge \underbrace{(a_{21} \vee \cdots \vee a_{2n_2})}_{\text{cláusula}} \wedge \cdots \wedge \underbrace{(a_{m1} \vee \cdots \vee a_{mn_m})}_{\text{cláusula}}$$

donde a_{ij} es un **literal**, es decir, de la forma p o $\neg p$ para algún $p \in \text{PROP}$.

Una fórmula está en 3CNF si está en CNF y cada cláusula tiene a lo sumo 3 literales.

Representamos una fórmula $\varphi \in \text{CNF}$ con una palabra $\langle \varphi \rangle$ en $\{0, 1\}^*$

Problema: SAT (satisfacción booleana en CNF)

$$\text{SAT} = \{\langle \varphi \rangle : \varphi \in \text{CNF} \text{ es satisfacible}\}$$

Problema: 3SAT (satisfacción booleana en 3CNF)

$$3\text{SAT} = \{\langle \varphi \rangle : \varphi \in 3\text{CNF} \text{ es satisfacible}\}$$

4-27

SAT y 3SAT son NP

Teorema

$\text{SAT}, 3\text{SAT} \in \text{NP}$.

Demostración.

Alcanza con verlo para SAT. Definimos una máquina determinística M que recibe como entrada $\langle x, u \rangle$ y hace esto:

si x no representa una fórmula, rechazar.

si no, supongamos que $x = \langle \varphi \rangle$ y supongamos que φ tiene m variables

si $|u| < m$ rechazar

si no, aceptar si $u \upharpoonright m \models \varphi$ y rechazar en caso contrario.

Tenemos

$$\langle \varphi \rangle \in \text{SAT} \quad \text{sii} \quad \exists u \in \{0, 1\}^{|\langle \varphi \rangle|} \quad M(\langle x, u \rangle) = 1$$

Como M corre en tiempo polinomial, $\text{SAT} \in \text{NP}$.

El certificado de M es la valuación que hace verdadera a φ : es una palabra en $\{0, 1\}^{p(|\langle \varphi \rangle|)}$, donde $p(n) = n$. □

4-28

SAT y 3SAT son NP

Ejemplo

Sea

$$\varphi = (\neg p_1 \vee p_2 \vee \neg p_3) \wedge (p_1 \vee p_2 \vee p_3).$$

$u = 001$ es un certificado para φ porque

$$001 \models \varphi$$

4-29

Representación de funciones booleanas

Proposición

Para toda $F : \{0, 1\}^\ell \rightarrow \{0, 1\}$ existe una fórmula $\varphi_F(p_1, \dots, p_\ell)$ en CNF tal que $v \models \varphi_F$ si $F(v) = 1$ para todo $v \in \{0, 1\}^\ell$. Más aún, φ_F se computa en tiempo polinomial a partir de $\langle F \rangle$ y tiene tamaño $O(\ell 2^\ell)$.

Demostración.

$$\begin{aligned} \varphi &= \bigwedge_{v:F(v)=0} \neg \left(\bigwedge_{i:v(i)=1} p_i \wedge \bigwedge_{i:v(i)=0} \neg p_i \right) \\ &= \bigwedge_{v:F(v)=0} \underbrace{\left(\bigvee_{i:v(i)=1} \neg p_i \vee \bigvee_{i:v(i)=0} p_i \right)}_{\text{tamaño } O(\ell)} \\ &\quad \underbrace{\hspace{10em}}_{\text{tamaño } O(\ell 2^\ell)} \end{aligned}$$

4-30

Representación de funciones booleanas

Ejemplo

$v(0)$	$v(1)$	$v(2)$	$F(v)$
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

$$\begin{aligned}\varphi = & (p_1 \vee p_2 \vee p_3) \wedge \\ & (p_1 \vee p_2 \vee \neg p_3) \wedge \\ & (\neg p_1 \vee p_2 \vee p_3) \wedge \\ & (\neg p_1 \vee p_2 \vee \neg p_3)\end{aligned}$$

4-31

Representación de funciones booleanas

Corolario

Para toda $F : \{0, 1\}^\ell \rightarrow \{0, 1\}^k$ existe una fórmula $\varphi_F(p_1, \dots, p_{\ell+k})$ en CNF tal que $uv \models \varphi_F$ sii $F(u) = v$ para todo $u \in \{0, 1\}^\ell$, $v \in \{0, 1\}^k$. Más aún, φ_F se computa en tiempo polinomial a partir de $\langle F \rangle$ y tiene tamaño $O((\ell + k)2^{\ell+k})$.

Demostración.

Considerar $G : \{0, 1\}^{\ell+k} \rightarrow \{0, 1\}$ definida como

$$G(uv) = 1 \quad \text{sii} \quad F(u) = v$$

y aplicar el resultado anterior. □

4-32

Clase 5

Clase 1

Preliminares
Representaciones en binario
Problemas de decisión
Máquinas de Turing
Configuraciones y cálculos

Clase 2

Funciones computables
Tiempo de cómputo
Codificación de máquinas
Variantes de máquinas

Clase 3

Halt y máquina universal
La clase **P**
La clase **NP**

Clase 4

Máquinas no-determinísticas
Reducibilidad polinomial
Lógica proposicional

Clase 5

Mini-configuraciones
Teorema de Cook-Levin
Problemas **NP-completos**
La clase **coNP**
Las clases **EXPTIME** y **NEXPTIME**

Clase 6

La jerarquía de tiempos determinísticos
La jerarquía de tiempos no-determinísticos
Teorema de Ladner

Clase 7

Espacio usado por un cómputo
Espacio polinomial: **PSPACE** y **NPSPACE**
Teorema de Savitch

Clase 8

Espacio logarítmico: **L** y **NL**
Teorema de Immerman-Szelepcsényi

Clase 9

La jerarquía polinomial
Problemas Σ_i^P -completos

Clase 10

Máquinas con oráculo
Teorema de Baker, Gill, Solovay
La jerarquía polinomial y **NP** con oráculos

Clase 11

Circuitos booleanos
La clase **P/poly**
Tamaño y profundidad de circuitos

Clase 12

Las clases **NC_{nu}** y **AC_{nu}**
Uniformidad: las clases **NC** y **AC**
P-completitud

5-1

Mini-configuraciones

Clase 5

Mini-configuraciones
Teorema de Cook-Levin
Problemas **NP-completos**
La clase **coNP**
Las clases **EXPTIME** y **NEXPTIME**

5-2

Mini-configuración

Para simplificar, consideremos una máquina determinística $M = (\Sigma, Q, \delta)$

- sin cinta de salida,
- con una cinta de entrada y
- con una sola cinta de trabajo (que funciona como salida también)

Todo lo que digamos a continuación se puede generalizar a cualquier cantidad de cintas de trabajo.

5-3

Mini-configuración

Supongamos un cómputo

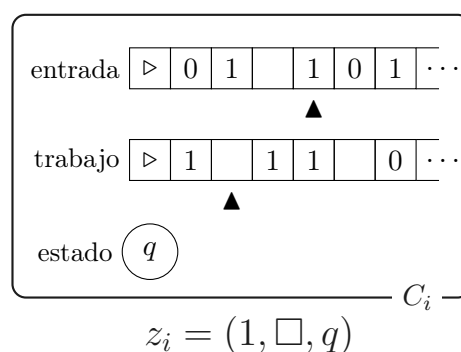
$$C_0, \dots, C_\ell$$

de M una con entrada y . La **mini-configuración en el paso i** es una tupla

$$z_i = (a_i, b_i, c_i) \in \Sigma \times \Sigma \times Q$$

tal que

- a_i es el símbolo leído por la cabeza de entrada en C_i
- b_i es el símbolo leído en la cinta de trabajo en C_i
- c_i es el estado de C_i



5-4

Máquina *oblivious*

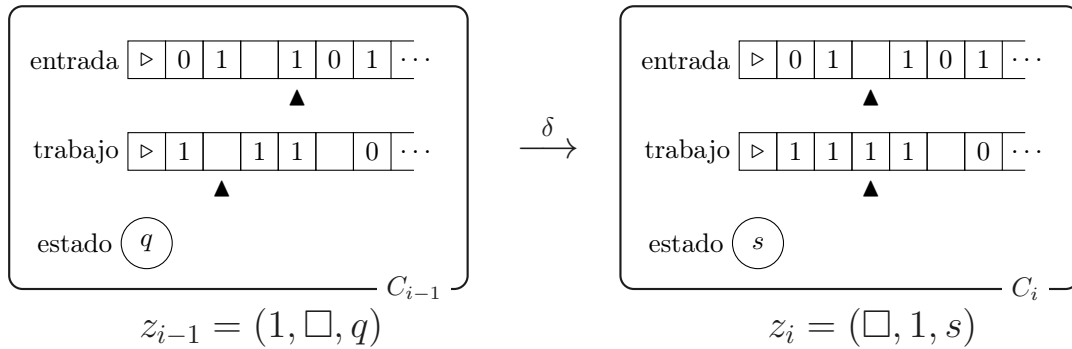
Supongamos que, además, M es *oblivious*: podemos calcular la posición de las cabezas de entrada y trabajo en el cómputo de $M(y)$ en función de $|y|$ y el número de paso (pero independiente de y).

Estas tres funciones son computables en tiempo polinomial:

- $e(i, n) =$ posición de la cabeza de entrada en el paso i en el cómputo de M con entrada 0^n
- $t(i, n) =$ posición de la cabeza de trabajo en el paso i en el cómputo de M con entrada 0^n
- $prev(i, n) = \max\{j < i : t(j, n) = t(i, n)\} \cup \{1\}$, es decir $prev(i, n)$ es el máximo paso $j < i$ en el cómputo de M con entrada 0^n tal que la posición de la cabeza de trabajo en el paso j coincide con la posición en el paso i ; o 1 si no existe tal j

5-5

Mini-configuración: del paso $i - 1$ al paso i



Sea z_i la i -ésima mini-configuración en el cómputo de M con entrada x . $z_0 = (x(0), \square, q_0)$ y para $i > 0$ calculamos z_i con:

- el estado y los símbolos leídos en el paso $i - 1$; esto está en z_{i-1}
- la función de transición δ de M
- el contenido de la cinta de entrada (solo lectura; no cambia a lo largo del cómputo) en la posición $e(i, |x|)$
- el contenido de la cinta de trabajo en la posición $prev(i, |x|)$; esto está en $z_{prev(i, |x|)}$

5-6

La función F representa la evolución en un paso

Supongamos el cómputo $C_0, \dots, C_\ell$ de $M = (\Sigma, Q, \delta)$ determinística, *oblivious*, sin cinta de salida con entrada y y la correspondiente secuencia de mini-configuraciones

$$z_0, \dots, z_\ell$$

Podemos codificar cada mini-configuración $z \in \Sigma \times \Sigma \times Q$ con $\langle z \rangle \in \{0, 1\}^k$ con $k = 4 + \lceil \log |Q| \rceil$ (k depende solo de M).

- $00, 11, 01, 10$ codifica cada símbolo de $\Sigma = \{0, 1, \triangleright, \square\}$.
- cada estado de Q se codifica con una cadena en $\{0, 1\}^{\lceil \log |Q| \rceil}$; $0 \dots 0$ codifica q_0 ; $0 \dots 1$ codifica q_f

Para $i > 0$, definimos $F : \{0, 1\}^k \times \{0, 1\}^k \times \{0, 1\}^2 \rightarrow \{0, 1\}^k$:

$$F\left(\underbrace{\langle z_{i-1} \rangle}_{\substack{\text{configuración} \\ \text{anterior} \\ (k \text{ variables})}}, \underbrace{\langle z_{prev(i, |y|)} \rangle}_{\substack{\text{información} \\ \text{de la cinta de} \\ \text{trabajo} \\ (k \text{ variables})}}, \underbrace{\langle x(e(i, |y|)) \rangle}_{\substack{\text{codificación} \\ \text{del bit actual} \\ \text{leído} \\ (2 \text{ variables})}} \right) = \langle z_i \rangle.$$

y $F(x) = 0^k$ para los otros casos (no nos importa).

5-7

La función F representada en CNF

Tenemos

$$F : \{0, 1\}^k \times \{0, 1\}^k \times \{0, 1\}^2 \rightarrow \{0, 1\}^k.$$

Existe una fórmula booleana

$$\varphi_F(\bar{p}, \bar{q}, \bar{r}, \bar{s})$$

con variables libres

- $\bar{p} = p_1, \dots, p_k$, que codifica $\langle z_{i-1} \rangle$
- $\bar{q} = q_1, \dots, q_k$ que codifica $\langle z_{prev(i, |y|)} \rangle$
- $\bar{r} = r_1, r_2$, que codifica $\langle y(e(i, |y|)) \rangle$
- $\bar{s} = s_1, \dots, s_k$, que codifica $\langle z_i \rangle$

en CNF tal que para todo $\bar{a}, \bar{b}, \bar{d} \in \{0, 1\}^k$ y $\bar{c} \in \{0, 1\}^2$:

$$\bar{a}\bar{b}\bar{c}\bar{d} \models \varphi_F(\bar{p}, \bar{q}, \bar{r}, \bar{s}) \quad \text{sii} \quad \bar{d} = F(\bar{a}, \bar{b}, \bar{c})$$

Más aun, podemos computar φ_F a partir de $\langle F \rangle$ en tiempo polinomial y φ_F tiene tamaño $\leq (3k + 2)2^{3k+2}$.

Fijada la máquina M , k es constante y luego φ_F tiene tamaño constante.

5-8

Teorema de Cook-Levin

Clase 5

Mini-configuraciones

Teorema de Cook-Levin

Problemas **NP-completos**

La clase **coNP**

Las clases **EXPTIME** y **NEXPTIME**

5-9

SAT es **NP-hard**

Teorema

SAT $\in$ **NP-hard**.

Demostración ¡incorrecta!

Fijemos $\mathcal{L} \in \mathbf{NP}$ y veamos que $\mathcal{L} \leq_p \mathbf{SAT}$.

Como $\mathcal{L} \in \mathbf{NP}$, existe una máquina determinística M tal que

- M corre en tiempo $t(n)$, con t un polinomio
- existe un polinomio p tal que

$$x \in \mathcal{L} \quad \text{sii} \quad \exists u \in \{0, 1\}^{p(|x|)} \quad M(xu) = 1$$

Definimos $F_x : \{0, 1\}^{p(|x|)} \rightarrow \{0, 1\}$ como $F_x(u) = M(xu)$.

Podemos computar $\varphi_x(q_1, \dots, q_{p(|x|)}) \in \mathbf{CNF}$ tal que

$$u \models \varphi_x \quad \text{sii} \quad F_x(u) = 1 \quad \text{sii} \quad M(xu) = 1$$

Luego $x \in \mathcal{L}$ sii φ_x es satisfacible sii $\varphi_x \in \mathbf{SAT}$.

Problema: φ_x tiene tamaño exponencial: $O(p(|x|)2^{p(|x|)})$.

5-10

Demostración.

Fijemos $\mathcal{L} \in \mathbf{NP}$ y veamos que $\mathcal{L} \leq_p \mathbf{SAT}$.

Como $\mathcal{L} \in \mathbf{NP}$, existe una máquina determinística M tal que

- M corre en tiempo $t(n)$, con t un polinomio
- M es *oblivious*, sin cinta de salida y con única cinta de trabajo
- existe un polinomio p tal que

$$x \in \mathcal{L} \quad \text{sii} \quad \exists u \in \{0, 1\}^{p(|x|)} \quad M(xu) = 1$$

Dado x construimos $\varphi_x \in \mathbf{CNF}$ en tiempo polinomial tal que

$$x \in \mathcal{L} \quad \text{sii} \quad \varphi_x \in \mathbf{SAT}.$$

La máquina M está fija; x es variable.

5-11

$x \in \mathcal{L}$

sii $\exists u \in \{0, 1\}^{p(|x|)} \quad M(xu) = 1$

sii existe $u \in \{0, 1\}^{p(|x|)}$ y existe un cómputo $C_0, \dots, C_{t(|xu|)}$ de M a partir de la entrada xu tal que la salida en $C_{t(|xu|)}$ es 1

sii existe una codificación de la entrada $y \in \{0, 1\}^{2n+4}$ con $n = |x| + p(|x|)$ y una secuencia de mini-configuraciones $z_0, \dots, z_m$, $z_i \in \{0, 1\}^k$ (k depende solo de M), con $m = t(n) = t(|x| + p(|x|))$, tal que:

$\psi_1 =$ “la entrada empieza con x y $u \in \{0, 1\}^*$ ”

$\psi_2 =$ “ z_0 es la configuración inicial”

$\psi_3 =$ “ z_j evoluciona en z_{j+1} para $j = 0, \dots, m-1$ ”

$\psi_4 =$ “ z_m es una configuración final de M aceptadora”

Veamos que cada una de estas condiciones se expresa con una fórmula

$$\psi_j(\underbrace{p_0, \dots, p_{2n+3}}_{\text{entrada } y}, \underbrace{q_1^0, \dots, q_k^0}_{z_0}, \dots, \underbrace{q_1^m, \dots, q_k^m}_{z_m})$$

cantidad polinomial de variables

($j = 1 \dots 4$) en CNF computable en tiempo polinomial a partir de x .

5-12

ψ_1 = “la entrada empieza con x y $u \in \{0, 1\}^*$ ”

= y codifica la cinta con contenido “ $\triangleright x u \square$ ”:

- recordar que 00, 11, 01, 10 codifica cada símbolo de $\Sigma = \{0, 1, \triangleright, \square\}$.
- $y(0)y(1) = 01$ (marca $\triangleright$)
- $y(2j+2)$ y $y(2j+3)$ codifican a $x(j)$ para $0 \leq j \leq |x| - 1$
- $y(2j)$ y $y(2j+1)$ tienen el mismo valor para $|x| + 1 \leq j \leq n$
- $y(2n+2)y(2n+3) = 10$ (marca $\square$)

se expresa con

$$\begin{aligned} \psi_1 = & \neg p_0 \wedge p_1 \\ & \wedge \bigwedge_{j=0 \dots |x|-1} \begin{cases} p_{2j+2} \wedge p_{2j+3} & \text{si } x(j) = 1 \\ \neg p_{2j+2} \wedge \neg p_{2j+3} & \text{si } x(j) = 0 \end{cases} \\ & \wedge \bigwedge_{i=|x|+1 \dots n} p_{2i} \leftrightarrow p_{2i+1} \\ & \wedge p_{2n+2} \wedge \neg p_{2n+3} \end{aligned}$$

Notar que no se especifican los valores de las variables correspondientes a u . Observar que $|\psi_1| = O(n)$.

5-13

ψ_2 = “ z_0 es la configuración inicial”

= “ $z_0 = (x(0), \square, q_0)$ ” se expresa con

$$\psi_2 = \underbrace{\begin{cases} q_1^0 \wedge q_2^0 & \text{si } x(0) = 1 \\ \neg q_1^0 \wedge \neg q_2^0 & \text{si } x(0) = 0 \end{cases}}_{x(0)} \wedge \underbrace{q_3^0 \wedge \neg q_4^0}_{\square} \wedge \underbrace{\bigwedge_{i=5 \dots k} \neg p_i^0}_{q_0}$$

Observar que $|\psi_1| = O(1)$

(recordar que k depende solo de M pero M está fija)

5-14

$\psi_3 = “z_j \text{ evoluciona en } z_{j+1} \text{ para } j = 0, \dots, m-1”$. Para $0 < i \leq m$ la condición

$$\underbrace{\langle z_i \rangle}_k = F(\underbrace{\langle z_{i-1} \rangle}_k, \underbrace{\langle z_{prev(i,n)} \rangle}_k, \underbrace{\langle y(e(i,n)) \rangle}_2)$$

se expresa con una $\psi_3^i \in \text{CNF}$, con ayuda la fórmula φ_F que ya analizamos, en concreto $\psi_3^i =$

$$\varphi_F \left(\underbrace{q_1^{i-1}, \dots, q_k^{i-1}}_{\langle z_{i-1} \rangle}, \underbrace{q_1^{prev(i,n)}, \dots, q_k^{prev(i,n)}}_{\langle z_{prev(i,n)} \rangle}, \underbrace{p_{2e(i,n)}, p_{2e(i,n)+1}}_{\langle xu(e(i,n)) \rangle}, \underbrace{q_1^i, \dots, q_k^i}_{\langle z_i \rangle} \right)$$

Podemos suponer que $e(i, n) \leq n+1$; primera celda de la cinta de entrada es posición 0; M con entrada xu no necesita leer más allá del primer blanco después de xu

Computamos ψ_3^i en tiempo polinomial y tiene tamaño $O(k2^{3k})$, pero k es constante.

Entonces $\psi_3 = \bigwedge_{i=1, \dots, m} \psi_3^i$ se construye en tiempo polinomial y expresa $(\forall i = 1, \dots, t(n)) \ z_i = F(z_{i-1}, z_{prev(i,|y|)}, \langle xu(e(i, n)) \rangle)$

Observar que $|\psi_3| = O(k2^{3k}m) = O(t(|x| + p(|x|)))$ porque k es constante y $m = t(|x| + n(|x|))$

5-15

$\psi_4 = “z_m \text{ es una configuración final de } M \text{ aceptadora}”$
 $= z_m$ es de la forma $(*, 1, q_f)$, se expresa con ψ_4 análogo a ψ_2 .
 $|\psi_4| = O(1)$ (k depende solo de M pero M está fija)

Finalmente,

$$\varphi_x = \psi_1 \wedge \psi_2 \wedge \psi_3 \wedge \psi_4$$

Observar que $|\varphi_x| = O(|x| + t(|x| + p(|x|)))$.

- φ_x se construye en tiempo polinomial en $|x| + p(|x|)$, o sea en tiempo polinomial en $|x|$
- $x \in \mathcal{L}$ sii φ_x es satisfacible sii $\varphi_x \in \text{SAT}$

□

Corolario

SAT $\in$ NP-completo.

3SAT es **NP-completo**

Ya vimos que 3SAT es **NP**. Para ver que 3SAT es **NP-hard** probamos

Ejercicio

SAT $\leq_p$ 3SAT.

Corolario

SAT, 3SAT $\in$ NP-completos .

5-17

Problemas **NP-completos**

Clase 5

Mini-configuraciones

Teorema de Cook-Levin

Problemas **NP-completos**

La clase **coNP**

Las clases **EXPTIME** y **NEXPTIME**

5-18

Ejemplo de problema **NP-completo**

Proposición

INDSET $\in$ **NP-completo**.

Demostración.

Ya vimos que INDSET es **NP**. Para ver que INDSET es **NP-hard**, probamos que $3SAT \leq_p$ INDSET. Supongamos una fórmula

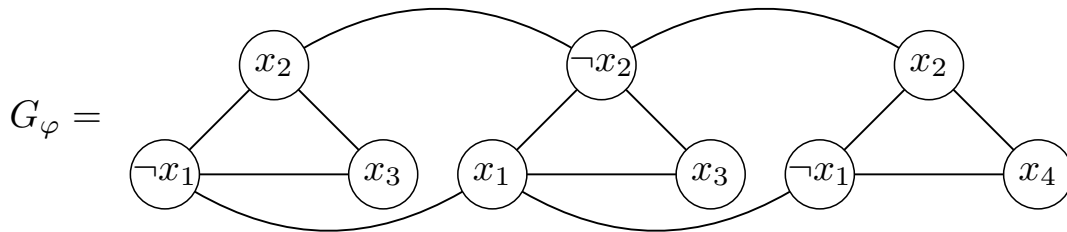
$$\varphi = (l_{11} \vee l_{12} \vee l_{13}) \wedge (l_{21} \vee l_{22} \vee l_{23}) \wedge \cdots \wedge (l_{m1} \vee l_{m2} \vee l_{m3})$$

en 3CNF con m cláusulas (l_{ij} son literales) y variables $x_1, \dots, x_n$.

- Definimos un grafo G_φ con $3m$ vértices. Cada vértice corresponde a cada variable de cada cláusula. Supongamos que z es un vértice de G_φ correspondiente a l_{ij} y que z' es un vértice de G_φ correspondiente a $l_{i'j'}$. Definimos una arista entre z y z' si $i = i'$ o l_{ij} es la negación de $l_{i'j'}$ (o viceversa). G_φ es construible en tiempo polinomial en $|\varphi|$.
- Probamos que φ es satisfacible sii G_φ tiene un conjunto independiente de al menos m vértices.

5-19

$$\varphi = (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_4)$$



- Supongamos que $v \models \varphi$. Entonces para todo $i = 1, \dots, m$ tenemos $v \models l_{i1} \vee l_{i2} \vee l_{i3}$, de modo que $v \models l_{ij}$ para algún j . Sea S el conjunto de vértices $z_1, \dots, z_m$ tal que z_i corresponde a l_{ij} y $v \models l_{ij}$. S es independiente: si existiera arista entre z_i y $z_{i'}$ sería porque 1) $i = i'$ o 2) z_i corresponde a l_{ij} , $z_{i'}$ corresponde a $l_{i'j'}$ y l_{ij} es la negación de $l_{i'j'}$ (o viceversa). Ninguna puede pasar.
- Si S es un conjunto independiente en G_φ de m vértices, tenemos exactamente un vértice en cada “triángulo”. Para cada $z \in S$:
 - si z corresponde a x_i definimos $v(x_i) = 1$
 - si z corresponde a $\neg x_i$ definimos $v(x_i) = 0$

Para todas las otras variables x para las que no está definido v , definimos $v(x)$ de forma arbitraria. v está bien definida porque S es independiente; $v \models \varphi$. □

5-20

Camino hamiltoniano

Un **camino hamiltoniano** en un grafo dirigido G es un camino que visita todos los vértices de G_φ exactamente una vez.

Problema: CAMHAM (Camino hamiltoniano)

$\text{CAMHAM} = \{\langle G \rangle : G \text{ tiene un camino hamiltoniano}\}$

Proposición

$\text{CAMHAM} \in \text{NP-completo}$.

5-21

Problema del viajante de comercio (*travelling salesman problem*)

Dadas n ciudades, representamos la distancia entre cada par de ciudades por medio de una matriz M de $n \times n$.

Problema: TSP (Problema del viajante de comercio o *Travelling Salesman Problem*)

$\text{TSP} = \{\langle M, k \rangle : \begin{array}{l} \text{hay una ruta de distancia } \leq k \text{ (de acuerdo a } M\text{)} \\ \text{que visita todas las ciudades de } G \text{ exactamente } \\ \text{una vez y al finalizar vuelve a la ciudad de origen} \end{array}\}$

Proposición

$\text{TSP} \in \text{NP-completo}$.

5-22

Problema de la mochila (*knapsack problem*)

Representamos una lista de n ítems en una mochila con su valor y su peso por medio de una lista

$$M = [(v_1, p_1), (v_2, p_2), \dots, (v_n, p_n)].$$

Problema: KNAPSACK (Problema de la mochila o *knapsack problem*)

$\text{KNAPSACK} = \{ \langle M, v, p \rangle : \begin{array}{l} \text{existe un conjunto de ítems por un valor} \\ \text{total} \geq v \text{ y con un peso} \leq p \end{array} \}$

Proposición

KNAPSACK $\in$ NP-completo.

5-23

La clase **coNP**

Clase 5

Mini-configuraciones

Teorema de Cook-Levin

Problemas **NP-completos**

La clase **coNP**

Las clases **EXPTIME** y **NEXPTIME**

5-24

En general: problemas **C-COMPLETOS** y **C-HARD**

La noción de **NP-hard** y **NP-completo** se aplica a otras clases de complejidad.

En general, si **C** es una clase de complejidad, entonces

Clase de complejidad: C-hard, C-completo
$\mathcal{L}$ es C-hard si $\mathcal{L}' \leq_p \mathcal{L}$ para todo $\mathcal{L}' \in \mathbf{C}$. $\mathcal{L}$ es C-completo si $\mathcal{L} \in \mathbf{C}$ y $\mathcal{L}$ es C-hard .

- En realidad, estas son nociones de completitud y hardness para la reducción $\leq_p$.
- Más adelante veremos clases para las que no tiene sentido usar $\leq_p$ y necesitamos reducciones más débiles.

5-25

En general: problemas **coC**

Notación: Complemento de un lenguaje
$\overline{\mathcal{L}} = \{0, 1\}^* \setminus \mathcal{L}$ es el complemento de $\mathcal{L}$.

Clase de complejidad: coC
Si C es una clase de complejidad, definimos
$\mathbf{coC} = \{\mathcal{L} : \overline{\mathcal{L}} \in \mathbf{C}\}.$

5-26

La clase **coNP**

Clase de complejidad: **coNP**

coNP = $\{\mathcal{L} : \overline{\mathcal{L}} \in \mathbf{NP}\}$.

Es decir, **coNP** es la clase de lenguajes $\mathcal{L}$ tal que existe un polinomio $p : \mathbb{N} \rightarrow \mathbb{N}$ y una máquina determinística M tal que

- M corre en tiempo polinomial
- para todo x :

$$x \in \mathcal{L} \quad \text{sii} \quad \text{para todo } u \in \{0, 1\}^{p(|x|)} \quad M(\langle x, u \rangle) = 1$$

5-27

Relación de **coNP** con **P** y **NP**

Ejercicio

$\mathbf{P} \subseteq \mathbf{NP} \cap \mathbf{coNP}$.

Ejercicio

Si $\mathbf{P} = \mathbf{NP}$ entonces $\mathbf{NP} = \mathbf{coNP}$.

5-28

Ejemplo de problema **coNP-completo**

Problema: Tautología

$$\text{TAUT} = \{\langle \varphi \rangle : \varphi \in \text{CNF} \text{ es una tautología}\}$$

Observar que φ es una tautología sii $\neg\varphi$ es insatisfacible.

$$\langle \varphi \rangle \in \text{TAUT} \quad \text{sii} \quad \langle \neg\varphi \rangle \notin \text{SAT}$$

Ejercicio

TAUT $\in$ **coNP-completo**.

5-29

Las clases **EXPTIME** y **NEXPTIME**

Clase 5

Mini-configuraciones

Teorema de Cook-Levin

Problemas **NP-completos**

La clase **coNP**

Las clases **EXPTIME** y **NEXPTIME**

5-30

Las clases **EXPTIME** y **NEXPTIME**

Clase de complejidad: **EXPTIME** y **NEXPTIME**

$$\mathbf{EXPTIME} = \bigcup_{c>0} \mathbf{DTIME}(2^{n^c}).$$
$$\mathbf{NEXPTIME} = \bigcup_{c>0} \mathbf{NDTIME}(2^{n^c}).$$

Son los análogos de **P** y **NP** pero con tiempo exponencial.

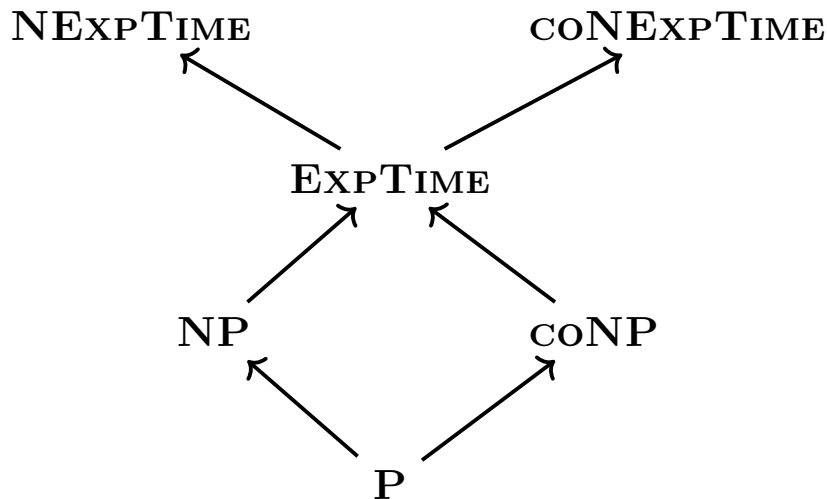
5-31

$$\mathbf{P} \subseteq \mathbf{NP} \subseteq \mathbf{EXPTIME} \subseteq \mathbf{NEXPTIME}$$

Ejercicio

NP $\subseteq$ **EXPTIME**.

5-32



5-33

Teorema

Si $\mathbf{P} = \mathbf{NP}$ entonces $\mathbf{EXPTIME} = \mathbf{NEXPTIME}$.

Demostración.

Supongamos que $\mathbf{P} = \mathbf{NP}$. Tomemos $\mathcal{L} \in \mathbf{NDTIME}(2^{n^c})$ y N una máquina no-determinística que decide $\mathcal{L}$ en tiempo $c \cdot 2^{n^c}$.

Consideremos $\mathcal{L}_{\text{pad}} = \{\langle x, 1^{2^{|x|^c}} \rangle : x \in \mathcal{L}\}$.

Veamos que $\mathcal{L}_{\text{pad}} \in \mathbf{NP}$. Definimos una máquina no-determinística N' tal que con entrada y hace esto:

si no existe z tal que $y = \langle z, 1^{2^{|z|^c}} \rangle$, rechazar
si no, y es de la forma $\langle z, 1^{2^{|z|^c}} \rangle$
simular N con entrada z por $c \cdot 2^{|z|^c}$ pasos
devolver la salida de esta simulación

N' corre en tiempo polinomial (en $|y|$) y por lo tanto

$\mathcal{L}_{\text{pad}} \in \mathbf{NP} = \mathbf{P}$. Veamos que $\mathcal{L}_{\text{pad}} \in \mathbf{DTIME}(2^{n^c})$. Definimos una máquina determinística M tal que dada la entrada x :

computa $y = \langle x, 1^{2^{|x|^c}} \rangle$ (tiempo $O(2^{|x|^c})$)
decide si $y \in \mathcal{L}_{\text{pad}}$ (sii $x \in \mathcal{L}$) (tiempo poli en $|y|$)

M corre en tiempo $O(2^{|x|^{c+d}})$ y decide $\mathcal{L}$, luego $\mathcal{L} \in \mathbf{DTIME}(2^{|x|^c})$. □

5-34

P vs NP

- ¿ $P = NP$ o $P \subsetneq NP$? Es una pregunta abierta.
- ¿Reconocer la corrección de una solución es esencialmente más fácil que generarla?
 - Por ejemplo, dado un sistema axiomático $\mathcal{S}$ (= axiomas + reglas de inferencia) el lenguaje

$\{\langle \varphi, 1^n \rangle : \varphi \text{ tiene una demostración en } \mathcal{S} \text{ de longitud } \leq n\}$

es **NP**. Pero verificar que una secuencia de pasos es una demostración es **P**.

- En algunos casos, ¿lo mejor que podemos hacer es usar la fuerza bruta para llegar a la solución?

5-35

Clase 6

Clase 1

Preliminares
Representaciones en binario
Problemas de decisión
Máquinas de Turing
Configuraciones y cálculos

Clase 2

Funciones computables
Tiempo de cómputo
Codificación de máquinas
Variantes de máquinas

Clase 3

Halt y máquina universal
La clase **P**
La clase **NP**

Clase 4

Máquinas no-determinísticas
Reducibilidad polinomial
Lógica proposicional

Clase 5

Mini-configuraciones
Teorema de Cook-Levin
Problemas **NP-completos**
La clase **coNP**
Las clases **EXPTIME** y **NEXPTIME**

Clase 6

La jerarquía de tiempos determinísticos
La jerarquía de tiempos no-determinísticos
Teorema de Ladner

Clase 7

Espacio usado por un cómputo
Espacio polinomial: **PSPACE** y **NPSPACE**
Teorema de Savitch

Clase 8

Espacio logarítmico: **L** y **NL**
Teorema de Immerman-Szelepcsényi

Clase 9

La jerarquía polinomial
Problemas Σ_i^P -completos

Clase 10

Máquinas con oráculo
Teorema de Baker, Gill, Solovay
La jerarquía polinomial y **NP** con oráculos

Clase 11

Circuitos booleanos
La clase **P/poly**
Tamaño y profundidad de circuitos

Clase 12

Las clases **NC_{nu}** y **AC_{nu}**
Uniformidad: las clases **NC** y **AC**
P-completitud

La jerarquía de tiempos determinísticos

Clase 6

La jerarquía de tiempos determinísticos

La jerarquía de tiempos no-determinísticos

Teorema de Ladner

6-2

Notación de ‘o chica’

Definición

Sean $f, g : \mathbb{N} \rightarrow \mathbb{N}$. Decimos que $f = o(g)$ si para todo $\epsilon > 0$

$$f(n) \leq \epsilon \cdot g(n)$$

para todo n suficientemente grande o, equivalentemente, si

$$\lim_n f(n)/g(n) = 0.$$

Notar que si $f = o(g)$ entonces $f = O(g)$.

Ejemplos

- $4(n+2) \neq o(n)$ pero observar que $4(n+2) = O(n)$
- $4(n+2) = o(n^2)$
- $n^c = o(2^n)$ para todo $c \in \mathbb{N}$
- $k^n = o(2^{n^2})$ para todo $k \in \mathbb{N}$

6-3

Re-codificación de máquinas determinísticas

En clases pasadas

- codificamos cualquier máquina M con $i = \langle M \rangle \in \{0, 1\}^*$.
 - M_i representaba la única máquina M con índice i , es decir $\langle M \rangle = i$.
- definimos la máquina universal U que recibía como parámetros $\langle i, x \rangle$ y simulaba $M_i(x)$.

En esta clase vamos a

- codificar cada máquina M con *infinitas* cadenas $\langle \langle M \rangle, z \rangle$ para cualquier $z \in \{0, 1\}^*$
 - Cualquier $\langle \langle M \rangle, z \rangle$ representa M
- usar una máquina universal U que usa la *nueva* codificación de máquinas:
 - $U(\langle w, x \rangle)$ simula $M_w(x)$ como antes, solo que ahora w es un índice de la *nueva* codificación.

6-4

Re-codificación de máquinas determinísticas

- Lo importante es que para la nueva codificación

$$(M_i)_{i \in \{0,1\}^*}$$

vale que dada una máquina M existen infinitos i tal que $M = M_i$.

- Seguimos teniendo que

Existe una máquina U que computa la función $u(\langle i, x \rangle) = M_i(x)$. Más aún, si M_i con entrada x termina en t pasos, entonces U con entrada $\langle i, x \rangle$ termina en $c \cdot t \cdot \log t$ pasos, donde c depende solo de i .

6-5

Teorema

Si f, g son construibles en tiempo y cumplen que

$$f(n) \cdot \log f(n) = o(g(n))$$

entonces $\mathbf{DTIME}(f(n)) \subsetneq \mathbf{DTIME}(g(n))$.

Ejemplo

$\mathbf{DTIME}(n^c) \subsetneq \mathbf{DTIME}(n^d)$ si $c < d$.

6-6

Demostración.

Definimos una máquina determinística D que con entrada x hace esto:

*calcular $n = |x|$ y $t = g(n)$
simular $U(\langle x, x \rangle)$ por t pasos
si $U(\langle x, x \rangle)$ no terminó en $\leq t$ pasos, devolver 0
si no, devolver $1 - U(\langle x, x \rangle)$*

D corre en tiempo $O(g(n))$, de modo que $\mathcal{L}(D) \in \mathbf{DTIME}(g(n))$.

Supongamos $\mathcal{L}(D) \in \mathbf{DTIME}(f(n))$.

Sea M una máquina determinística que decide $\mathcal{L}(D)$ en tiempo $c \cdot f(n)$. Existe n_0 tal que si $|x| \geq n_0$, entonces M con entrada x termina en $\leq c \cdot f(|x|)$ pasos.

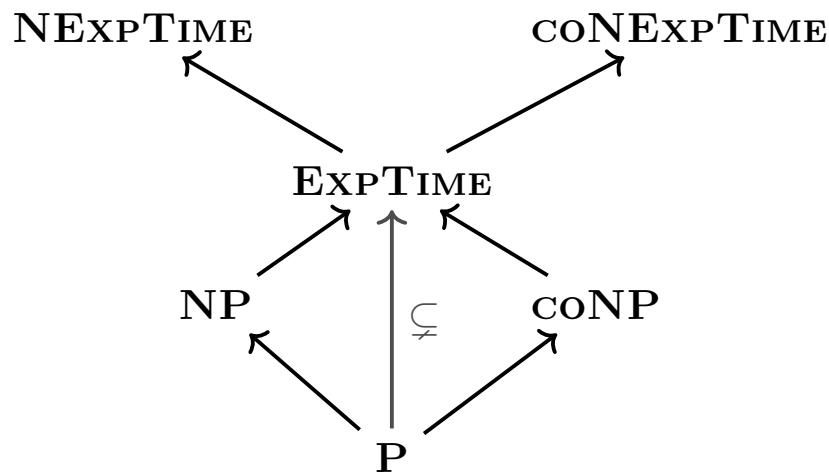
$U(\langle x, x \rangle)$ simula $M_x(x)$. Si $M_x(x)$ termina en $\leq c \cdot f(|x|)$ pasos, entonces $U(\langle x, x \rangle)$ necesita $\leq c' \cdot c \cdot f(|x|) \cdot \log(c \cdot f(|x|))$ pasos para terminar. Existen $d, n_1 \geq n_0$ tal que para todo $n \geq n_1$

$$c' \cdot c \cdot f(n) \cdot \log(c \cdot f(n)) \leq d \cdot f(n) \cdot \log f(n) \leq g(n)$$

Tomemos x tal que $|x| \geq n_1$ y $M = M_x$.

$U(\langle x, x \rangle)$ termina en $\leq g(|x|)$ pasos y $D(x) = 1 - M_x(x) = 1 - M(x)$.

Pero $\mathcal{L}(D) = \mathcal{L}(M)$. Absurdo. $\square$



6-8

Diagonalización

Parecido a lo que hicimos para demostrar que *halt* no es computable.

Definimos una máquina D tal que

- D corre en tiempo $O(g(n))$
- D se porta distinto a todas las máquinas que corren en tiempo $O(f(n) \cdot \log f(n))$.

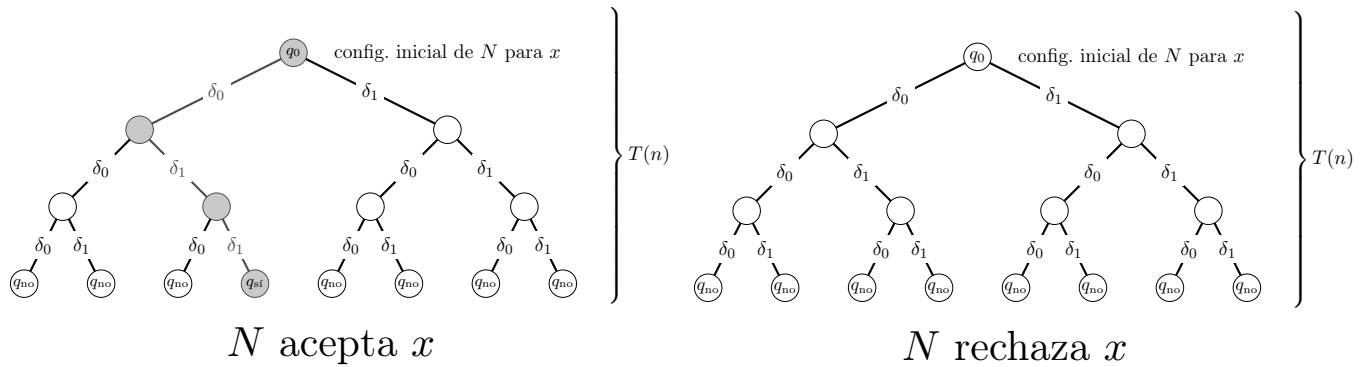
$D(x)$ ‘niega’ la salida de $M_x(x)$:

- cuando $M_x(x) = 1$, $D(x) = 0$
- cuando $M_x(x) = 0$, $D(x) = 1$
- D puede *diagonalizar* en tiempo $O(n^3)$ y definir una función que no es computable por ninguna máquina en tiempo $O(n^2)$

6-9

Diagonalización

¿Cómo diagonalizamos una máquina no-determinística N ?



- ¿Podemos diagonalizar en tiempo $O(n^3)$ las máquinas no-determinísticas que corren en tiempo $O(n^2)$?
- ¿Cómo ‘negamos’ la salida de una máquina no-determinística? Se conjetura que $\mathbf{NP} \neq \mathbf{coNP}$.
- Pero sí sabemos que si $\mathcal{L} \in \mathbf{NDTIME}(T(n))$, entonces $\overline{\mathcal{L}} \in \mathbf{DTIME}(2^{T(n)^c})$ para alguna constante c

6-10

La jerarquía de tiempos no-determinísticos

Clase 6

La jerarquía de tiempos determinísticos

La jerarquía de tiempos no-determinísticos

Teorema de Ladner

6-11

Re-codificación de máquinas no-determinísticas

- Lo mismo que hicimos para las máquinas determinísticas
- Nueva codificación

$$(N_i)_{i \in \{0,1\}^*}$$

tal que para toda máquina no-determinística N existen infinitos i tal que $N = N_i$.

- Seguimos teniendo que
Existe una máquina no-determinística NU que tal que NU acepta $(\langle i, x \rangle)$ sii N_i acepta x y si N_i corre en tiempo $T(n)$ entonces $NU(\langle i, x \rangle)$ decide si N_i acepta o rechaza x en $c \cdot T(|x|)$ pasos, donde c depende solo de i .
- A veces vamos a usar una enumeración $(N_i)_{i \in \mathbb{N}}$ de las máquinas no-determinísticas indexadas por números naturales:
 - para $i \in \mathbb{N}$, N_i representa N_z , donde z es la representación de i en binario sin el 1 inicial

6-14

La jerarquía de tiempos no-determinísticos

Teorema

Si f, g son construibles en tiempo, no decrecientes y cumplen que

$$f(n+1) = o(g(n)),$$

entonces $\mathbf{NDTIME}(f(n)) \subsetneq \mathbf{NDTIME}(g(n))$.

Ejemplo

$\mathbf{NDTIME}(n^c) \subsetneq \mathbf{NDTIME}(n^d)$ si $c < d$.

6-15

Demostración.

Sea $(N_i)_{i \in \mathbb{N}}$ una enumeración de todas las máquinas no-determinísticas. Definimos la máquina no-determinística N con entrada x así:

si x no es de la forma $1^i 0 y$, rechazar
si no, supongamos $x = 1^i 0 y$
 si $|y| < g(i)$,
 simular no determinísticamente $N_i(x0)$ y
 $N_i(x1)$ por $g(|x|)$ pasos (*)
 si alguna no terminó: rechazar
 si las dos terminaron:
 aceptar sii $N_i(x0)$ acepta y $N_i(x1)$ acepta
 si $|y| = g(i)$,
 simular $N_i(1^i 0)$ siguiendo el cómputo
 codificado por y (**)
 si no terminó: rechazar
 si terminó: aceptar sii y es no aceptador
 si $|y| > g(i)$, rechazar

Como es no-determinística, en (*) simulamos en tiempo $O(g(|x|))$.

En (**) simulamos en tiempo $O(|y|)$.

N corre en tiempo $O(g(n))$, luego $\mathcal{L}(N) \in \mathbf{NDTIME}(g(n))$.

6-16

Supongamos que $\mathcal{L}(N) \in \mathbf{NDTIME}(f(n))$

Sea N' una máquina no-determinística tal que

- $\mathcal{L}(N') = \mathcal{L}(N)$ y
- con entrada x suficientemente larga termina en $\leq c \cdot f(|x|)$ pasos.

Como $f(n+1) = o(g(n))$, para todo n suficientemente grande,

$$c \cdot f(n+1) \leq g(n).$$

Tomar i suficientemente grande tal que

- $N' = N_i$ y
- para todo $x \in \{0, 1\}^*$, si $|x| > i$ entonces $N_i(x0)$ y $N_i(x1)$ terminan en tiempo no-determinístico $g(|x|)$
 - $|x0| = |x1| = |x| + 1$ y N_i corre en tiempo $c \cdot f(n)$

Recordemos que N con entrada x hacía esto:

si x no es de la forma $1^i 0 y$, rechazar si no, supongamos $x = 1^i 0 y$ si $y < g(i)$, simular no determinísticamente $N_i(x0)$ y $N_i(x1)$ por $g(x)$ pasos si alguna no terminó: rechazar si las dos terminaron: aceptar sii $N_i(x0)$ acepta y $N_i(x1)$ acepta	si $y = g(i)$, simular $N_i(1^i 0)$ siguiendo el cómputo codificado por y si no terminó: rechazar si terminó: aceptar sii y es no aceptador si $y > g(i)$, rechazar
---	---

N_i es tal que $\mathcal{L}(N_i) = \mathcal{L}(N)$ y corre en tiempo $c \cdot f(n)$

Para $x = 1^i 0 y$, $N_i(x0)$ y $N_i(x0)$ terminan en $c \cdot f(|x| + 1) \leq g(|x|)$ pasos. $N_i(1^i 0)$ termina en $c \cdot f(i + 1) \leq g(i)$ pasos.

- $1^i 0 \in \mathcal{L}(N)$ sii $\forall y \in \{0, 1\}^1 \ 1^i 0 y \in \mathcal{L}(N_i)$
- sii $\forall y \in \{0, 1\}^2 \ 1^i 0 y \in \mathcal{L}(N_i)$
- ...
- sii $\forall y \in \{0, 1\}^{g(i)} \ 1^i 0 y \in \mathcal{L}(N_i)$
- sii $N_i(1^i 0)$ rechaza siguiendo todos los cálculos y
- tal que $|y| = g(i)$
- sii $1^i 0 \notin \mathcal{L}(N_i) = \mathcal{L}(N)$

☐

Teorema de Ladner

Clase 6

La jerarquía de tiempos determinísticos

La jerarquía de tiempos no-determinísticos

Teorema de Ladner

6-20

Problemas **NP**-intermedios

Muchos problemas **NP** terminan siendo **P** o **NP-completos**.

¿Les pasa esto a todos los problemas **NP**?

Teorema (Ladner)

Si $\mathbf{P} \neq \mathbf{NP}$ entonces existe $\mathcal{L}$ tal que $\mathcal{L} \in \mathbf{NP}$, $\mathcal{L} \notin \mathbf{P}$, y $\mathcal{L} \notin \mathbf{NP-completo}$.

6-21

Demostración.

Como siempre, sea M_i la i -ésima máquina determinística.
Para simplificar, escribimos ψ en lugar de $\langle \psi \rangle$.

Definimos el problema SAT_H relativo a la función $H : \mathbb{N} \rightarrow \mathbb{N}$:

$$\text{SAT}_H = \{\psi 01^{n^{H(n)}} : \psi \in \text{SAT}, n = |\psi|\}$$

- cuando H crece relativamente rápido, por ejemplo $H(n) = n$, entonces $\text{SAT}_H \in \mathbf{P}$: si $x_\psi = \psi 01^{|\psi|^{|\psi|}}$ entonces $|x_\psi| \geq 2^{|\psi|}$ para toda ψ salvo finitos casos. Dada x_ψ decidimos si ψ es satisfacible en tiempo $O(2^{|\psi|}) = O(|x_\psi|)$, o sea tiempo lineal.
- cuando H es constante, $\text{SAT}_H \in \mathbf{NP}$ -completo.

En la demostración que sigue, vamos a elegir una H adecuada:
 H tiende a infinito pero crece lentamente

6-22

Definimos H así:

$$H(n) = M_i(x) = \chi_{\text{SAT}_H}(x) \text{ en } \begin{cases} \text{mínimo } i < \log \log n \text{ tq } \forall x \in \{0,1\}^{\leq \log n}, \\ \leq i \cdot |x|^i \text{ pasos; o} \\ \log \log n \text{ si no existe tal } i \end{cases}$$

$$\text{SAT}_H = \{\psi 01^{n^{H(n)}} : \psi \in \text{SAT}, n = |\psi|\}$$

SAT_H y H están definidas por recursión mutua, pero están bien definidas:

- $H(n)$ usa $\text{SAT}_H(x)$ para palabras x de longitud $\leq \log n$
- $x \in \text{SAT}_H$ depende de $H(n)$ para $n < |x|$
(aquí $x = \psi 01^{n^{H(n)}}$, con $n = |\psi|$)

Ejercicio

H es computable en tiempo $O(n^3)$.

Entonces $\text{SAT}_H \in \mathbf{NP}$.

6-23

mínimo $i < \log \log n$ tq $\forall x \in \{0,1\}^{\leq \log n}$,
 $H(n) = M_i(x) = \chi_{\text{SAT}_H}(x)$ en $\leq i \cdot |x|^i$ pasos; o
 $\log \log n$ si no existe tal i

$$\text{SAT}_H = \{\psi 01^{n^{H(n)}} : \psi \in \text{SAT}, n = |\psi|\}$$

Proposición 1

$$\text{SAT}_H \in \mathbf{P} \implies \exists c \forall n H(n) \leq c$$

Demostración.

Supongamos que la máquina determinística M decide SAT_H en tiempo $c \cdot n^c$. Existe $i > c$ tal que $\mathcal{L}(M) = \mathcal{L}(M_i)$. Entonces para todo $n \geq 2^{2^i}$, tenemos que $H(n) \leq i$. Luego

$$H(n) \leq \max \left(\{H(m) : m < 2^{2^i}\} \cup \{i\} \right).$$

□

6-24

mínimo $i < \log \log n$ tq $\forall x \in \{0,1\}^{\leq \log n}$,
 $H(n) = M_i(x) = \chi_{\text{SAT}_H}(x)$ en $\leq i \cdot |x|^i$ pasos; o
 $\log \log n$ si no existe tal i

$$\text{SAT}_H = \{\psi 01^{n^{H(n)}} : \psi \in \text{SAT}, n = |\psi|\}$$

Proposición 2

$$\exists c \exists^\infty n H(n) \leq c \implies \text{SAT}_H \in \mathbf{P}.$$

Demostración.

Supongamos $\exists c \exists^\infty n H(n) \leq c$. Entonces $\exists i \exists^\infty n H(n) = i$. Veamos que M_i acepta SAT_H en tiempo $i \cdot n^i$: Sup. que M_i no acepta SAT_H . Existe x tal que $M_i(x) \neq \chi_{\text{SAT}_H}(x)$. Entonces $\forall n > 2^{|x|} H(n) \neq i$. Absurdo. Similar caso en que M_i no corre en tiempo $i \cdot n^i$. □

Contra-recíproco: $\text{SAT}_H \notin \mathbf{P} \implies \lim_{n \rightarrow \infty} H(n) = \infty$

6-25

Veamos que $\text{SAT}_H \notin \mathbf{P}$:

Supongamos $\text{SAT}_H \in \mathbf{P}$. Por la Proposición 1, $\exists c \forall n H(n) \leq c$.

$$\begin{aligned}\text{SAT}_H &= \{\psi 0 1^{n^{H(n)}} : \psi \in \text{SAT}, n = |\psi|\} \\ &= \{\psi 0 \underbrace{1 \dots 1}_{|\psi|^{H(|\psi|)} \leq |\psi|^c} : \psi \in \text{SAT}\}\end{aligned}$$

Pero entonces un algoritmo de tiempo polinomial para SAT_H se puede usar para SAT , por lo que $\text{SAT} \in \mathbf{P}$. Como SAT es **NP-completo** concluimos que $\mathbf{P} = \mathbf{NP}$. Absurdo.

6-26

Veamos que $\text{SAT}_H \notin \mathbf{NP-completo}$:

Supongamos $\text{SAT}_H \in \mathbf{NP-completo}$. Hay una reducción polinomial f de SAT a SAT_H :

$$\begin{aligned}\psi \in \text{SAT} &\quad \text{sii} \quad f(\psi) \in \text{SAT}_H \\ &\quad \text{sii} \quad f(\psi) \text{ es de la forma } F_\psi 0 1^{|F_\psi|^{H(|F_\psi|)}} \text{ con } F_\psi \in \text{SAT}.\end{aligned}$$

Supongamos que f es computable en tiempo $c \cdot n^c$ por una cierta máquina determinística. Como esa máquina no puede escribir más que $c \cdot n^c$ símbolos en la salida,

$$|f(x)| \leq c \cdot |x|^c$$

Luego

$$\begin{aligned}|f(\psi)| &= |F_\psi 0 1^{|F_\psi|^{H(|F_\psi|)}}| \\ &= |F_\psi| + 1 + |F_\psi|^{H(|F_\psi|)} \leq c \cdot |\psi|^c\end{aligned}$$

6-27

Sabemos:

- $\psi \in \text{SAT}$ sii $f(\psi) = F_\psi 01^{|F_\psi|^{H(|F_\psi|)}}$ con $F_\psi \in \text{SAT}$
- $|F_\psi| + 1 + |F_\psi|^{H(|F_\psi|)} \leq c \cdot |\psi|^c$

Proposición 3

Existe k tal que para todo ψ , $|\psi| > k \implies |F_\psi| < |\psi|$.

Demostración.

Supongamos que no. Para todo k , existe ψ tal que

$$|\psi| > k \quad \text{y} \quad |F_\psi| \geq |\psi| > k.$$

Como $\text{SAT}_H \notin \mathbf{P}$, por Prop. 2, $\lim_{n \rightarrow \infty} H(n) = \infty$.

Entonces $\exists k' \forall n > k' \quad n^{H(n)} > c \cdot n^c$

Existe ψ tal que $|F_\psi| \geq |\psi| > k'$. Luego

$$c \cdot |\psi|^c \leq c \cdot |F_\psi|^c < |F_\psi|^{H(|F_\psi|)} \leq c \cdot |\psi|^c$$

Absurdo. □

6-28

Algoritmo recursivo G para decidir $\psi \in \text{SAT}$:

entrada: ψ

si $|\psi| \leq k$,

devolver ‘sí’ si $\psi \in \text{SAT}$ y ‘no’ en caso contrario
(finitos casos porque k es constante)

si no,

computar $f(\psi)$

si $f(\psi)$ no es de la forma $F_\psi 01^{|F_\psi|^{H(|F_\psi|)}}$,

devolver ‘no’

si no, *devolver* $G(F_\psi)$ (por Prop. 3, $|F_\psi| < |\psi|$)

Notar que G corre en tiempo polinomial y

$$\psi \in \text{SAT} \quad \text{sii} \quad G(\psi) \in \text{SAT}$$

Entonces $\mathbf{P} = \mathbf{NP}$. Absurdo. □

6-29

Clase 7

Clase 1

Preliminares
Representaciones en binario
Problemas de decisión
Máquinas de Turing
Configuraciones y cálculos

Clase 2

Funciones computables
Tiempo de cálculo
Codificación de máquinas
Variantes de máquinas

Clase 3

Halt y máquina universal
La clase **P**
La clase **NP**

Clase 4

Máquinas no-determinísticas
Reducibilidad polinomial
Lógica proposicional

Clase 5

Mini-configuraciones
Teorema de Cook-Levin
Problemas **NP-completos**
La clase **coNP**
Las clases **EXPTIME** y **NEXPTIME**

Clase 6

La jerarquía de tiempos determinísticos
La jerarquía de tiempos no-determinísticos
Teorema de Ladner

Clase 7

Espacio usado por un cálculo
Espacio polinomial: $\mathbf{PSPACE}$ y $\mathbf{NPSpace}$
Teorema de Savitch

Clase 8

Espacio logarítmico: **L** y **NL**
Teorema de Immerman-Szelepcsényi

Clase 9

La jerarquía polinomial
Problemas Σ_i^P -**completos**

Clase 10

Máquinas con oráculo
Teorema de Baker, Gill, Solovay
La jerarquía polinomial y **NP** con oráculos

Clase 11

Circuitos booleanos
La clase **P**/**poly**
Tamaño y profundidad de circuitos

Clase 12

Las clases **NC_{nu}** y **AC_{nu}**
Uniformidad: las clases **NC** y **AC**
P-completitud

7-1

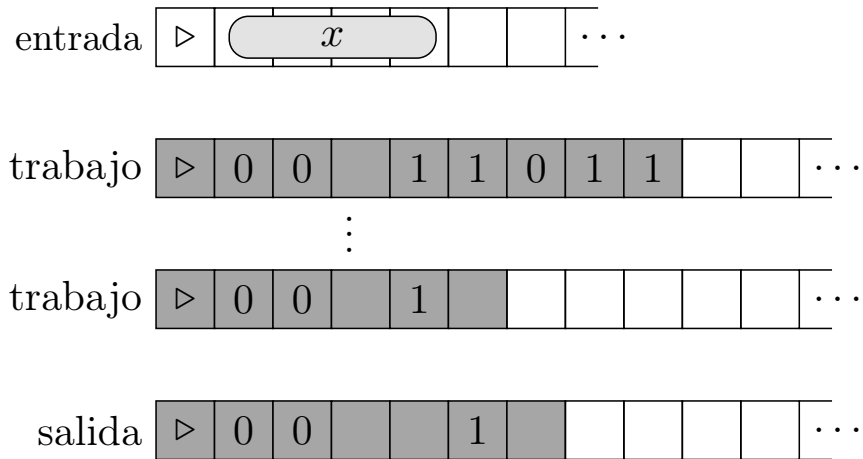
Espacio usado por un cálculo

Clase 7

Espacio usado por un cálculo
Espacio polinomial: **PSPACE** y **NPSpace**
Teorema de Savitch

7-2

Espacio en una máquina determinística



Definición

Dada una máquina determinística M y $x \in \{0, 1\}^*$ el **espacio** que usa M con entrada x es la cantidad de celdas por las que alguna vez pasó la cabeza en las cintas de trabajo y de salida a lo largo del cómputo de M a partir de x .

7-3

Espacio en una máquina no-determinística

Definición

Dada una máquina no-determinística N y $x \in \{0, 1\}^*$ el **espacio** que usa N con entrada x es la cantidad de celdas por las que alguna vez pasó la cabeza en las cintas de trabajo y de salida a lo largo de *todos* los cálculos de N a partir de x .

7-4

Espacio usado en una máquina

Definición

La máquina M (determinística o no-determinística)

- **usa espacio $S(n)$** si para toda entrada x , el espacio que usa M con entrada x es a lo sumo $S(|x|)$
- **usa espacio $O(S(n))$** si existe una constante c tal que para todo x , salvo finitos, M con entrada x usa espacio $c \cdot S(|x|)$

Definición

Una función f es **computable en espacio $S(n)$, $O(S(n))$** si existe una máquina determinística que computa f y usa espacio $S(n)$, $O(S(n))$.

7-5

SPACE($S(n)$) y NSPACE($S(n)$)

Clase de complejidad: SPACE($S(n)$), NSPACE($S(n)$)

SPACE($S(n)$) es la clase de lenguajes $\mathcal{L}$ tal que existe una máquina determinística M tal que

- M decide $\mathcal{L}$
- M usa espacio $O(S(n))$

NSPACE($S(n)$) es la clase de lenguajes $\mathcal{L}$ tal que existe una máquina no-determinística N tal que

- N decide $\mathcal{L}$
- N usa espacio $O(S(n))$

7-6

Funciones construibles en espacio

Definición

Una función $S : \mathbb{N} \rightarrow \mathbb{N}$ es **construible en espacio** si la función $1^n \mapsto [S(n)]$ es computable en espacio $O(S(n))$

Ejemplos de funciones construibles en espacio

$\log n, n, n^2, 2^n$

- No eran interesantes las funciones $T(n) < n$ cuando medíamos el tiempo, porque se espera que las máquinas al menos lean la entrada y eso lleva tiempo n .
- Cuando medimos espacio, la cosa cambia. Como no medimos el espacio de la entrada, tiene sentido estudiar funciones de espacio $S(n) < n$.
- Pero a veces vamos a requerir que $S(n) \geq \log n$, dado que queremos poder ‘recordar’ cualquier *índice* de la cinta de entrada.
 - La entrada tiene largo n , entonces representamos cualquier $i \leq n$ con $\leq \log n$ bits.

7-7

$$\text{DTIME}(S(n)) \subseteq \text{SPACE}(S(n)) \subseteq \text{NSPACE}(S(n))$$

Proposición

$\text{DTIME}(S(n)) \subseteq \text{SPACE}(S(n))$.

Demostración.

En cada paso, una máquina determinística solo puede usar una celda más de cada cinta. Si M corre en tiempo $O(S(n))$, usa espacio $O(S(n))$. □

Proposición

$\text{SPACE}(S(n)) \subseteq \text{NSPACE}(S(n))$.

7-8

Grafo de configuraciones

Definición

Sea $S : \mathbb{N} \rightarrow \mathbb{N}$ y sea M una máquina (determinística o no-determinística) que usa espacio $S(n)$. El **grafo de configuraciones** de M con entrada x , notado $G_{M,x}$ es un grafo dirigido tal que

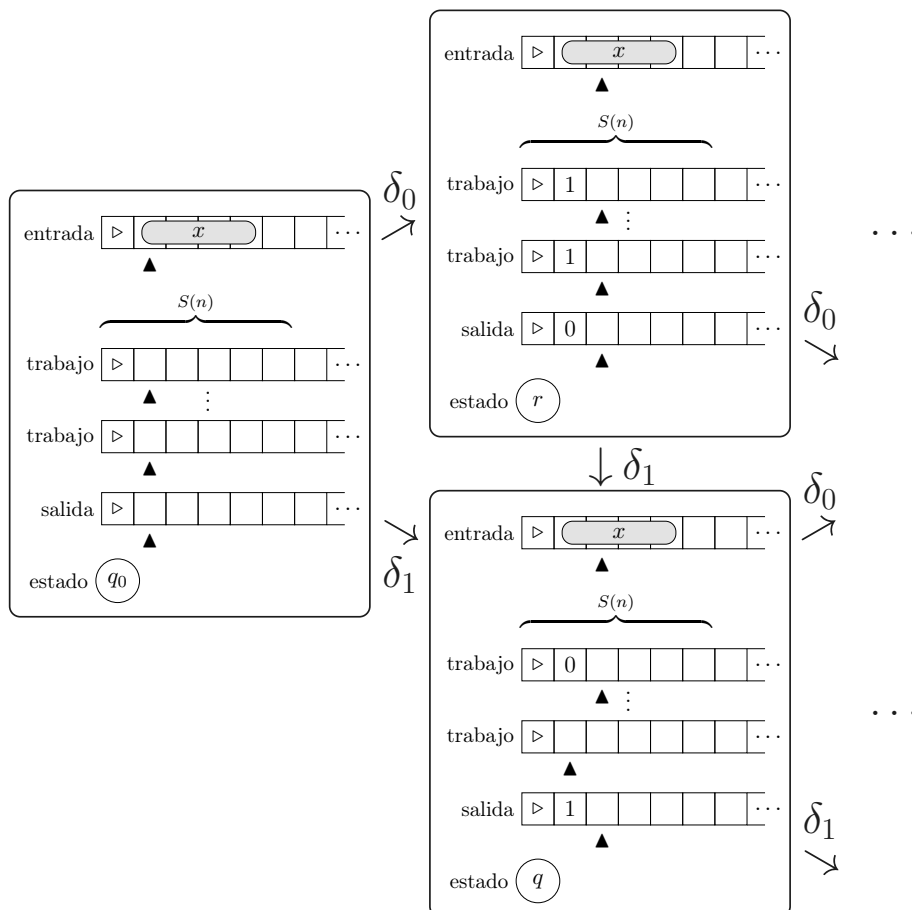
- el conjunto de vértices son las configuraciones de hasta $S(|x|)$ celdas en cada cinta de trabajo y en la de salida
- hay una arista de C a C' si C' es una evolución en un paso de C dado por M

Observación

Si M es una máquina determinística o no-determinística, M acepta x si y solo si existe un camino en $G_{M,x}$ desde la configuración inicial de M para x hasta alguna configuración final.

7-9

Grafo de configuraciones



7-10

Grafo de configuraciones

Como siempre, estamos interesados en máquinas que aceptan lenguajes.

- si M es determinística
 - $G_{M,x}$ tiene *out-degree* 1
 - podemos suponer que M borra todas las cintas de trabajo y deja todas las cabezas en la primera celda antes de entrar a q_f . Así hay un *único* estado final $C_{\text{sí}}$ aceptador: es el que tiene el estado en q_f y en la cinta de salida hay un 1 seguido de blancos
- si N es no-determinística
 - $G_{N,x}$ tiene *out-degree* ≤ 2
 - podemos suponer que N borra todas las cintas de trabajo y salida y deja todas las cabezas en la primera celda antes de entrar a $q_{\text{sí}}$. Así hay un único estado final $C_{\text{sí}}$ aceptador.

7-11

Codificación de configuraciones

Proposición

Sea M una máquina (determinística o no-determinística) que usa espacio $S(n)$ tal que $S(n) \geq \log n$. Existe una constante c tal que para todo x , cada vértice de $G_{M,x}$ se puede describir usando $c \cdot S(|x|)$ bits (c depende de M). Luego, $G_{M,x}$ tiene $2^{c \cdot S(|x|)}$ nodos.

7-12

Demostración.

Sea $M = (\Sigma, Q, \delta)$ con k cintas de trabajo.

- numeramos los estados de Q y usamos la codificación usual (codificamos q con $\langle q \rangle$)
- contenido de i -ésima cinta de trabajo T_i :
 - contenido: $\triangleright b_1^i b_2^i \dots b_{S(|x|)}^i$ (codificado en binario)
 - posición cabeza: $0 \leq j_i \leq S(|x|)$
 - codificación: $\langle T_i \rangle = 0b_1^i \dots 0b_{j_i-1}^i 10 0b_{j_i}^i \dots 0b_{S(|x|)}^i$
- configuración de la cinta de salida Z : ídem, $\langle Z \rangle$
- configuración de la cinta de entrada E :
 - solo guardamos la *posición* p de la cabeza (en binario): $\langle E \rangle = \langle p \rangle$. No usamos la misma configuración que antes porque podría ser $n > S(n)$

Sea $n = |x|$. Cada C se codifica con $\langle C \rangle = \langle \langle q \rangle, \langle E \rangle, \langle T_1 \rangle, \dots, \langle T_k \rangle, \langle Z \rangle \rangle$.

$$|C| = d \cdot ((k+1) \cdot S(n) + \log n) = c \cdot S(n).$$

Cada $C \in \{0,1\}^*$, luego hay $2^{c \cdot S(n)}$ posibles configuraciones (para c dependiente de M pero independiente de x). □

7-13

$$\text{NSPACE}(S(n)) \subseteq \text{DTIME}(2^{O(S(n))})$$

Proposición

Si S es construible en espacio,
 $\text{NSPACE}(S(n)) \subseteq \text{DTIME}(2^{O(S(n))})$.

Demostración.

Sea N una máquina no-determinística que decide $\mathcal{L}$ en espacio $O(S(n))$. Definimos la máquina determinística M con entrada x :

Construir $G_{N,x}$ (toma tiempo $2^{O(S(n))}$). Usar BFS para decidir si existe un camino entre la configuración inicial de N a partir de x y la (única) configuración final. Si existe tal camino, escribir 1 en la salida; si no escribir 0. Luego pasar a q_f .

Para cualquier grafo $G = (V, E)$, BFS corre en tiempo $O(|V| + |E|)$. Como el *out-degree* de cada vértice de $G_{N,x}$ es ≤ 2 , la cantidad de aristas de $G_{N,x}$ es $O(2^{O(S(|x|))})$. Luego BFS sobre $G_{N,x}$ toma tiempo $O(2^{O(S(|x|))})$.

Entonces $\mathcal{L}(M) = \mathcal{L}$ y M corre en tiempo $O(2^{O(S(n))})$. □

7-14

Espacio polinomial: **PSPACE** y **NPSpace**

Clase 7

Espacio usado por un cómputo

Espacio polinomial: **PSPACE** y **NPSpace**

Teorema de Savitch

7-15

PSPACE, **NPSpace**

Clase de complejidad: PSPACE y NPSpace
$\mathbf{PSPACE} = \bigcup_{c>0} \mathbf{SPACE}(n^c)$ $\mathbf{NPSpace} = \bigcup_{c>0} \mathbf{NSpace}(n^c)$

Observación
$\mathbf{PSPACE} \subseteq \mathbf{NPSpace} \subseteq \mathbf{EXPTIME}.$

7-16

$\mathbf{NP} \subseteq \mathbf{PSPACE}$

Proposición

$\mathbf{NP} \subseteq \mathbf{PSPACE}$.

Demostración.

Supongamos $\mathcal{L} \in \mathbf{NP}$. Existe una máquina determinística M y un polinomio $p : \mathbb{N} \rightarrow \mathbb{N}$ tal que M corre en tiempo polinomial y para todo x :

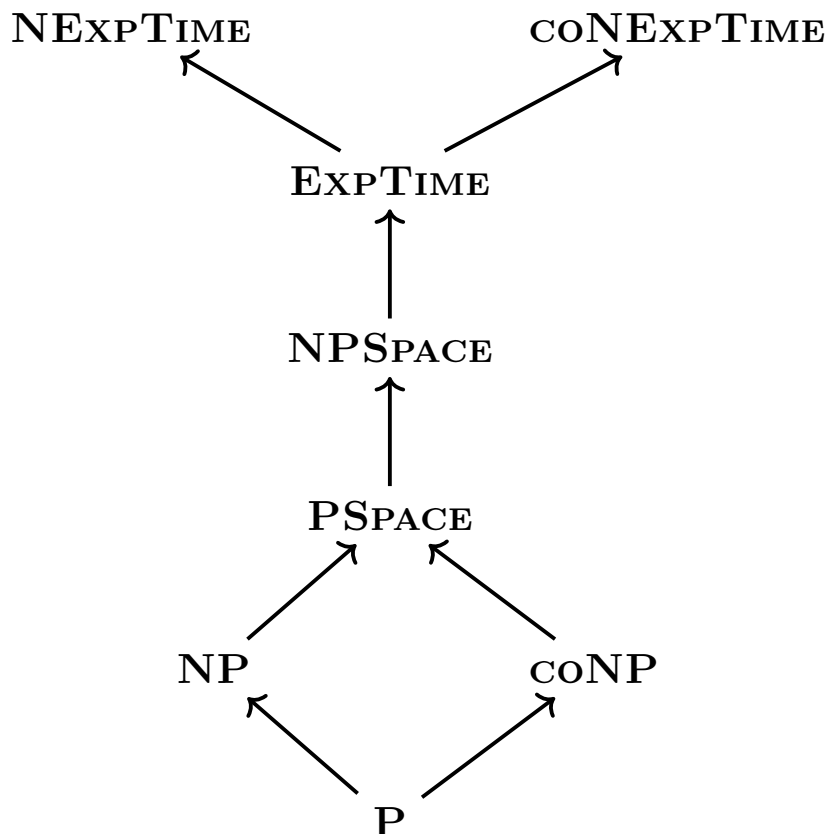
$$x \in \mathcal{L} \quad \text{sii} \quad \text{existe } u \in \{0, 1\}^{p(|x|)} \text{ tal que } M(\langle x, u \rangle) = 1$$

Definimos una máquina determinística M' que con entrada x hace esto:

Simula $M(\langle x, u \rangle)$ para cada $u \in \{0, 1\}^{p(|x|)}$. Cada u lo escribe en las mismas $p(|x|)$ celdas. Si encuentra u tal que $M(\langle x, u \rangle) = 1$, escribe 1 en la salida. Si no, escribe 0. Luego pasa a q_f .

M' corre en tiempo exponencial pero usa espacio $p(|x|)$ más el que use M con entrada $\langle x, u \rangle$. Luego $\mathcal{L} \in \mathbf{PSPACE}$. $\square$

7-17



7-18

La jerarquía de espacios

Teorema

Si f, g son construibles en espacio y cumplen que

$$f(n) = o(g(n))$$

entonces $\mathbf{SPACE}(f(n)) \subsetneq \mathbf{SPACE}(g(n))$.

Ejercicio

Demostrarlo. Misma idea que para la jerarquía de tiempos determinísticos, solo que podemos simular cualquier máquina con un factor constante de costo extra en lugar de logarítmico.

7-19

Fórmulas booleanas cuantificadas (QBF)

Definición

Una **fórmula booleana cuantificada** (o **QBF**) es una expresión de la forma

$$Q_1x_1 \ Q_2x_2 \ \dots \ Q_nx_n \ \varphi(x_1, x_2, \dots, x_n)$$

donde

- Q_i es $\forall$ o $\exists$
- $\varphi(x_1, x_2, \dots, x_n)$ es una fórmula booleana con variables entre $x_1, \dots, x_n$ y posiblemente con constantes 0, 1

Los cuantificadores $\forall$ o $\exists$ tienen la semántica usual de ‘para todo’ y ‘existe’, pero las variables x_i son *booleanas*, es decir solo pueden tomar valor 0 (falso) o 1 (verdadero).

Las QBF están en forma *prenexa* (cuantificadores adelante)

7-20

Fórmulas booleanas cuantificadas (QBF)

Ejemplo

$$\psi = \forall x_1 \exists x_2 \forall x_3 (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3)$$

es falsa: si x_1 es verdadera, entonces x_3 tiene que ser verdadera, pero x_3 está cuantificada universalmente.

7-21

Fórmulas booleanas cuantificadas (QBF)

El problema TQBF

Notar que todas las variables de una QBF están cuantificadas (se llaman *sentencias*), entonces son verdaderas o falsas independiente de cualquier asignación de variables.

Notación: $\models$

Sea ψ una QBF. Notamos

- $\models \psi$ cuando ψ es verdadera
- $\not\models \psi$ cuando ψ es falsa

Problema: TQBF (*True Quantified Boolean Formula*)

$$\text{TQBF} = \{\langle \psi \rangle : \psi \text{ es una QBF tal que } \models \psi\}$$

7-22

Fórmulas booleanas cuantificadas (QBF)

Eliminación de cuantificadores

Supongamos la QBF

$$\psi = Q_1 x_1 \overbrace{Q_2 x_2 \dots Q_n x_n}^{\rho} \varphi(x_1, x_2, \dots, x_n)$$

Para $b \in \{0, 1\}$, definimos $\psi \upharpoonright_{x_1=b}$ como el resultado de reemplazar en ρ todas las ocurrencias de x_1 por b .

Ejemplo

$$(\forall y \exists x \ y \wedge x) \upharpoonright_{y=1} = \exists x \ 1 \wedge x.$$

- si $Q_1 = \forall$ entonces $\models \psi$ sii $\models \psi \upharpoonright_{x_1=0}$ y $\models \psi \upharpoonright_{x_1=1}$
- si $Q_1 = \exists$ entonces $\models \psi$ sii $\models \psi \upharpoonright_{x_1=0}$ o $\models \psi \upharpoonright_{x_1=1}$

7-23

TQBF $\in$ PSPACE

Teorema

TQBF $\in$ PSPACE.

7-24

Demostración.

Definimos una función recursiva F con

- entrada: (codificación de) una QBF

$$\psi = Q_1x_1 \ Q_2x_2 \dots Q_nx_n \ \varphi(x_1, x_2, \dots, x_n)$$

- salida: 1 si $\models \psi$ y 0 en caso contrario.

*si $n = 0$, ψ no tiene variables (solo tiene constantes)
decidir si es verdadera o falsa (tiempo $O(|\psi|)$)*

si no, ψ es de la forma $\psi = Qx \dots$

*computar $i = F(\psi \upharpoonright_{x=0})$ y $j = F(\psi \upharpoonright_{x=1})$
(se reutiliza el espacio en los dos llamados)*

solo nos quedamos con los bits i, j

si $Q = \forall$, devolver 1 si $i = j = 1$; si no, 0

si $Q = \exists$, devolver 1 si $i = 1$ o $j = 1$; si no, 0

Necesita espacio $O(|\psi|)$ para escribir $\psi \upharpoonright_{x=b}$ y en cada llamado decrementamos el tamaño de la entrada. En total usa espacio $O(|\psi|^2)$. □

7-25

Fórmulas booleanas cuantificadas (QBF)

QBFs con variables libres

Podemos extender las QBFs con variables libres:

$$\psi(y_1, \dots, y_m) = Q_1x_1 \ Q_2x_2 \dots Q_nx_n \ \varphi(x_1, x_2, \dots, x_n, y_1, \dots, y_m)$$

donde

- Q_i es $\forall$ o $\exists$
- $\psi(x_1, x_2, \dots, x_n, y_1, \dots, y_m)$ es una fórmula booleana con variables entre $x_1, \dots, x_n, y_1, \dots, y_m$
- $y_1, \dots, y_m$ son variables libres de ψ
- $x_1, \dots, x_n$ son variables ligadas de ψ

Cuando hay variables libres, la verdad o falsedad depende de una valuación $\{y_1, \dots, y_m\} \rightarrow \{0, 1\}$. Como con las CNF:

- representamos valuaciones $\{y_1, \dots, y_m\} \rightarrow \{0, 1\}$ con palabras v en $\{0, 1\}^m$.
- notamos $v \models \psi(y_1, \dots, y_m)$ cuando ψ es verdadera para v
- notamos $v \not\models \psi(y_1, \dots, y_m)$ cuando ψ es falsa para v

7-26

Fórmulas booleanas cuantificadas (QBF)

Sin la restricción de forma prenexa

Ejemplo

$$\psi(y_1, y_2) = \exists x (x \leftrightarrow y_1) \wedge (x \leftrightarrow y_2)$$

(acá $u \leftrightarrow v$ es una abreviatura para $(u \wedge v) \vee (\neg u \wedge \neg v)$)

$00 \models \psi(y_1, y_2)$, $11 \models \psi(y_1, y_2)$, $01 \not\models \psi(y_1, y_2)$

Podemos considerar también una gramática más general:

$$\varphi ::= x \mid 0 \mid 1 \mid \neg \varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid \forall x \varphi \mid \exists x \varphi$$

con la semántica esperada. Llamémoslas **QBF generalizadas**

No son QBFs porque no están en forma prenexa.

Ejemplo

$$\psi(y) = \exists x_1 (\forall x_2 (y \wedge \neg x_1 \wedge x_2) \vee \forall x_3 (\neg y \vee x_1 \vee x_3))$$

no es una QBF pero es una QBF generalizada.

7-27

Fórmulas booleanas cuantificadas (QBF)

Sin la restricción de forma prenexa

Proposición

En tiempo polinomial se puede transformar cualquier QBF generalizada $\psi(y_1, \dots, y_m)$ en una QBF $\psi'(y_1, \dots, y_m)$ equivalente, es decir, tal que para todo $v \in \{0, 1\}^m$

$$v \models \psi(y_1, \dots, y_m) \quad \text{sii} \quad v \models \psi'(y_1, \dots, y_m)$$

Ejercicio

Demostrarlo. Idea: renombrar variables ligadas para no repetir y aplicar reglas:

$$\neg \forall x \psi = \exists x \neg \psi$$

$$\neg \exists x \psi = \forall x \neg \psi$$

$$\psi * (Qx \varphi(x)) = Qx (\varphi * \psi)$$

($Q \in \{\forall, \exists\}$, $*$ $\in \{\wedge, \vee\}$, ψ no contiene a x libre)

7-28

Chequeo de QBF generalizada

Problema: CHECKQBF (*Chequeo de QBF generalizada*)

$\text{CHECKQBF} = \{ \langle \psi, v \rangle : \begin{array}{l} \psi \text{ es una QBF generalizada con } m \text{ variables} \\ \text{libres, } v \in \{0, 1\}^m \text{ y } v \models \psi \end{array} \}$

Proposición

$\text{CHECKQBF} \leq_p \text{TQBF}$.

Demostración.

Sea ψ una QBF generalizada con variables libres $y_1, \dots, y_m$ y sea $v \in \{0, 1\}^m$. Primero llevamos ψ a una ψ' equivalente pero en forma prenexa (tiempo polinomial). Luego definimos ψ'' como el resultado de reemplazar cada variable y_j en ψ' por la constante $v(j)$; nos queda una QBF tal que $v \models \psi(y_1, \dots, y_m)$ sii $\models \psi''$. $\square$

7-29

‘Evolución en un paso’ en CNF

Proposición

Sea M una máquina (determinística o no-determinística) que usa espacio $S(n) \geq \log n$ y sea c una constante tal que para todo $x \in \{0, 1\}^*$ cualquier configuración C de un cómputo de M con entrada x se representa con $|\langle C \rangle| = c \cdot S(|x|)$ bits.

Dado x , podemos computar en tiempo polinomial en $S(n)$ una fórmula $\varphi_{M,x}(\bar{s}, \bar{t})$ en CNF con variables libres

$\bar{s} = s_1, \dots, s_{c \cdot S(|x|)}$, $\bar{t} = t_1, \dots, t_{c \cdot S(|x|)}$ tal que

$\langle C \rangle \langle C' \rangle \models \varphi_{M,x}(\bar{s}, \bar{t})$ sii hay una flecha de C a C' en $G_{M,x}$

Ejercicio

Demostrarlo (parecido a lo que hicimos con la demostración del teorema de Cook-Levin).

7-30

TQBF $\in$ PSPACE-hard

Teorema

TQBF $\in$ PSPACE-hard

7-31

Demostración.

Sea $\mathcal{L} \in \mathbf{PSPACE}$ y sea M una máquina determinística que decide $\mathcal{L}$ usando espacio $S(n)$, con S un polinomio.

Consideramos $G_{M,x}$. Cada vértice $\langle C \rangle$ (C es una configuración) de $G_{M,x}$ se representa con $c \cdot S(n)$ bits, donde $n = |x|$. Sea C_0 la configuración inicial y C_f la final del cómputo de M a partir de x .

Definimos fórmulas $\psi_i(\bar{s}, \bar{t})$ con variables libres $\bar{s}, \bar{t}$, que son tuplas de dimensión $c \cdot S(|x|)$.

$$\langle C \rangle \langle C' \rangle \models \psi_i(\bar{s}, \bar{t}) \quad \text{sii} \quad \begin{array}{l} \text{existe un camino dirigido de longi-} \\ \text{tud} \leq 2^i \text{ en } G_{M,x} \text{ entre } C \text{ y } C' \end{array}$$

Sea $n = |x|$. Si hay un camino de C a C' en $G_{M,x}$, entonces hay uno de longitud $\leq 2^{c \cdot S(n)}$, porque $G_{M,x}$ tiene $\leq 2^{c \cdot S(n)}$ vértices.

7-32

Luego,

$$\begin{aligned} x \in \mathcal{L} \quad \text{sii} \quad \langle C_0 \rangle \langle C_f \rangle &\models \psi_{c.S(n)}(\bar{s}, \bar{t}) \\ \text{sii} \quad \langle \psi_{c.S(n)}(\bar{s}, \bar{t}), \langle C_0 \rangle \langle C_f \rangle \rangle &\in \text{CHECKQBF} \end{aligned}$$

Entonces

$$\mathcal{L} \leq_p \text{CHECKQBF} \leq_p \text{TQBF}$$

siempre y cuando podamos definir $\psi_{c.S(n)}$ en tiempo polinomial en $n = |x|$. Definimos $\psi_i(\bar{s}, \bar{t})$ por recursión en i .

- $\psi_0(\bar{s}, \bar{t})$ se define como $\bar{s} = \bar{t} \vee \varphi_{M,x}(\bar{s}, \bar{t})$. Computamos ψ_0 en tiempo polinomial en n .
($\bar{s} = \bar{t}$ abrevia $\bigwedge_{i=1}^{2^{c.S(n)}} (s_i \wedge t_i) \vee (\neg s_i \wedge \neg t_i)$)
- Para $i > 0$ podemos definir

$$\psi_i(\bar{s}, \bar{t}) = \exists \bar{r} \, \psi_{i-1}(\bar{s}, \bar{r}) \wedge \psi_{i-1}(\bar{r}, \bar{t})$$

pero el tamaño de ψ_i sería exponencial en i y por lo tanto no podríamos computarla en tiempo polinomial. No sirve.

7-33

- Para $i > 0$, cambiamos por:

$$\begin{aligned} \psi_i(\bar{s}, \bar{t}) &= \exists \bar{r} \forall \bar{u}, \bar{v} \\ &((\bar{u} = \bar{s} \wedge \bar{v} = \bar{r}) \vee (\bar{u} = \bar{r} \wedge \bar{v} = \bar{t})) \rightarrow \psi_{i-1}(\bar{u}, \bar{v}) \end{aligned}$$

($\phi \rightarrow \rho$ abrevia $\neg \phi \vee \rho$)

$$\begin{array}{c} (\bar{u} = \bar{s} \wedge \bar{v} = \bar{r}) \rightarrow \psi_{i-1}(\bar{u}, \bar{v}) \\ \underbrace{\bar{s} \rightarrow \dots \rightarrow \bar{r}}_{(\bar{u} = \bar{r} \wedge \bar{v} = \bar{t}) \rightarrow \psi_{i-1}(\bar{u}, \bar{v})} \rightarrow \dots \rightarrow \bar{t} \end{array}$$

- Para computar ψ_i necesitamos i llamados recursivos hasta llegar al caso base.
- Entonces computamos $\psi_{c.S(n)}$ en tiempo polinomial en $S(n)$, que es polinomial en n .
- $\psi_{c.S(n)}$ es una QBF generalizada. En tiempo polinomial la convertimos en QBF.

□

7-34

Corolario
TQBF $\in$ PSPACE-completo.

7-35

Teorema de Savitch

Clase 7

Espacio usado por un cómputo

Espacio polinomial: **PSPACE** y **NPSpace**

Teorema de Savitch

7-36

Teorema de Savitch

Teorema (Savitch)

$\mathbf{NSPACE}(S(n)) \subseteq \mathbf{SPACE}(S(n)^2)$.

Corolario

$\mathbf{PSPACE} = \mathbf{NPSpace}$.

7-37

Demostración de $\mathbf{NSPACE}(S(n)) \subseteq \mathbf{SPACE}(S(n)^2)$.

Parecido a la demostración que vimos recién.

Sea $\mathcal{L} \in \mathbf{NSPACE}(S(n))$, sea N una máquina no-determinística tal que para todo x , N acepta x sii $x \in \mathcal{L}$ sii hay un camino de longitud $\leq 2^{c \cdot S(|x|)}$ en $G_{N,x}$ desde C_0 hasta C_f .

Definimos una función booleana recursiva (traducible a una máquina determinística) $Reach$ tal que $Reach(C, C', i) = 1$ si hay un camino de longitud $\leq 2^i$ en $G_{N,x}$ desde C hasta C' . Luego

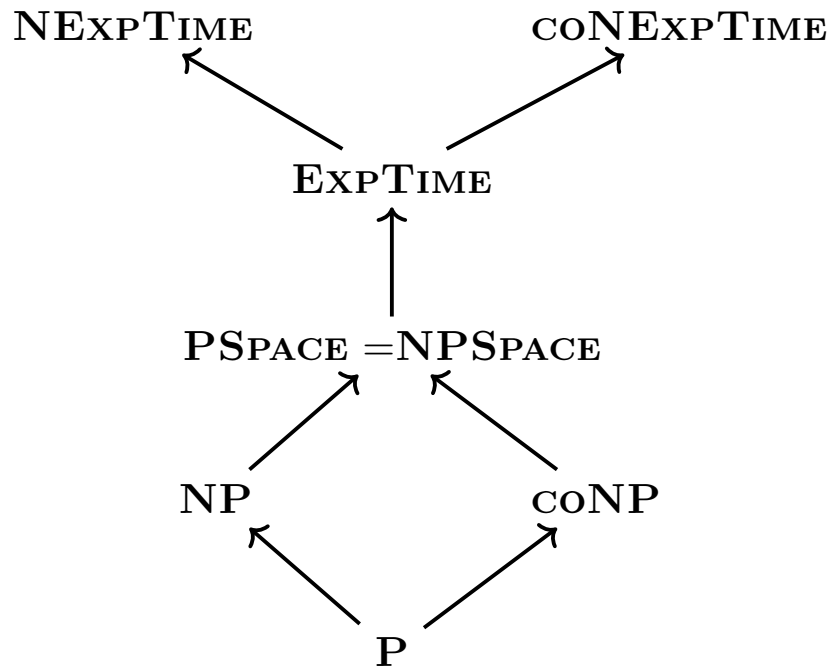
$$x \in \mathcal{L} \quad \text{sii} \quad Reach(C_0, C_f, c \cdot S(n)) = 1.$$

$Reach(C, C', i)$ hace esto:

si $i = 0$, devolver 1 si $C = C'$ o $C \rightarrow C'$ en $G_{N,x}$; 0 si no
para cada posible configuración C'' de $G_{N,x}$:
 $j = Reach(C, C'', i - 1)$; $k = Reach(C'', C', i - 1)$
 si $j = k = 1$, devolver 1
devolver 0

Sea $n = |x|$. $Reach(C, C', i)$ enumera las configuraciones C'' usando espacio $O(S(n))$. Reusa el espacio para calcular j y k . Entonces $Reach(C, C', i)$ usa espacio $O(i \cdot S(n))$. Luego, computamos $Reach(C_0, C_f, c \cdot S(n))$ usando espacio $O(S(n)^2)$. □

7-38



7-39

Clase 8

Clase 1

Preliminares
Representaciones en binario
Problemas de decisión
Máquinas de Turing
Configuraciones y cálculos

Clase 2

Funciones computables
Tiempo de cómputo
Codificación de máquinas
Variantes de máquinas

Clase 3

Halt y máquina universal
La clase **P**
La clase **NP**

Clase 4

Máquinas no-determinísticas
Reducibilidad polinomial
Lógica proposicional

Clase 5

Mini-configuraciones
Teorema de Cook-Levin
Problemas **NP-completos**
La clase **coNP**
Las clases **EXPTIME** y **NEXPTIME**

Clase 6

La jerarquía de tiempos determinísticos
La jerarquía de tiempos no-determinísticos
Teorema de Ladner

Clase 7

Espacio usado por un cómputo
Espacio polinomial: **PSPACE** y **NPSpace**
Teorema de Savitch

Clase 8

Espacio logarítmico: L y NL
Teorema de Immerman-Szelepcsényi

Clase 9

La jerarquía polinomial
Problemas Σ_i^P -completos

Clase 10

Máquinas con oráculo
Teorema de Baker, Gill, Solovay
La jerarquía polinomial y **NP** con oráculos

Clase 11

Circuitos booleanos
La clase **P/poly**
Tamaño y profundidad de circuitos

Clase 12

Las clases **NC_{nu}** y **AC_{nu}**
Uniformidad: las clases **NC** y **AC**
P-completitud

Espacio logarítmico: **L** y **NL**

Clase 8

Espacio logarítmico: **L** y **NL**

Teorema de Immerman-Szelepcsényi

8-2

L y **NL**

Clase de complejidad: **L**, **NL**

$$\mathbf{L} = \mathbf{SPACE}(\log n)$$

$$\mathbf{NL} = \mathbf{NSPACE}(\log n)$$

Observación

$$\mathbf{L} \subseteq \mathbf{NL} \subseteq \mathbf{P}.$$

Demostración.

Si S es construible en espacio, sabemos que

$$\mathbf{NSPACE}(S(n)) \subseteq \mathbf{DTIME}(2^{O(S(n))}). \text{ Tomar } S(n) = \log n. \quad \square$$

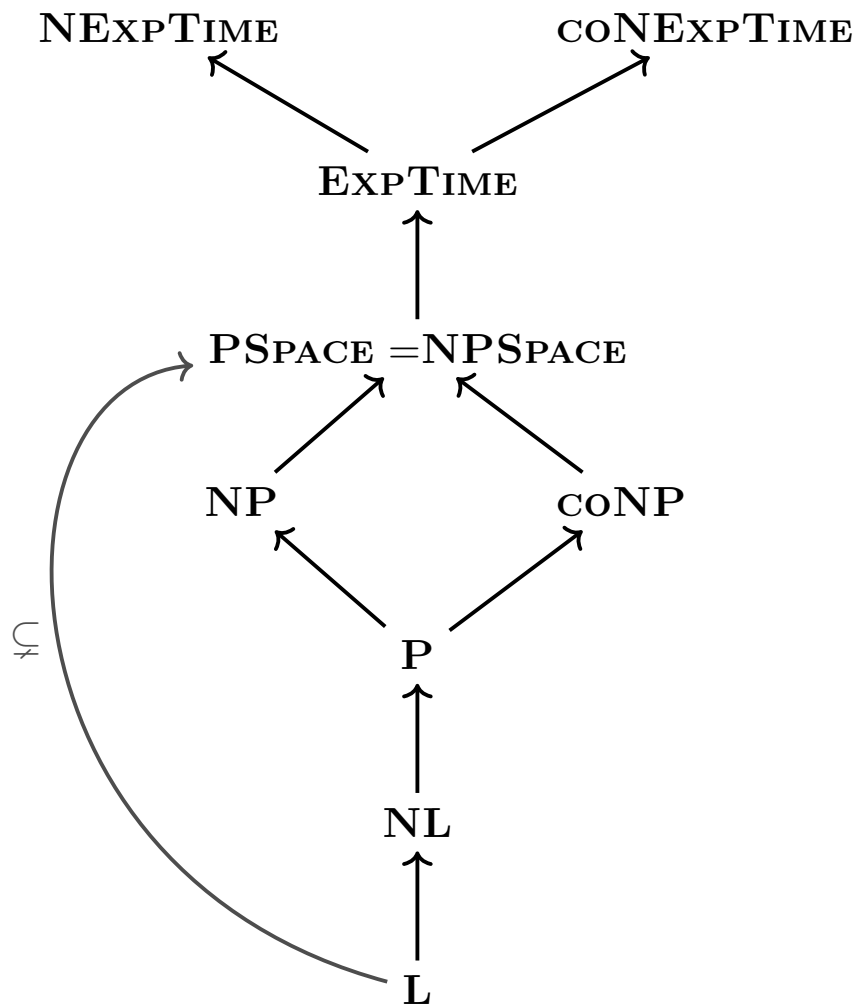
Problema: **EVEN** (cantidad par de 1s)

$$\mathbf{EVEN} = \{x \in \{0, 1\}^* : \text{hay una cantidad par de 1s en } x\}$$

Ejercicio

$$\mathbf{EVEN} \in \mathbf{L}.$$

8-3



8-4

Ejemplo de problema en **NL**

Problema: PATH (existencia de camino)

$\text{PATH} = \{\langle G, s, t \rangle : \text{hay un camino de } s \text{ a } t \text{ en el grafo dirigido } G\}$

No se sabe si $\text{PATH} \in \mathbf{L}$, pero

Proposición

$\text{PATH} \in \mathbf{NL}$.

8-5

Demostración.

Suponemos que los nodos de $G = \langle V, E \rangle$ están codificados como números del 0 a $|V| - 1$.

Definimos la máquina no-determinística N que con entrada $x = \langle G, s, t \rangle$ inventa un camino de s a t :

$y \leftarrow s; m \leftarrow 0$
mientras $m < |V|$:
 $z \leftarrow$ inventar un valor en $\{0, \dots, |V| - 1\}$
 si $(y, z) \notin E$ (para esto, revisa G), pasar a q_{no}
 si no:
 si $z = t$, pasar a $q_{sí}$
 si no:
 $y \leftarrow z; m \leftarrow m + 1$
 pasar a q_{no}

Las variables y, z, m son posiciones contiguas de celdas en la cinta de trabajo. Solo almacenan números $\leq |V|$, de modo que alcanzan $\log |V|$ celdas para cada variable. Entonces M usa espacio $O(\log n)$.

N acepta x sii $x \in \text{PATH}$,

luego $\text{PATH} \in \mathbf{NL}$. □

8-6

Reducibilidad para $\mathbf{NL}$

Para la pregunta $\mathbf{P} \stackrel{?}{=} \mathbf{NP}$ usamos la (Karp) reducibilidad polinomial $\leq_p$.

Para la pregunta $\mathbf{L} \stackrel{?}{=} \mathbf{NL}$ no nos sirve $\leq_p$:

Proposición

Si $\mathcal{L} \notin \{\{0, 1\}^*, \emptyset\}$ entonces $\mathcal{L}$ es **NL-hard** con respecto a $\leq_p$.

Demostración.

Sea $a \in \mathcal{L}$ y $b \notin \mathcal{L}$. Tomemos $\mathcal{L}' \in \mathbf{NL}$. Sea f la función

$$f(x) = \begin{cases} a & \text{si } x \in \mathcal{L}' \\ b & \text{si } x \notin \mathcal{L}' \end{cases}$$

Como $\mathbf{NL} \subseteq \mathbf{P}$, f es computable en tiempo polinomial. Además,

$$x \in \mathcal{L}' \quad \text{sii} \quad f(x) \in \mathcal{L}$$

Entonces $\mathcal{L}' \leq_p \mathcal{L}$. □

8-7

Funciones computables implícitamente en **L**

Definición

Una función f es computable **implícitamente en L** si

- existe un polinomio p tal que para todo $x \in \{0, 1\}^*$,
 $|f(x)| \leq p(x)$
- $\{\langle x, i \rangle : f(x)(i) = 1\}$ y $\{\langle x, i \rangle : i \leq |f(x)|\}$ están en **L**.

La función booleana

$$\langle x, i \rangle \mapsto \begin{cases} f(x)(i) & \text{si } i \leq |f(x)| \\ 0 & \text{si no} \end{cases}$$

es **L**

(Dado $\langle x, i \rangle$, decidir si $i \leq |f(x)|$ también es **L**.)

8-8

Funciones computables en **L** sin contar salida

Definición

Una función f es **trabajo-L** computable si existe una máquina determinística que computa f en espacio $O(\log n)$ pero donde solo se cuenta el espacio de las cintas de trabajo y no de la cinta de salida. Como siempre,

- la cinta de salida es de solo escritura
- la cabeza escribe y se mueve a la derecha

Ejercicio

Una función es computable implícitamente en **L** sii es trabajo-L computable.

8-9

Composición de funciones computables implícitamente en $\mathbf{L}$

Proposición

Sean f, g computables implícitamente en $\mathbf{L}$. Entonces $g \circ f$ es computable implícitamente en $\mathbf{L}$.

8-10

Demostración

Sean M_f y M_g máquinas determinísticas que corren en espacio $O(\log n)$ tales que $M_f(\langle x, i \rangle) = f(x)(i)$ y $M_g(\langle x, i \rangle) = g(x)(i)$. Definimos la máquina M que con entrada $\langle x, i \rangle$ hace esto:

$j \leftarrow 1$ (índice de la entrada ficticia $M_f(\langle x, j \rangle)$)
 $k \leftarrow 1$ (índice de la salida de $M_f(\langle x, j \rangle)$)
 $b \leftarrow$ resultado de simular $M_f(\langle x, j \rangle)$
simular M_g con entrada $M_f(x)$ paso a paso pero con estos cambios:
engañamos a M_g con la entrada:
el símbolo leído de la entrada de M_g es b
sea I la instrucción de M_g a ejecutar
(depende de b , de las cintas de trabajo de M_g y del estado de M_g)
si I mueve la cabeza de entrada a R/L ,
incrementa/decrementa j en uno
 $b \leftarrow$ resultado de simular $M_f(\langle x, j \rangle)$
si I mueve la cabeza de salida (solo puede a R),
incrementa k en uno
si I escribe $y \in \{0, 1, \square\}$ en la salida y $k = i$,
escribir y en la cinta de salida de M

8-11

- Cada vez que M_g lee un bit de $f(x)$, M lo calcula
 - no tiene espacio para calcular $f(x)$ entero, porque $|f(x)| \leq |x|^c$ para todo x suficientemente largo, pero $|x|^c$ es demasiado grande
- Para simular $M_f(\langle x, j \rangle)$ necesita leer el x de la entrada original de M porque no tiene espacio para copiar x en otro lado
- Lleva cuenta de las cintas de trabajo de M_g
- $M(\langle x, i \rangle) = g(f(x))(i)$
- Para las variables j y k usa

$$\leq \log |f(x)| = \log(|x|^c) = O(\log |x|)$$

- El resto del espacio que usa es el de que usan M_g y M_f .

□

8-12

Reducibilidad **L**

Definición

$\mathcal{L}$ es **L-reducible** a $\mathcal{L}'$, notado $\mathcal{L} \leq_\ell \mathcal{L}'$, si existe una función f computable implícitamente en **L** tal que para todo x ,

$$x \in \mathcal{L} \quad \text{sii} \quad f(x) \in \mathcal{L}'.$$

En este caso decimos que $\mathcal{L} \leq_\ell \mathcal{L}'$ vía f .

Clase de complejidad: NL-hard, NL-completo

$\mathcal{L}$ es **NL-hard** si $\mathcal{L}' \leq_\ell \mathcal{L}$ para todo $\mathcal{L}' \in \text{NL}$.

$\mathcal{L}$ es **NL-completo** si $\mathcal{L} \in \text{NL}$ y $\mathcal{L} \in \text{NL-hard}$.

8-13

Reducibilidad $\mathbf{L}$

Ejercicio

Si $\mathcal{L} \leq_{\ell} \mathcal{L}'$ y $\mathcal{L}' \in \mathbf{L}$, entonces $\mathcal{L} \in \mathbf{L}$.

Ejercicio

La relación $\leq_{\ell}$ es transitiva.

8-14

Teorema de Immerman-Szelepcsényi

Clase 8

Espacio logarítmico: $\mathbf{L}$ y $\mathbf{NL}$

Teorema de Immerman-Szelepcsényi

8-15

PATH $\in$ NL-completo

Teorema

PATH $\in$ NL-completo.

Demostración.

Ya vimos que PATH $\in$ NL.

Solo falta ver PATH $\in$ NL-hard. □

Corolario

$\overline{\text{PATH}} \in \text{coNL-completo}$.

8-16

Demostración de PATH $\in$ NL-hard.

Sea $\mathcal{L} \in \text{NL}$ y N una máquina no-determinística tal que $\mathcal{L}(N) = \mathcal{L}$ y N usa espacio $O(\log n)$. Veamos que $\mathcal{L} \leq_\ell \text{PATH}$ vía f .

$$f(x) = \langle G_{N,x}, C_0, C_f \rangle$$

donde C_0 es la configuración inicial de N para x y C_f es la final.
Cada configuración C se codifica con $c \cdot \log |x|$ bits.

- | | | |
|---------------------|-----|---|
| $x \in \mathcal{L}$ | sii | N acepta x |
| | sii | existe un camino desde C_0 hasta C_f en $G_{N,x}$ |
| | sii | $f(x) \in \text{PATH}$ |

Falta ver que f es computable implícitamente en $\mathbf{L}$.

- El grafo $G_{N,x}$ tiene $2^{c \cdot \log |x|}$ nodos
- Lo representamos con la matriz de adyacencia, de dimensión $2^{c \cdot \log |x|} \times 2^{c \cdot \log |x|}$.
- $|f(x)| = O((2^{c \cdot \log |x|})^2) = O(|x|^{2 \cdot c})$
- Dado $\langle x, i \rangle$, calculamos el i -ésimo bit de $f(x)$ usando espacio logarítmico (enumera configuraciones reusando el espacio).

8-17

Caracterización de **NL** con certificados

Teorema

$\mathcal{L} \in \mathbf{NL}$ sii existe un polinomio $p : \mathbb{N} \rightarrow \mathbb{N}$ y una máquina determinística M con una cinta adicional de lectura de una única vez (lee y pasa a la siguiente celda a la derecha; no puede volver atrás) tal que

- M usa espacio $O(\log n)$
(como siempre, solo cuentan sus cintas de trabajo y salida)
- para todo x :
$$x \in \mathcal{L} \quad \text{sii} \quad \text{existe } u \in \{0, 1\}^{p(|x|)} \text{ tal que } M(x, u) = 1$$
donde $M(x, u)$ denota la salida de M cuando
 - la cinta de entrada tiene x
 - la cinta adicional de lectura de una única vez tiene u
- la cinta de entrada y la cinta adicional no cuentan en el espacio usado por M

- M se llama **verificador**
- u se llama **certificado**

8-18

$\overline{\text{PATH}} \in \mathbf{NL}$

Teorema (Immerman-Szelepcsényi)

$\overline{\text{PATH}} \in \mathbf{NL}$

Corolario

$\mathbf{NL} = \mathbf{coNL}$.

Demostración.

Veamos $\mathbf{coNL} \subseteq \mathbf{NL}$ (el caso $\mathbf{NL} \subseteq \mathbf{coNL}$ es análogo). Sea $\mathcal{L} \in \mathbf{coNL}$. Como $\overline{\text{PATH}} \in \mathbf{coNL}$ -completo, tenemos $\mathcal{L} \leq_{\ell} \overline{\text{PATH}} \in \mathbf{NL}$. Luego $\mathcal{L} \in \mathbf{NL}$. □

Distinto a lo que pasa con $\mathbf{NP}$ vs $\mathbf{coNP}$.

- $\mathbf{NP} \stackrel{?}{=} \mathbf{coNP}$
- se cree que $\mathbf{NP} \neq \mathbf{coNP}$

8-19

Demostración de $\overline{\text{PATH}} \in \text{NL}$

Definimos un verificador M que

- usa espacio logarítmico; no cuenta cinta de entrada ni cinta de lectura de única vez donde va el certificado Z
- certifica que $x \notin \text{PATH}$

$x \in \overline{\text{PATH}}$ sii existe $Z \in \{0, 1\}^{p(|x|)}$ tal que $M(x, Z) = 1$

Supongamos $G = (V, E)$ y $V = \{1, \dots, n\}$. Sea

$$A_i = \{v \in V : v \text{ es alcanzable desde } s \text{ en } \leq i \text{ pasos}\}$$

Notar que A_n es la componente conexa de s en G . Luego

$$\langle G, s, t \rangle \notin \text{PATH} \quad \text{sii} \quad t \notin A_n.$$

8-20

Vamos a dar varios certificados Z de distintos hechos.

Hay un verificador que puede revisar que Z sea un certificado válido para el hecho que pretende certificar

- solo puede leer Z de izquierda a derecha una celda a la vez sin volver atrás
- solo puede usar espacio logarítmico para su revisión
- puede buscar lo que quiera en la cinta de entrada (mover la cabeza a izquierda y derecha)
- puede guardar partes de la entrada o partes del certificado en cintas de trabajo, siempre que no excedan el espacio logarítmico

Siempre codificamos a los nodos de G en binario (tamaño $O(\log n)$).

8-21

Recordar que

$$A_i = \{v \in V : v \text{ es alcanzable desde } s \text{ en } \leq i \text{ pasos}\}$$

Certificado de que $v \in A_i$: una lista de nodos

$$Z_{v \in A_i} = \langle v_0, v_1, \dots, v_k \rangle$$

tal que

- cada v_i es la codificación en binario de un nodo de V
- $v_0 = s$
- $(v_j, v_{j+1}) \in E$
- $v_k = v$
- $k \leq i$

La lista que muestra $Z_{v \in A_i}$ es una forma de probar que $v \in A_i$.

Notar que el tamaño de $Z_{v \in A_i}$ es polinomial.

El verificador chequea el certificado en espacio $O(\log n)$.

8-22

Certificado de que $v \notin A_i$ *conociendo* $|A_i|$: una lista de pares

$$Z_{v \notin A_i}^{|A_i|} = \langle (v_1, Z_{v_1 \in A_i}), (v_2, Z_{v_2 \in A_i}), \dots, (v_k, Z_{v_k \in A_i}) \rangle$$

tal que

- cada v_i es la codificación en binario de un nodo de V
- $k = |A_i|$
- $v_j < v_{j+1}$
- $v \notin \{v_1, \dots, v_k\}$

$Z_{v \notin A_i}^{|A_i|}$ muestra k elementos distintos en A_i tal que ninguno es v y $|A_i| = k$. Es una forma de probar que $v \notin A_i$.

Notar que el tamaño de $Z_{v \notin A_i}^{|A_i|}$ es polinomial.

El verificador chequea el certificado en espacio $O(\log n)$.

8-23

Certificado de que $v \notin A_i$ *conociendo* $|A_{i-1}|$: una lista de pares

$$Z_{v \notin A_i}^{|A_{i-1}|} = \langle (v_1, Z_{v_1 \in A_{i-1}}), (v_2, Z_{v_2 \in A_{i-1}}), \dots, (v_k, Z_{v_k \in A_{i-1}}) \rangle$$

tal que

- cada v_i es la codificación en binario de un nodo de V
- $k = |A_{i-1}|$
- $v_j < v_{j+1}$
- $v \notin \{v_1, \dots, v_k\}$
- $v \notin \bigcup_{1 \leq j \leq k} E(v_j)$, donde $E(x) = \{y \in V : (x, y) \in E\}$

Notar que el tamaño de $Z_{v \notin A_i}^{|A_{i-1}|}$ es polinomial.

El verificador chequea el certificado en espacio $O(\log n)$.

8-24

Recordar que $V = \{1, \dots, n\}$

Certificado de que $|A_i| = a$ *conociendo* $|A_{i-1}|$: una lista de pares

$$Z_{|A_i|=a}^{|A_{i-1}|} = \langle (1, Z_1), \dots, (n, Z_n) \rangle$$

tal que

- si $v \in A_i$ entonces $Z_v = Z_{v \in A_i}$
- si $v \notin A_i$ entonces $Z_v = Z_{v \notin A_i}^{|A_i|-1}$
- $|v : Z_v = Z_{v \in A_i}| = |A_i| = a$

Notar que el tamaño de $Z_{|A_i|=a}^{|A_{i-1}|}$ es polinomial.

El verificador chequea el certificado en espacio $O(\log n)$.

8-25

Certificado final de que $t \notin A_n$:

$$Z = \langle Z_{|A_1|=a_1}^{|A_0|}, Z_{|A_2|=a_2}^{|A_1|}, \dots, Z_{|A_n|=a_n}^{|A_{n-1}|}, Z_{t \notin A_n}^{|A_n|} \rangle$$

tal que

- $a_i = |A_i|$ para todo i

Notar que el tamaño de Z es polinomial.

El verificador chequea el certificado en espacio $O(\log n)$:

- sabemos que $A_0 = \{s\}$ y por lo tanto $|A_0| = 1$
- a medida que lee de izquierda a derecha, va guardando $|A_{i-1}|$ para verificar $Z_{|A_i|=a_i}^{|A_{i-1}|}$ (reusa espacio)

□

8-26

Teorema de Immerman-Szelepcsényi más general

Teorema

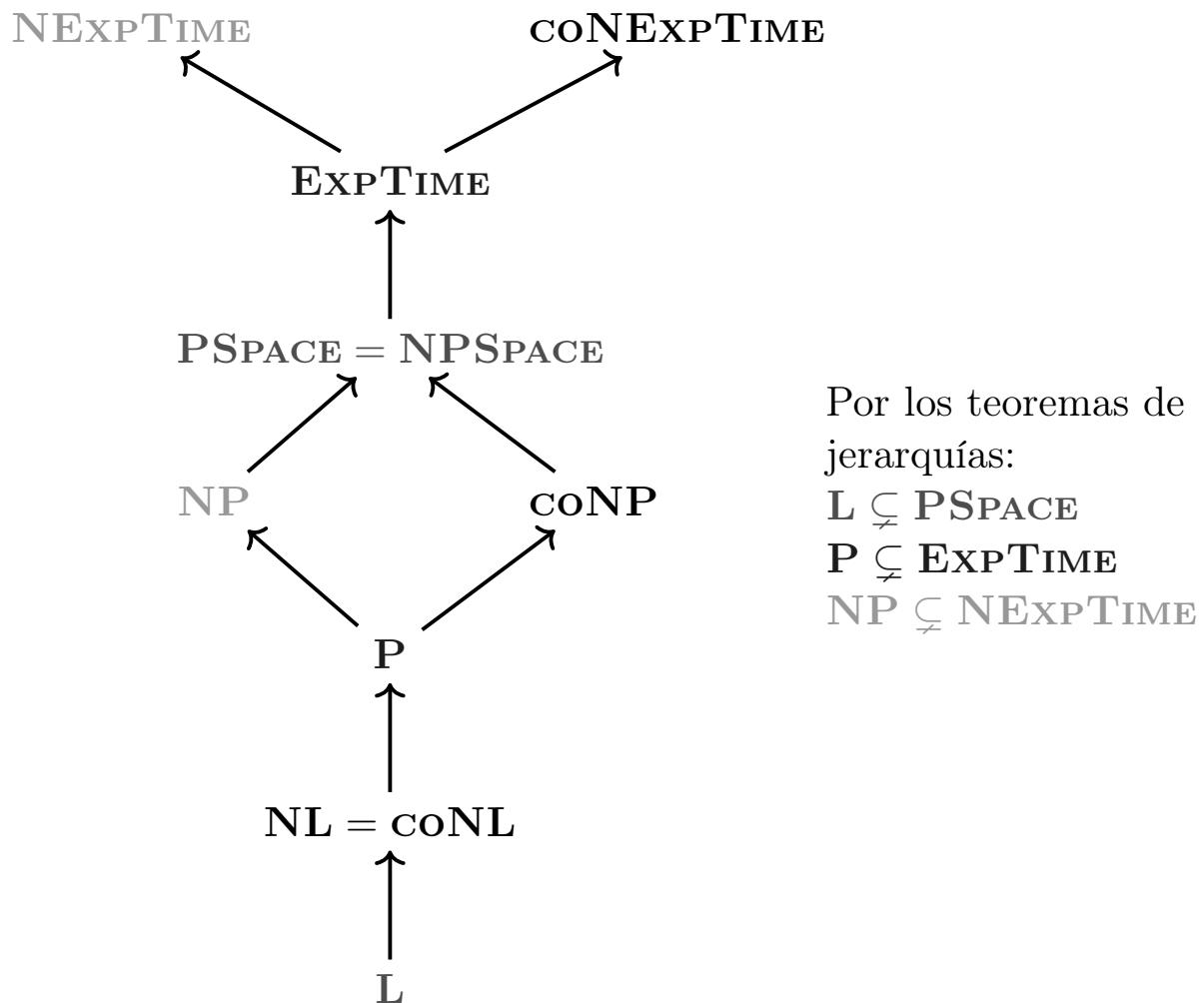
Si $S(n) \geq \log n$ es construible en espacio entonces
 $\mathbf{NSPACE}(S(n)) = \mathbf{CONSPACE}(S(n))$.

Ejercicio

Demostrarlo.

$\mathbf{NL} = \mathbf{coNL}$ es un caso particular cuando $S(n) = \log n$.

8-27



8-28

Clase 9

Clase 1

- Preliminares
- Representaciones en binario
- Problemas de decisión
- Máquinas de Turing
- Configuraciones y cálculos

Clase 2

- Funciones computables
- Tiempo de cómputo
- Codificación de máquinas
- Variantes de máquinas

Clase 3

- Halt y máquina universal
- La clase P
- La clase NP

Clase 4

- Máquinas no-determinísticas
- Reducibilidad polinomial
- Lógica proposicional

Clase 5

- Mini-configuraciones
- Teorema de Cook-Levin
- Problemas NP -completos
- La clase $coNP$
- Las clases $EXPTIME$ y $NEXPTIME$

Clase 6

- La jerarquía de tiempos determinísticos
- La jerarquía de tiempos no-determinísticos
- Teorema de Ladner

Clase 7

- Espacio usado por un cómputo
- Espacio polinomial: $PSPACE$ y $NPSpace$
- Teorema de Savitch

Clase 8

- Espacio logarítmico: L y NL
- Teorema de Immerman-Szelepcsényi

Clase 9

- La jerarquía polinomial
- Problemas Σ_i^P -completos

Clase 10

- Máquinas con oráculo
- Teorema de Baker, Gill, Solovay
- La jerarquía polinomial y NP con oráculos

Clase 11

- Circuitos booleanos
- La clase P_{poly}
- Tamaño y profundidad de circuitos

Clase 12

- Las clases NC_{nu} y AC_{nu}
- Uniformidad: las clases NC y AC
- P -completitud

La jerarquía polinomial

Clase 9

La jerarquía polinomial

Problemas Σ_i^P -completos

9-2

Problemas NP

Problema: SAT (satisfacción booleana en CNF)

$$\text{SAT} = \{\langle \varphi \rangle : \varphi \in \text{CNF} \text{ es satisfacible}\}$$

$$\langle \varphi \rangle \in \text{SAT} \quad \text{sii} \quad \exists v \quad \underbrace{v \models \varphi}_{\substack{\text{"}v \text{ es una} \\ \text{valuación} \\ \text{que satis-} \\ \text{face } \varphi \text{"} \rightsquigarrow \\ \text{polinomial}}}$$

Problema: Conjunto independiente

$$\text{INDSET} = \{\langle G, k \rangle \mid \begin{array}{l} G \text{ tiene un conjunto inde-} \\ \text{pendiente de } \geq k \text{ vértices} \end{array} \}$$

$$\underbrace{\langle (V, E), k \rangle}_G \in \text{INDSET} \quad \text{sii} \quad \underbrace{\exists C \ C \subseteq V, |C| = k, \neg \exists u, v \in C, (u, v) \in E}_{\substack{\text{"}C \text{ es un conjunto independiente de} \\ G \text{ de } k \text{ vértices"} \rightsquigarrow \text{polinomial}}}$$

9-3

Problemas **coNP**

Recordar

Problema: Tautología

$$\text{TAUT} = \{ \langle \varphi \rangle : \varphi \in \text{CNF es una tautología} \}$$

$$\langle \varphi \rangle \in \text{TAUT} \quad \text{sii} \quad \forall v \quad \underbrace{v \models \varphi}_{\substack{\text{"v es una} \\ \text{valuación} \\ \text{que satis-} \\ \text{face } \varphi"} \rightsquigarrow \text{polinomial}}$$

9-4

Otros problemas

Problema: Conjunto independiente máximo

$$\text{MAXINDSET} = \{ \langle G, k \rangle : \begin{array}{l} \text{el conjunto independiente más} \\ \text{grande de } G \text{ tiene } k \text{ vértices} \end{array} \}$$

$$\begin{array}{llll} \text{¿Podemos esto?} & \langle G, k \rangle \in \text{MAXINDSET} & \text{sii} & \exists \underbrace{\dots\dots\dots}_{\text{polinomial}} \\ & & & \text{polinomial} \\ \text{¿Y esto?} & \langle G, k \rangle \in \text{MAXINDSET} & \text{sii} & \forall \underbrace{\dots\dots\dots}_{\text{polinomial}} \end{array}$$

Pero sí podemos hacer esto:

$$\begin{array}{l} \overbrace{\langle (V, E), k \rangle}^G \in \text{MAXINDSET} \quad \text{sii} \\ \exists C \forall D \left(\underbrace{\begin{array}{l} C \subseteq V, |C| = k \wedge \neg \exists u, v \in C \wedge (u, v) \in E \wedge \\ (D \subseteq V \wedge \neg \exists u, v \in D \wedge (u, v) \in E) \rightarrow |D| \leq |C| \end{array}} \right) \\ \text{"C es un conjunto independiente de } G \text{ de } k \text{ vértices y si } D \text{ es un conjunto independiente de } G, \text{ no} \\ \text{puede ser más grande que } C" \rightsquigarrow \text{polinomial} \end{array}$$

9-5

La jerarquía polinomial

Clase de complejidad: Σ_i^P, Π_i^P

- Para $i > 0$, Σ_i^P es la clase de lenguajes $\mathcal{L}$ tales que existe una máquina determinística M que corre en tiempo polinomial y un polinomio q tal que

$$\begin{aligned} x \in \mathcal{L} \quad \text{sii} \quad & \exists u_1 \in \{0, 1\}^{q(|x|)} \\ & \forall u_2 \in \{0, 1\}^{q(|x|)} \\ & \vdots \\ & Q_i u_i \in \{0, 1\}^{q(|x|)} \quad M(\langle x, u_1, \dots, u_i \rangle) = 1 \end{aligned}$$

$$\text{donde } Q_i = \begin{cases} \forall & \text{si } i \text{ es par} \\ \exists & \text{si } i \text{ es impar} \end{cases}$$

- $\Sigma_0^P = P$
- $PH = \bigcup_{i \geq 0} \Sigma_i^P$ es la **jerarquía polinomial**
- Para $i \geq 0$, $\Pi_i^P = \{\overline{\mathcal{L}} : \mathcal{L} \in \Sigma_i^P\}$

9-6

Alternancia de cuantificadores

Lo que importa en Σ_i^P [resp. Π_i^P] es que la fórmula empiece con $\exists$ [resp. $\forall$] y haya $i - 1$ *alternancias* de cuantificadores.
Bloques $\exists\exists$ se pueden unir en un solo $\exists$

Ejemplo

$$\begin{aligned} & \exists u_1 \in \{0, 1\}^{q(|x|)} \\ & \forall u_2 \in \{0, 1\}^{q(|x|)} \\ & \exists u_3 \in \{0, 1\}^{q(|x|)} \\ & \exists u_4 \in \{0, 1\}^{q'(|x|)} \\ & M(\langle x, u_1, u_2, u_3, u_4 \rangle) = 1 \end{aligned} \quad \text{sii} \quad \begin{aligned} & \exists u_1 \in \{0, 1\}^{q(|x|)+q'(|x|)} \\ & \forall u_2 \in \{0, 1\}^{q(|x|)+q'(|x|)} \\ & \exists u'_3 \in \{0, 1\}^{q(|x|)+q'(|x|)} \\ & M'(\langle x, u_1, u_2, u'_3 \rangle) = 1 \end{aligned}$$

- M' solo lee los primeros $q(|x|)$ bits de u_1, u_2 (el resto los ignora)
- la información de u_3 y u_4 ahora la tiene pegada en $u'_3 = u_3 u_4$

Lo mismo vale para bloques $\forall\forall$.

9-7

Observaciones de la definición de Σ_i^P

$$\begin{aligned}
 x \in \mathcal{L} \quad \text{sii} \quad & \exists u_1 \in \{0, 1\}^{q(|x|)} \\
 & \forall u_2 \in \{0, 1\}^{q(|x|)} \\
 & \vdots \\
 & \exists u_i \in \{0, 1\}^{q(|x|)} \quad M(\langle x, u_1, \dots, u_i \rangle) = 1
 \end{aligned}$$

- podemos pedir $M(xu_1 \dots u_i) = 1$ en vez de $M(\langle x, u_1, \dots, u_i \rangle) = 1$.
- podemos pedir distintos polinomios $q_1, \dots, q_i$ para las longitudes de $u_1 \dots, u_i$ en vez del mismo q para todos.

9-8

Ejemplo de problema en Σ_2^P

Proposición

MAXINDSET $\in \Sigma_2^P$.

Demostración.

Existe una máquina determinística M que corre en tiempo polinomial tal que

$M(\langle \overbrace{V, E}^G, k, C, D \rangle) = 1$ sii C es un conjunto independiente de G de k vértices y, si D es un conjunto independiente de G , el tamaño de C es mayor o igual al tamaño de D

$\underbrace{\langle (V, E), k \rangle}_x \in \text{MAXINDSET}$ sii $\exists C \forall D \quad M(\langle \underbrace{V, E}_x, k, C, D \rangle) = 1$

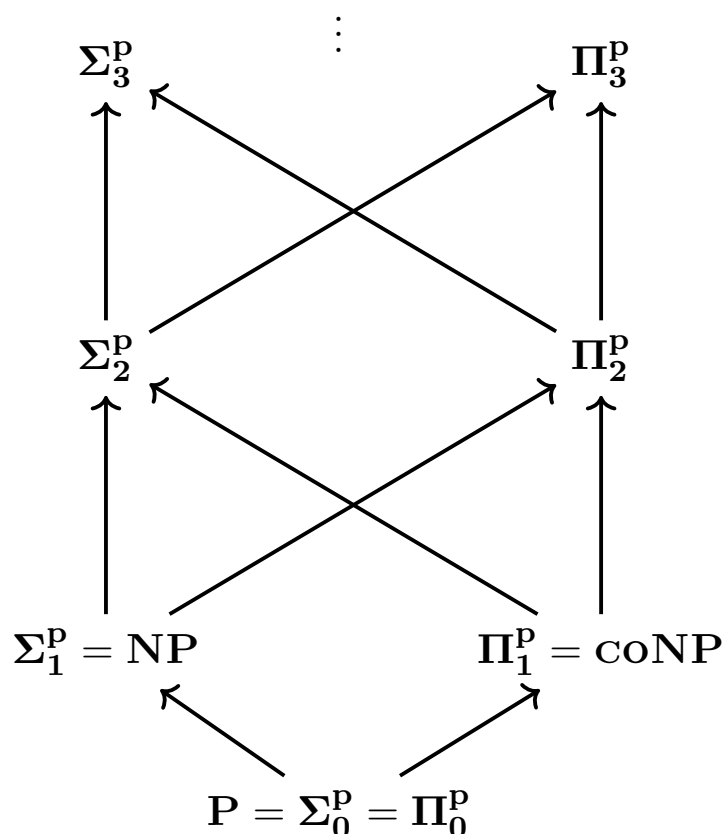
Suponemos que $V = \{0, \dots, n-1\}$. Los conjuntos C y D los codificamos con palabras en $\{0, 1\}^{|V|}$. Como $|V| \leq |x|$, podemos codificar C, D como palabras de $\{0, 1\}^{|x|}$. □

9-9

Propiedades de la jerarquía polinomial

Proposición

- $\Sigma_1^P = \text{NP}$
- $\Pi_1^P = \text{coNP}$
- $\Sigma_i^P \subseteq \Sigma_{i+1}^P$
- $\Sigma_i^P \subseteq \Pi_{i+1}^P$
- $\Pi_i^P \subseteq \Sigma_{i+1}^P$
- $\Pi_i^P \subseteq \Pi_{i+1}^P$
- $\text{PH} = \bigcup_{i \geq 0} \Sigma_i^P$



9-10

$\Sigma_i^P \subsetneq \Sigma_{i+1}^P$?

$P \stackrel{?}{=} \text{NP}$ se puede generalizar a $\Sigma_i^P \stackrel{?}{=} \Sigma_{i+1}^P$; ninguna se conoce.

Proposición

Si $P = \text{NP}$ entonces $\text{PH} = P$.

Ejercicio

Para todo $i > 0$, si $\Sigma_i^P = \Pi_i^P$ entonces $\text{PH} = \Sigma_i^P$.

9-11

Demostración de $\mathbf{P} = \mathbf{NP} \Rightarrow \mathbf{PH} = \mathbf{P}$.

Sup. $\mathbf{P} = \mathbf{NP}$. Probamos $\Sigma_i^{\mathbf{P}}, \Pi_i^{\mathbf{P}} \subseteq \mathbf{P}$ por inducción en $i \geq 1$.

$i = 1$: trivial porque $\mathbf{P} = \mathbf{NP} = \Sigma_1^{\mathbf{P}}$ y $\mathbf{P} = \mathbf{coNP} = \Pi_1^{\mathbf{P}}$.

Sup. $\Sigma_i^{\mathbf{P}}, \Pi_i^{\mathbf{P}} \subseteq \mathbf{P}$ y probemos $\Sigma_{i+1}^{\mathbf{P}}, \Pi_{i+1}^{\mathbf{P}} \subseteq \mathbf{P}$. Sea $\mathcal{L} \in \Sigma_{i+1}^{\mathbf{P}}$.

Existe una máquina determinística M que corre en tiempo polinomial y un polinomio q tal que

$$x \in \mathcal{L} \quad \text{sii} \quad \begin{aligned} &\exists u_1 \in \{0, 1\}^{q(|x|)} \quad \forall u_2 \in \{0, 1\}^{q(|x|)} \dots \\ &\dots Q_{i+1} u_{i+1} \in \{0, 1\}^{q(|x|)} \quad M(\langle x, u_1, \dots, u_{i+1} \rangle) = 1 \end{aligned}$$

Definimos $\mathcal{L}'$ así: $\langle x, u_1 \rangle \in \mathcal{L}'$ sii

$$\forall u_2 \in \{0, 1\}^{q(|x|)} \dots Q_{i+1} u_{i+1} \in \{0, 1\}^{q(|x|)} \quad M(\langle x, u_1, \dots, u_{i+1} \rangle) = 1$$

Por HI $\mathcal{L}' \in \Pi_i^{\mathbf{P}} \subseteq \mathbf{P}$. Luego existe la máquina determinística M' tal que $\mathcal{L}(M') = \mathcal{L}'$ y M' corre en tiempo polinomial.

$$\begin{aligned} \langle x, u_1 \rangle \in \mathcal{L}' &\quad \text{sii} \quad M'(\langle x, u_1 \rangle) = 1 \\ x \in \mathcal{L} &\quad \text{sii} \quad \exists u_1 \in \{0, 1\}^{q(|x|)} \quad M'(\langle x, u_1 \rangle) = 1 \end{aligned}$$

Luego $\mathcal{L} \in \mathbf{NP} = \mathbf{P}$. Como $\mathcal{L}$ era arbitrario en $\Sigma_{i+1}^{\mathbf{P}}$, concluimos $\Sigma_{i+1}^{\mathbf{P}} \subseteq \mathbf{P}$ (y también $\Pi_{i+1}^{\mathbf{P}} \subseteq \mathbf{P}$). □

9-12

Proposición

Si $\mathcal{L} \in \Sigma_i^{\mathbf{P}}$ y $\mathcal{L}' \leq_p \mathcal{L}$ entonces $\mathcal{L}' \in \Sigma_i^{\mathbf{P}}$.

Si $\mathcal{L} \in \Pi_i^{\mathbf{P}}$ y $\mathcal{L}' \leq_p \mathcal{L}$ entonces $\mathcal{L}' \in \Pi_i^{\mathbf{P}}$.

Demostración.

Supongamos una máquina determinística M que corre en tiempo polinomial y un polinomio q tal que para todo x

$$x \in \mathcal{L} \quad \text{sii} \quad \exists u_1 \in \{0, 1\}^{q(|x|)} \dots Q_i u_i \in \{0, 1\}^{q(|x|)} \quad M(\langle x, u_1, \dots, u_i \rangle) = 1.$$

Supongamos que f es computable en tiempo polinomial y para todo x

$$x \in \mathcal{L}' \quad \text{sii} \quad f(x) \in \mathcal{L}$$

Juntando las dos, tenemos $x \in \mathcal{L}'$ sii

$$\exists u_1 \in \{0, 1\}^{q(|x|)} \dots Q_i u_i \in \{0, 1\}^{q(|x|)} \quad \underbrace{M(\langle f(x), u_1, \dots, u_i \rangle)}_{M'(\langle x, u_1, \dots, u_i \rangle)} = 1.$$

M' también corre en tiempo polinomial. □

9-13

Problemas Σ_i^P -completos

Clase 9

La jerarquía polinomial

Problemas Σ_i^P -completos

9-14

Complejidad

Clase de complejidad: Σ_i^P-hard, Σ_i^P-completo
$\mathcal{L}$ es Σ_i^P -hard si $\mathcal{L}' \leq_p \mathcal{L}$ para todo $\mathcal{L}' \in \Sigma_i^P$. $\mathcal{L}$ es Σ_i^P -completo si $\mathcal{L} \in \Sigma_i^P$ y $\mathcal{L} \in \Sigma_i^P$ -hard

Análogamente definimos Π_i^P -hard, Π_i^P -completo, PH-hard, PH-completo.

9-15

¿Problemas **PH-completos**?

Proposición

Si existe $\mathcal{L} \in \mathbf{PH-completo}$ entonces existe i tal que $\mathbf{PH} = \Sigma_i^P$.

Demostración.

Sea $\mathcal{L} \in \mathbf{PH} = \bigcup_i \Sigma_i^P$ tal que $\mathcal{L}' \leq_p \mathcal{L}$ para todo $\mathcal{L}' \in \mathbf{PH}$.

Sea i tal que $\mathcal{L} \in \Sigma_i^P$.

Sea $\mathcal{L}' \in \mathbf{PH}$ arbitrario. Como $\mathcal{L}' \leq_p \mathcal{L}$, concluimos $\mathcal{L}' \in \Sigma_i^P$.

Luego $\mathbf{PH} \subseteq \Sigma_i^P$ (y $\mathbf{PH} \supseteq \Sigma_i^P$ es trivial). $\square$

Se cree que para todo i , $\Sigma_i^P \subsetneq \Sigma_{i+1}^P$. Se dice que la jerarquía polinomial **colapsa al i -ésimo nivel** si $\Sigma_i^P = \Sigma_{i+1}^P$ y en este caso, $\Sigma_P^P = \mathbf{PH}$.

Como se cree que la jerarquía polinomial no colapsa al i -ésimo nivel, se cree que no existen los problemas **PH-completos**.

9-16

Ejercicio

$\mathbf{PH} \subseteq \mathbf{PSPACE}$.

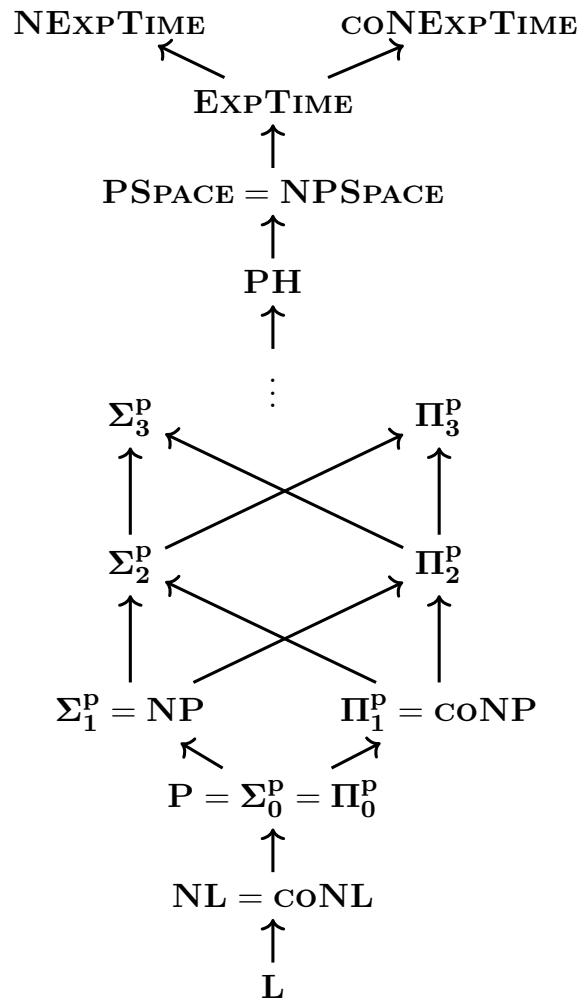
Corolario

Si la jerarquía polinomial no colapsa en ningún nivel, $\mathbf{PH} \neq \mathbf{PSPACE}$.

Demostración.

Si $\mathbf{PH} = \mathbf{PSPACE}$ entonces existirían problemas en $\mathbf{PH-completo} = \mathbf{PSPACE-completo}$. $\square$

9-17



9-18

Problemas Σ_i^P -completos

Problema: Satisfacibilidad de QBF acotada

$\Sigma_i SAT = \{ \langle \varphi \rangle : \begin{array}{l} \varphi \text{ es una QBF de la forma} \\ \exists \bar{y}_1 \forall \bar{y}_2 \dots Q_i \bar{y}_i \psi(\bar{y}_1, \dots, \bar{y}_i) \text{ donde las } \bar{y}_i \text{ son tu-} \\ \text{plas de variables booleanas, } \psi \text{ es una fórmula} \\ \text{booleana, los cuantificadores se alternan y } \models \varphi \end{array} \}$

Proposición

Para todo $i > 0$, $\Sigma_i SAT \in \Sigma_i^P$ -completo.

9-19

Demostración de $\Sigma_i\text{SAT} \in \Sigma_i^P$.

Consideremos la máquina determinística M que con entrada $\langle \varphi, u_1, \dots, u_i \rangle$ hace esto :

si φ no es una QBF de la forma $\exists \bar{y}_1 \forall \bar{y}_2 \dots Q_i \bar{y}_i \psi(\bar{y}_1, \dots, \bar{y}_i)$, o si para algún j , $|u_j| < |\bar{y}_j|$, rechazar. Si no, devolver 1 si $(u_1 \upharpoonright |y_1|) \dots (u_i \upharpoonright |y_i|) \models \psi$ y 0 en caso contrario

Es claro que M corre en tiempo polinomial y $\langle \varphi \rangle \in \Sigma_i\text{SAT}$ sii

$$\exists \bar{y}_1 \in \{0, 1\}^k \forall \bar{y}_2 \in \{0, 1\}^k \dots Q_i \bar{y}_i \in \{0, 1\}^k M(\langle \varphi, u_1, \dots, u_i \rangle)$$

- $k = |\langle \varphi \rangle|$ es suficientemente largo
- la cantidad de variables de φ es siempre menor que la longitud de la codificación de φ

□

9-20

Demostración de $\Sigma_i\text{SAT} \in \Sigma_i^P\text{-hard}$

Recordemos $\text{SAT} \in \text{NP-completo}$. Sea $\mathcal{L} \in \text{NP}$ tal que

$$x \in \mathcal{L} \quad \text{sii} \quad \exists u \in \{0, 1\}^{p(|x|)} M(xu) = 1$$

donde p es un polinomio y M es una máquina determinística que corre en tiempo polinomial $t(n)$

Para $j = 1 \dots 4$ definimos fórmulas booleanas

$$\psi_j \left(\underbrace{\bar{e}}_{\text{codifica } x}, \underbrace{\bar{c}}_u, \underbrace{\bar{q}_0}_{z_0}, \dots, \underbrace{\bar{q}_m}_{z_m} \right) \quad \psi_1 = \text{“la entrada empieza con } x \text{ y } u \in \{0, 1\}^* \text{”}$$

donde $z_0, \dots, z_m, z_i \in \{0, 1\}^k$ (k depende solo de M) es una secuencia de mini-configuraciones, $m = t(|xu|)$, y $\bar{e}, \bar{c}, \bar{q}_0, \dots, \bar{q}_m$ son tuplas de variables de tamaño polinomial.

$$\psi_2 = \text{“} z_0 \text{ es la configuración inicial”}$$

$$\psi_3 = \text{“} z_j \text{ evoluciona en } z_{j+1} \text{ para } j = 0, \dots, m-1 \text{”}$$

$$\psi_4 = \text{“} z_m \text{ es una configuración final de } M \text{ aceptadora”}$$

9-21

$$\psi_j \left(\underbrace{\bar{e}}_x, \underbrace{\bar{c}}_u, \underbrace{\bar{q}_0}_{z_0}, \dots, \underbrace{\bar{q}_m}_{z_m} \right)$$

codifica

$$\begin{aligned} M(xu) = 1 & \quad \text{sii} \quad \tilde{u} \models \exists \bar{e}, \bar{q}_0, \dots, \bar{q}_m \psi_1 \wedge \psi_2 \wedge \psi_3 \wedge \psi_4 \\ & \quad \text{sii} \quad \tilde{u} \models \forall \bar{e}, \bar{q}_0, \dots, \bar{q}_m (\psi_1 \wedge \psi_2 \wedge \psi_3) \rightarrow \psi_4 \end{aligned}$$

donde

- $|u| = q(|x|)$
- $\tilde{u} = u(0)u(0)u(1)u(1) \dots u(|u| - 1)u(|u| - 1)$ es la codificación de u
- corresponde a la valuación para las variables de $\bar{c}$

Entonces

$$\begin{aligned} x \in \mathcal{L} & \quad \text{sii} \quad \exists u \in \{0, 1\}^{p(|x|)} M(xu) = 1 \\ & \quad \text{sii} \quad \models \exists \bar{c}, \exists \bar{e}, \bar{q}_0, \dots, \bar{q}_m \psi_1 \wedge \psi_2 \wedge \psi_3 \wedge \psi_4 \\ & \quad \text{sii} \quad \models \exists \bar{c} \forall \bar{e}, \bar{q}_0, \dots, \bar{q}_m (\psi_1 \wedge \psi_2 \wedge \psi_3) \rightarrow \psi_4 \end{aligned}$$

9-22

Supongamos que $\mathcal{L} \in \Sigma_i^P$. Sea M la máquina determinística *oblivious*, sin cinta de salida y con única cinta de trabajo que corre en tiempo polinomial $t(n)$ y para todo x

$$x \in \mathcal{L} \quad \text{sii} \quad \underbrace{\exists u_1 \in \{0, 1\}^{q(|x|)} \dots Q_i v_i \in \{0, 1\}^{q(|x|)}}_{i-1 \text{ alternancias}} M(xu_1 \dots u_i) = 1$$

Como antes:

$$\psi_j \left(\underbrace{\bar{e}}_x, \underbrace{\bar{c}}_{u=u_1 \dots u_i}, \underbrace{\bar{q}_0}_{z_0}, \dots, \underbrace{\bar{q}_m}_{z_m} \right)$$

codifica

donde $\bar{c}$ es una tupla de variables booleanas de tamaño $2 \cdot i \cdot q(|x|)$

$\psi_1 =$ “la entrada empieza con x y $u \in \{0, 1\}^*$ ”

$\psi_2 =$ “ z_0 es la configuración inicial”

$\psi_3 =$ “ z_j evoluciona en z_{j+1} para $j = 0, \dots, m-1$ ”

$\psi_4 =$ “ z_m es una configuración final de M aceptadora”

(no especifica las variables $\bar{c}$ que corresponden a $u = u_1 \dots u_i$)

$$M(xu_1 \dots u_i) = 1$$

$$\begin{aligned} & \quad \text{sii} \quad \tilde{u}_1 \dots \tilde{u}_i \models \exists \bar{e}, \bar{q}_0, \dots, \bar{q}_m \psi_1 \wedge \psi_2 \wedge \psi_3 \wedge \psi_4 \\ & \quad \text{sii} \quad \tilde{u}_1 \dots \tilde{u}_i \models \forall \bar{e}, \bar{q}_0, \dots, \bar{q}_m (\psi_1 \wedge \psi_2 \wedge \psi_3) \rightarrow \psi_4 \end{aligned}$$

9-23

$$x \in \mathcal{L} \text{ sii } \underbrace{\exists u_1 \in \{0, 1\}^{q(|x|)} \dots Q_i v_i \in \{0, 1\}^{q(|x|)}}_{i-1 \text{ alternancias}} M(xu_1 \dots u_i) = 1$$

- Si $Q_i = \exists$ tenemos $x \in \mathcal{L} \text{ sii } \models \rho_x \text{ sii } \rho_x \in \Sigma_i \text{SAT}$, donde

$$\rho_x = \underbrace{\exists \bar{c}_1 \forall \bar{c}_2 \dots \exists \bar{c}_i \exists \bar{e}, \bar{q}_0, \dots, \bar{q}_m}_{i-1 \text{ alternancias}} \psi_1 \wedge \psi_2 \wedge \psi_3 \wedge \psi_4$$

- ρ_x es una QBF con todas las tuplas cuantificadas de tamaño polinomial en $|x|$
- ρ_x se calcula en tiempo polinomial a partir de x
- cada $\bar{c}_i$ es una tupla de variables booleanas de dimensión $q(|x|)$ y $\bar{c} = \bar{c}_1, \dots, \bar{c}_i$
- el último bloque se convierte en un $\exists$, entonces ρ_x sigue teniendo $i-1$ alternancias de cuantificadores.

9-24

$$x \in \mathcal{L} \text{ sii } \underbrace{\exists u_1 \in \{0, 1\}^{q(|x|)} \dots Q_i v_i \in \{0, 1\}^{q(|x|)}}_{i-1 \text{ alternancias}} M(xu_1 \dots u_i) = 1$$

- Si $Q_i = \forall$ tenemos $x \in \mathcal{L} \text{ sii } \models \rho_x \text{ sii } \rho_x \in \Sigma_i \text{SAT}$, donde

$$\rho_x = \underbrace{\exists \bar{c}_1 \forall \bar{c}_2 \dots \forall \bar{c}_i \forall \bar{e}, \bar{q}_0, \dots, \bar{q}_m}_{i-1 \text{ alternancias}} (\psi_1 \wedge \psi_2 \wedge \psi_3) \rightarrow \psi_4$$

- El último bloque se convierte en un $\forall$, entonces ρ_x sigue teniendo $i-1$ alternancias de cuantificadores. □

9-25

Clase 10

Clase 1

Preliminares
Representaciones en binario
Problemas de decisión
Máquinas de Turing
Configuraciones y cálculos

Clase 2

Funciones computables
Tiempo de cómputo
Codificación de máquinas
Variantes de máquinas

Clase 3

Halt y máquina universal
La clase **P**
La clase **NP**

Clase 4

Máquinas no-determinísticas
Reducibilidad polinomial
Lógica proposicional

Clase 5

Mini-configuraciones
Teorema de Cook-Levin
Problemas **NP-completos**
La clase **coNP**
Las clases **EXPTIME** y **NEXPTIME**

Clase 6

La jerarquía de tiempos determinísticos
La jerarquía de tiempos no-determinísticos
Teorema de Ladner

Clase 7

Espacio usado por un cómputo
Espacio polinomial: **PSPACE** y **NPSPACE**
Teorema de Savitch

Clase 8

Espacio logarítmico: **L** y **NL**
Teorema de Immerman-Szelepcsényi

Clase 9

La jerarquía polinomial
Problemas Σ_i^P -completos

Clase 10

Máquinas con oráculo
Teorema de Baker, Gill, Solovay
La jerarquía polinomial y **NP** con oráculos

Clase 11

Circuitos booleanos
La clase **P**/**poly**
Tamaño y profundidad de circuitos

Clase 12

Las clases **NC_{nu}** y **AC_{nu}**
Uniformidad: las clases **NC** y **AC**
P-completitud

10-1

Máquinas con oráculo

Clase 10

Máquinas con oráculo
Teorema de Baker, Gill, Solovay
La jerarquía polinomial y **NP** con oráculos

10-2

Máquinas con oráculo

Son como las máquinas de Turing que vimos, pero con estas diferencias:

- su comportamiento depende de un lenguaje $\mathcal{X} \subseteq \{0, 1\}^*$
- tiene una cinta adicional de *consulta*:
- tiene 3 estados distinguidos más:
 - $q_{\text{consulta}}, q_{\text{resp:sí}}, q_{\text{resp:no}}$
- una nueva

INSTRUCCIÓN

*si estado == q_{consulta}
entonces*

*supongamos que en la cinta de consulta
está escrito " $\triangleright x \square$ ", con $x \in \{0, 1\}^*$
si $x \in \mathcal{X}$, pasar a $q_{\text{resp:sí}}$
si no, pasar a $q_{\text{resp:no}}$*

10-3

Máquinas con oráculo

- Mismas definiciones de cómputo, aceptación, rechazo, tiempo de cómputo, uso de espacio, etc.
- El comportamiento de M depende de la información del oráculo.
- Si M es una máquina (determinística o no-determinística) con oráculo, notamos $M^{\mathcal{X}}(x)$ a la salida de M con oráculo $\mathcal{X}$ y entrada x (si es que terminó).
- Si $M(x)$ termina, solo puede hacer una cantidad finita de consultas al oráculo. Si cambiamos el oráculo en elementos que nunca son consultados, el cómputo no cambia.
 - Ejemplo: $M^{\mathcal{X}}(x)$ termina y a lo largo del cómputo consulta $y_1, \dots, y_m$ al oráculo. Para cualquier $\mathcal{Y}$ tal que $y_j \in \mathcal{X}$ sii $y_j \in \mathcal{Y}$ para $j = 1, \dots, m$, tenemos $M^{\mathcal{X}}(x) = M^{\mathcal{Y}}(x)$.
 - Ejemplo: $M^{\mathcal{X}}(x)$ al paso t solo puede hacer finitas consultas al oráculo, independientemente de la respuesta que reciba. Al paso t no puede hacer más que t consultas.
- Las máquinas con oráculo se pueden listar, de la misma forma que hicimos con las máquinas determinísticas o las máquinas no-determinísticas.

10-4

Clase de complejidad: $P^{\mathcal{X}}, NP^{\mathcal{X}}$

- $P^{\mathcal{X}}$ es la clase de lenguajes decidibles por una máquina determinística que corre en tiempo polinomial y tiene acceso al oráculo $\mathcal{X}$.
- $NP^{\mathcal{X}}$ es la clase de lenguajes decidibles por una máquina no-determinística que corre en tiempo polinomial y tiene acceso al oráculo $\mathcal{X}$.

10-5

Ejemplo: $\overline{SAT} \in P^{SAT}$

Considerar la siguiente máquina determinística M con acceso a SAT y entrada x :

*preguntar al oráculo si $x \in SAT$ (escribir x en la cinta de consulta)
si responde ‘sí’ (entra a $q_{resp:sí}$), devolver 0
si no (entra a $q_{resp:no}$), devolver 1*

M corre en tiempo lineal independientemente del oráculo al que tenga acceso y $\mathcal{L}(M^{SAT}) = \overline{SAT}$.

10-6

Proposición

Si $\mathcal{X} \in \mathbf{P}$, entonces $\mathbf{P} = \mathbf{P}^{\mathcal{X}}$.

Demostración.

Probemos $\mathbf{P}^{\mathcal{X}} \subseteq \mathbf{P}$ (la otra inclusión es trivial).

Sea M una máquina determinística que corre en tiempo polinomial y decide $\mathcal{X}$.

Sea $\mathcal{L} \in \mathbf{P}^{\mathcal{X}}$ y supongamos que M' es una máquina determinística que corre en tiempo polinomial y decide $\mathcal{L}$ con acceso al oráculo $\mathcal{X}$.

La máquina M'' hace lo mismo que M' pero reemplaza cada pregunta x al oráculo $\mathcal{X}$ por la llamada a $M(x)$.

M'' corre en tiempo polinomial y decide $\mathcal{L}$.

Entonces $\mathcal{L} \in \mathbf{P}$. □

10-7

Problema: EXPCOM (Cómputos exponenciales)

$\text{EXPCOM} = \{ \langle M, x, 1^n \rangle : \begin{array}{l} \text{la máquina determinística } M \text{ con entra-} \\ \text{da } x \text{ devuelve 1 en } \leq 2^n \text{ pasos} \end{array} \}$

Proposición

$\text{EXPTIME} \subseteq \mathbf{P}^{\text{EXPCOM}}$.

Demostración

Sea $\mathcal{L} \in \mathbf{DTIME}(2^{n^c})$. Supongamos que $\mathcal{L} = \mathcal{L}(M)$ para una máquina determinística M que corre en tiempo $O(2^{n^c})$. Sea k tal que para todo $x \in \{0, 1\}^*$ con $|x| > k$ tenemos que M con entrada x termina en $\leq 2^{|x|^{c+1}}$ pasos.

Considerar la siguiente máquina determinística M_1 con entrada x y acceso al oráculo EXPCOM:

si $|x| \leq k$, devolver 1 si $x \in \mathcal{L}$ y 0 en caso contrario

si no, preguntar al oráculo si $\langle M, x, 1^{|x|^{c+1}} \rangle$ y

devolver su respuesta

M_1 corre en tiempo polinomial, entonces $\mathcal{L}(M_1^{\text{EXPCOM}}) \in \mathbf{P}^{\text{EXPCOM}}$ y

$x \in \mathcal{L}(M_1^{\text{EXPCOM}}) \quad \text{sii} \quad x \in \mathcal{L} = \mathcal{L}(M)$

□

10-8

Proposición

$\mathbf{NP}^{\mathbf{EXPCOM}} \subseteq \mathbf{EXPTIME}$.

Demostración.

Sea $\mathcal{L} \in \mathbf{NP}^{\mathbf{EXPCOM}}$ y sea N una máquina no-determinística que corre en tiempo polinomial y decide $\mathcal{L}$ con acceso al oráculo $\mathbf{EXPCOM}$.

En tiempo exponencial en $|x|$ podemos simular determinísticamente a N con entrada x y también cada consulta que hace N al oráculo $\mathbf{EXPCOM}$.

Luego $\mathcal{L} \in \mathbf{EXPTIME}$. □

Corolario

$\mathbf{P}^{\mathbf{EXPCOM}} = \mathbf{NP}^{\mathbf{EXPCOM}}$.

Demostración.

$\mathbf{EXPTIME} \subseteq \mathbf{P}^{\mathbf{EXPCOM}} \subseteq \mathbf{NP}^{\mathbf{EXPCOM}} \subseteq \mathbf{EXPTIME}$ □

10-9

Teorema de Baker, Gill, Solovay

Clase 10

Máquinas con oráculo

Teorema de Baker, Gill, Solovay

La jerarquía polinomial y $\mathbf{NP}$ con oráculos

Teorema de Baker, Gill, Solovay

Teorema (Baker, Gill, Solovay)

Existen oráculos $\mathcal{A}$ y $\mathcal{B}$ tal que $\mathbf{P}^{\mathcal{A}} = \mathbf{NP}^{\mathcal{A}}$ y $\mathbf{P}^{\mathcal{B}} \neq \mathbf{NP}^{\mathcal{B}}$.

$\mathbf{P}^{\mathcal{A}} = \mathbf{NP}^{\mathcal{A}}$.

Tomar $\mathcal{A} = \text{EXPCOM}$. □

10-11

$\mathbf{P}^{\mathcal{B}} \neq \mathbf{NP}^{\mathcal{B}}$: definición de $\mathcal{U}_{\mathcal{B}}$, dado $\mathcal{B}$

Para cualquier $\mathcal{B} \subseteq \{0, 1\}^*$ definimos

$$\mathcal{U}_{\mathcal{B}} = \{1^n : \exists x \in \mathcal{B}, |x| = n\}.$$

Veamos que para cualquier $\mathcal{B}$, $\mathcal{U}_{\mathcal{B}} \in \mathbf{NP}^{\mathcal{B}}$. Definimos la máquina no-determinística N que con oráculo $\mathcal{B}$ y entrada y hace esto:

si y no es de la forma 1^n para algún n , rechazar
si no (supongamos que $y = 1^n$),
inventar x tal que $|x| = n$
consultar si $x \in \mathcal{B}$ y devolver la respuesta

Para todo n tenemos

$$1^n \in \mathcal{L}(N^{\mathcal{B}}) \quad \text{sii} \quad \exists x \in \mathcal{B}, |x| = n \quad \text{sii} \quad 1^n \in \mathcal{U}_{\mathcal{B}}.$$

(y para y no de la forma 1^n tenemos $y \notin \mathcal{L}(N^{\mathcal{B}})$, $y \notin \mathcal{U}_{\mathcal{B}}$)

A continuación definimos un $\mathcal{B}$ para el cual $\mathcal{U}_{\mathcal{B}} \notin \mathbf{P}^{\mathcal{B}}$.

10-12

$\mathbf{P}^{\mathcal{B}} \neq \mathbf{NP}^{\mathcal{B}}$: propiedades de $\mathcal{B}$

Sea M_i la máquina determinística con oráculo representada por la expansión binaria de $i \in \mathbb{N}$ tal que para toda máquina M con oráculo existen infinitos i tal que $M = M_i$.

Definimos $\mathcal{B} = \bigcup_i \mathcal{B}_i$ en etapas y definimos $(n_i)_{i \in \mathbb{N}}$ con estas propiedades:

- $\mathcal{B}_0 = \emptyset$ y $n_0 = 1$
- $\mathcal{B}_i \subseteq \mathcal{B}_{i+1}$ y $n_i < n_{i+1}$
- $x \in \mathcal{B}_i \Rightarrow |x| \leq n_i$ (en particular, cada $\mathcal{B}_i$ es finito)

10-13

$\mathbf{P}^{\mathcal{B}} \neq \mathbf{NP}^{\mathcal{B}}$: idea de la prueba

Idea: diagonalizar.

si M_i con oráculo $\mathcal{B}$ corre en tiempo polinomial, entonces toma la decisión equivocada cuando la entrada es 1^{n_i} :

- si M_i acepta 1^{n_i} entonces nos aseguramos de que ninguna cadena de longitud n_i esté en $\mathcal{B}$ (y por lo tanto $1^{n_i} \notin \mathcal{U}_{\mathcal{B}}$)
- si M_i rechaza 1^{n_i} , entonces metemos en $\mathcal{B}$ alguna cadena de longitud n_i (y por lo tanto $1^{n_i} \in \mathcal{U}_{\mathcal{B}}$)

Así, ninguna máquina que corra en tiempo polinomial decide $\mathcal{U}_{\mathcal{B}}$, de modo que $\mathcal{U}_{\mathcal{B}} \notin \mathbf{P}^{\mathcal{B}}$.

10-14

$P^B \neq NP^B$: construcción de $\mathcal{B}_i$ y para $i > 0$

- definimos $n_i > n_{i-1}$ y $n_i >$ máximo de las longitudes consultadas por $M_k^{\mathcal{B}_{i-1}}(1^{n_k})$ al tiempo 2^{n_k-1} para todo $k < i$
- simular M_i con entrada 1^{n_i} por 2^{n_i-1} pasos
 - si M_i consulta al oráculo por un x con $|x| < n_i$, le respondemos lo mismo que ' $x \in \mathcal{B}_{i-1}$ '
 - si M_i consulta al oráculo por un x con $|x| \geq n_i$, le respondemos 'no'
- si M_i acepta 1^{n_i} en $\leq 2^{n_i-1}$ pasos, definimos $\mathcal{B}_i = \mathcal{B}_{i-1}$
 - $\mathcal{B}_i$ no contiene cadenas de tamaño n_i ($\mathcal{B}$ tampoco)
- si M_i rechaza 1^{n_i} o no llegó a una decisión todavía en 2^{n_i-1} pasos, elegimos un x , $|x| = n_i$ que no haya sido consultado y definimos $\mathcal{B}_i = \mathcal{B}_{i-1} \cup \{x\}$.
 - tal x existe porque simulamos M_i por 2^{n_i-1} pasos y por lo tanto hay $\leq 2^{n_i-1}$ consultas, pero hay 2^{n_i} cadenas de tamaño n_i
 - agregar x no 'rompe' ninguna de las simulaciones de M_k a tiempo 2^{n_k-1} , para $k < i$. A la simulación de M_k por 2^{n_k-1} pasos le daba lo mismo el oráculo $\mathcal{B}_k$ o $\mathcal{B}_i$ (o $\mathcal{B}$)

10-15

$P^B \neq NP^B$: verificación de que $\mathcal{U}_B \notin P^B$

Supongamos que M es una máquina determinística y p es un polinomio tal que

- M^B corre en tiempo $p(n)$
- M^B acepta $\mathcal{U}_B$

Sea i suficientemente grande tal que $M_i = M$ y $2^{n_i-1} > p(n_i)$. Simular $M_i = M$ por 2^{n_i-1} pasos es suficiente para saber si M_i acepta o rechaza 1^{n_i} .

- si M_i^B acepta 1^{n_i} , ninguna cadena de longitud n_i está en $\mathcal{B}$. Entonces $1^{n_i} \notin \mathcal{U}_B$.
- si M_i^B rechaza 1^{n_i} , $\mathcal{B}_i$ contiene una cadena de tamaño n_i . Entonces $1^{n_i} \in \mathcal{U}_B$.

Luego $M^B = M_i^B$ no puede decidir $\mathcal{U}_B$ porque falla para la entrada 1^{n_i} . □

10-16

Clases de complejidad relativizadas a clases de complejidad

Clase de complejidad: $\mathbf{P}^{\mathbf{C}}, \mathbf{NP}^{\mathbf{C}}$

$$\mathbf{P}^{\mathbf{C}} = \bigcup_{\mathcal{X} \in \mathbf{C}} \mathbf{P}^{\mathcal{X}}$$
$$\mathbf{NP}^{\mathbf{C}} = \bigcup_{\mathcal{X} \in \mathbf{C}} \mathbf{NP}^{\mathcal{X}}$$

Observación

Si $\mathcal{X} \in \mathbf{C}$ -completo, entonces $\mathbf{P}^{\mathbf{C}} = \mathbf{P}^{\mathcal{X}}$ y $\mathbf{NP}^{\mathbf{C}} = \mathbf{NP}^{\mathcal{X}}$.

Ejemplo

$$\mathbf{P}^{\mathbf{NP}} = \mathbf{P}^{\mathbf{SAT}} = \mathbf{P}^{\overline{\mathbf{SAT}}} = \mathbf{P}^{\mathbf{coNP}}$$

10-17

La jerarquía polinomial y $\mathbf{NP}$ con oráculos

Clase 10

Máquinas con oráculo

Teorema de Baker, Gill, Solovay

La jerarquía polinomial y $\mathbf{NP}$ con oráculos

10-18

$$\Sigma_{i+1}^P = \mathbf{NP}^{\Sigma_i \text{SAT}}$$

Teorema

Para $i \geq 1$, $\Sigma_{i+1}^P = \mathbf{NP}^{\Sigma_i \text{SAT}}$.

Ejemplo

- $\Sigma_1^P = \mathbf{NP}$ (ya lo vimos; no cae dentro del teorema, salvo que definamos $\Sigma_0 \text{SAT} = \emptyset$)
- $\Sigma_2^P = \mathbf{NP}^{\Sigma_1 \text{SAT}} = \mathbf{NP}^{\text{SAT}} = \mathbf{NP}^{\mathbf{NP}}$ (recordar que $\Sigma_1 \text{SAT} = \text{SAT}$)
- $\Sigma_3^P = \mathbf{NP}^{\Sigma_2 \text{SAT}} = \mathbf{NP}^{\mathbf{NP}^{\mathbf{NP}}}$

10-19

$$\Sigma_{i+1}^P \subseteq \mathbf{NP}^{\Sigma_i \text{SAT}}$$

Sea $\mathcal{L} \in \Sigma_{i+1}^P$ y una máquina determinística M que corre en tiempo polinomial tal que

$$x \in \mathcal{L} \quad \text{sii} \quad \exists u_1 \forall u_2 \dots Q u_{i+1} M(\langle x, u_1, \dots, u_i \rangle) = 1$$

donde $|u_i| = q(|x|)$ para algún polinomio fijo q . Definamos

$$\mathcal{L}' = \{ \langle x, u_1 \rangle : \forall u_2 \dots Q u_{i+1} u_{i+1} M(\langle x, u_1, \dots, u_i \rangle) = 1 \} \in \Pi_i^P \leq_p \overline{\Sigma_i \text{SAT}}$$

Definimos una máquina no-determinística N que con oráculo $\mathcal{L}'$ y entrada x hace esto:

inventar u_1
consultar si $\langle x, u_1 \rangle \in \mathcal{L}'$ y devolver su respuesta

N corre en tiempo lineal.

Tenemos $\mathcal{L}(N^{\mathcal{L}'}) = \mathcal{L}$ y $\mathcal{L}' \leq_p \overline{\Sigma_i \text{SAT}}$.

Entonces $\mathcal{L} \in \mathbf{NP}^{\overline{\Sigma_i \text{SAT}}} = \mathbf{NP}^{\Sigma_i \text{SAT}}$. □

10-20

$$\mathbf{NP}^{\Sigma_i \text{SAT}} \subseteq \Sigma_{i+1}^{\mathbf{P}}$$

Sea $\mathcal{L} \in \mathbf{NP}^{\Sigma_i \text{SAT}}$ y sea N una máquina no-determinística que corre en tiempo polinomial $t(n)$ tal que con oráculo $\Sigma_i \text{SAT}$ decide $\mathcal{L}$.

$$x \in \mathcal{L} \quad \text{sii} \quad \text{existe un cómputo } u \in \{0,1\}^{t(|x|)} \text{ de } N^{\Sigma_i \text{SAT}} \text{ con entrada } x \text{ que llega a } q_{\text{sí}}$$

A lo largo de este cómputo u , N hace consultas $\varphi_1, \dots, \varphi_k$ ($k \leq t(|x|)$), del tipo

$$\varphi_j = \underbrace{\exists \bar{u}_1^j \forall \bar{u}_2^j \dots Q \bar{u}_i^j}_{i-1 \text{ alternancias}} \quad \psi_j(\bar{u}_1, \dots, \bar{u}_i)$$

para $j = 1, \dots, k$ y recibe respuesta $r_j \in \{0,1\}$ ('sí' = $q_{\text{resp:sí}} = 1$ o no = $q_{\text{resp:no}} = 0$)

- si $r_j = 1$ entonces existe $\bar{v}_1^j$ tal que
 $\bar{v}_1^j \models \forall \bar{v}_2^j \dots Q \bar{v}_i^j \quad \psi_j(\bar{v}_1^j, \bar{v}_2^j, \dots, \bar{v}_i^j)$ ($i-2$ alternancias)
- si $r_j = 0$ entonces para todo $\bar{w}_1^j$ tenemos
 $\bar{w}_1^j \not\models \forall \bar{w}_2^j \dots Q \bar{w}_i^j \quad \psi_j(\bar{w}_1^j, \bar{w}_2^j, \dots, \bar{w}_i^j)$, o sea
 $\bar{w}_1^j \models \exists \bar{w}_2^j \dots \overline{Q} \bar{w}_i^j \quad \neg \psi_j(\bar{w}_1^j, \bar{w}_2^j, \dots, \bar{w}_i^j)$ ($i-2$ alternancias)

10-21

$$x \in \mathcal{L}$$

sii

$$\exists u \exists (r_j)_j \exists (\bar{v}_1^j)_j \forall (\bar{w}_1^j)_j$$

N acepta x siguiendo el cómputo u , recibe como respuestas $r_1, \dots, r_k$ (en orden) y para $j = 1, \dots, k$

$r_j = 1$ y $\bar{v}_1^j \models \forall \bar{v}_2^j \exists \bar{v}_3^j \dots Q \bar{v}_i^j \quad \psi_j(\bar{v}_1^j, \bar{v}_2^j, \dots, \bar{v}_i^j)$ o bien

$r_j = 0$ y $\bar{w}_1^j \models \exists \bar{w}_2^j \forall \bar{w}_3^j \dots \overline{Q} \bar{w}_i^j \quad \neg \psi_j(\bar{w}_1^j, \bar{w}_2^j, \dots, \bar{w}_i^j)$

sii

i alternancias

$$\overbrace{\underbrace{\exists}_{\exists} \underbrace{\forall}_{\forall} \underbrace{\exists (\bar{w}_2^j)_j \exists (\bar{v}_3^j)_j \forall (\bar{w}_3^j)_j \dots Q (\bar{v}_i^j)_j \overline{Q} (\bar{w}_i^j)_j}_{i-2 \text{ alternancias}}}_{\exists u, (r_j)_j, (\bar{v}_1^j)_j \forall (\bar{w}_1^j)_j \forall (\bar{v}_2^j)_j}$$

N acepta x siguiendo el cómputo u , recibe como respuestas $r_1, \dots, r_k$ (en orden) y para $j = 1, \dots, k$:

$r_j = 1$ y $\models \psi_j(\bar{v}_1^j, \bar{v}_2^j, \dots, \bar{v}_i^j)$ o bien

$r_j = 0$ y $\models \neg \psi_j(\bar{w}_1^j, \bar{w}_2^j, \dots, \bar{w}_i^j)$

computable
en tiempo
polinomial

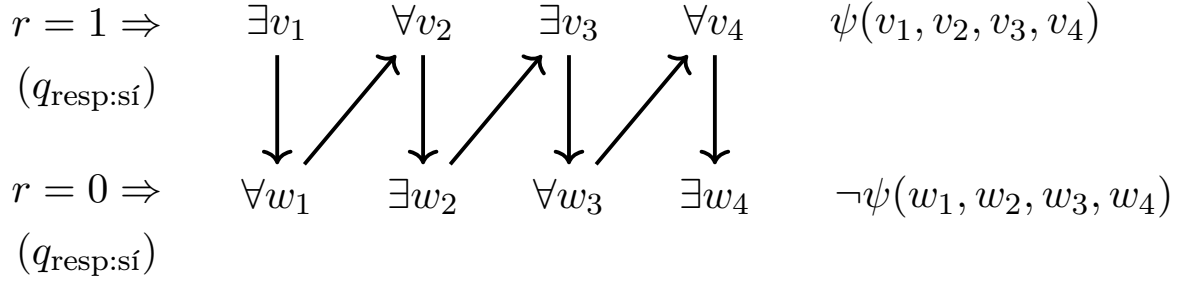
Entonces $\mathcal{L} \in \Sigma_{i+1}^{\mathbf{P}}$.

10-22

Ejemplo (esquema): Supongamos que N acepta x y a lo largo del computo u , N hace 1 sola consulta r al oráculo $\Sigma_4\text{SAT}$:

$$\varphi = \exists x_1 \forall x_2 \exists x_3 \forall x_4 \psi$$

Entonces:



$$\underbrace{\exists u \quad \exists r \quad \exists v_1 \quad \forall w_1 \forall v_2 \quad \exists w_2 \exists v_3 \quad \forall w_3 \forall v_4 \quad \exists w_4}_{4 \text{ alternancias} \Rightarrow \Sigma_5^P} \quad [\text{Polinomial}]$$

sii $x \in \mathcal{L}$

□

10-23

P relativizado a oráculos

Clase de complejidad: Δ_i^P

$$\Delta_1^P = P$$

$$\Delta_{i+1}^P = P^{\Sigma_i^P}$$

Ejemplo

- $\Delta_2^P = P^{\Sigma_1^P} = P^{NP}$
- $\Delta_3^P = P^{\Sigma_2^P} = P^{NP^{NP}}$

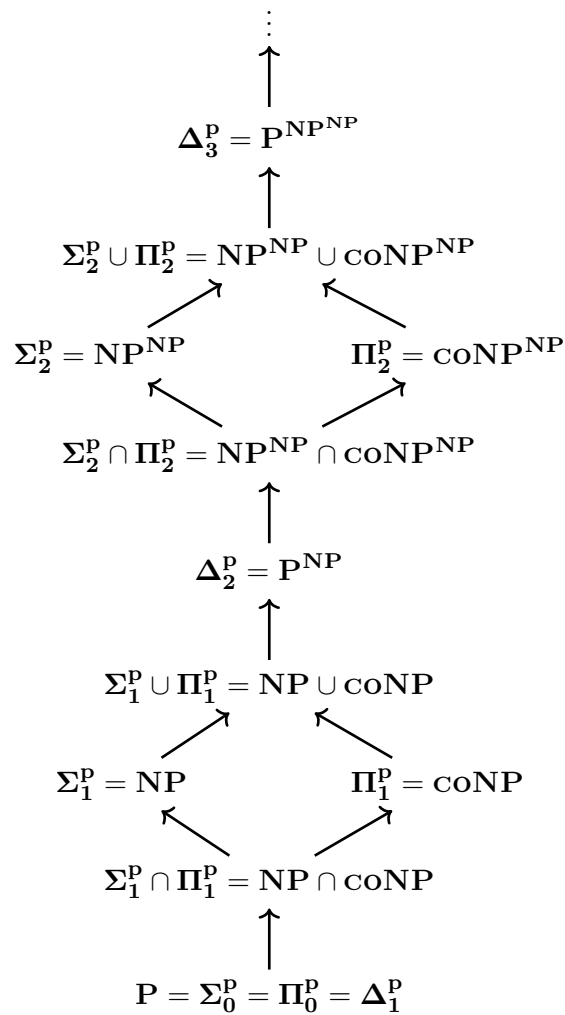
Problema: LEXSAT (Mínima valuación en orden lex)

$$\text{LEXSAT} = \{ \langle v, \varphi \rangle : \begin{array}{l} v \text{ es la valuación más chica en orden} \\ \text{lexicográfico tal que } v \models \varphi \end{array} \}$$

Teorema

LEXSAT es Δ_2^P -completo.

10-24



10-25

Clase 11

Clase 1

Preliminares
Representaciones en binario
Problemas de decisión
Máquinas de Turing
Configuraciones y cálculos

Clase 2

Funciones computables
Tiempo de cómputo
Codificación de máquinas
Variantes de máquinas

Clase 3

Halt y máquina universal
La clase **P**
La clase **NP**

Clase 4

Máquinas no-determinísticas
Reducibilidad polinomial
Lógica proposicional

Clase 5

Mini-configuraciones
Teorema de Cook-Levin
Problemas **NP-completos**
La clase **coNP**
Las clases **EXPTime** y **NEXPTime**

Clase 6

La jerarquía de tiempos determinísticos
La jerarquía de tiempos no-determinísticos
Teorema de Ladner

Clase 7

Espacio usado por un cómputo
Espacio polinomial: **PSPACE** y **NPSpace**
Teorema de Savitch

Clase 8

Espacio logarítmico: **L** y **NL**
Teorema de Immerman-Szelepcsényi

Clase 9

La jerarquía polinomial
Problemas **Σᵢᵖ-completos**

Clase 10

Máquinas con oráculo
Teorema de Baker, Gill, Solovay
La jerarquía polinomial y **NP** con oráculos

Clase 11

Circuitos booleanos
La clase P/poly
Tamaño y profundidad de circuitos

Clase 12

Las clases **NC_{nu}** y **AC_{nu}**
Uniformidad: las clases **NC** y **AC**
P-completitud

Circuitos booleanos

Clase 11

Circuitos booleanos

La clase $\mathbf{P}_{/poly}$

Tamaño y profundidad de circuitos

11-2

Circuitos booleanos

Definición

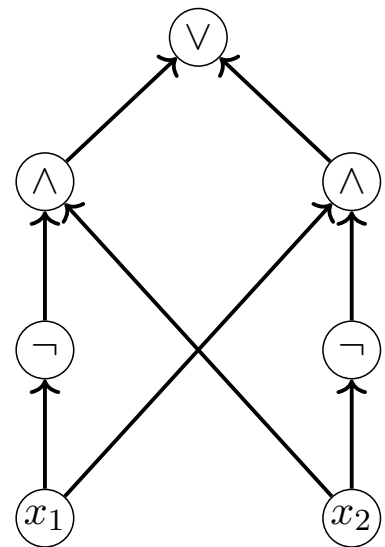
Un **circuito booleano** de n entradas es un grafo acíclico dirigido tal que

- tiene n **entradas** (*sources*) = nodos sin flechas de entrada
- tiene 1 nodo destino o **salida** (*sink*, sumidero) = nodo sin flecha de salida

El resto de los nodos se llaman compuertas (*gates*) y están etiquetados con

- $\neg$: *fan-in* = 1, *fan-out* = 1
- $\wedge$: *fan-in* = 2, *fan-out* = 1
- $\vee$: *fan-in* = 2, *fan-out* = 1

El **tamaño** $|C|$ de un circuito booleano C es la cantidad de vértices de C .



Entrada: x_1, x_2

Tamaño: 7

11-3

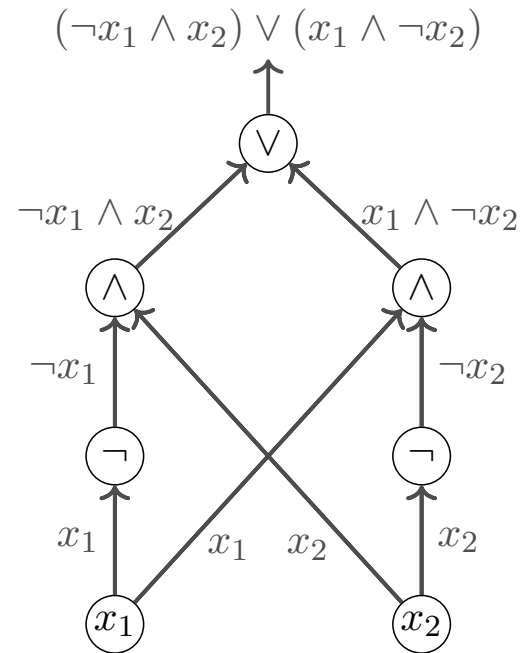
Semántica de circuitos booleanos

Definición

Dado un circuito booleano $C = (V, E)$ con entradas $X = \{x_1, \dots, x_n\}$ y $x \in \{0, 1\}^n$, definimos recursivamente $v : V \rightarrow \{0, 1\}$ de esta forma:

- $v(x_i) = x(i)$
- si en C tenemos $u_1 \rightarrow u$ y u tiene *fan-in* = 1 entonces $v(u) = \neg v(u_2)$ (u tenía etiqueta $\neg$)
- si en C tenemos $u_1 \rightarrow u \leftarrow u_2$, y
 - u tiene etiqueta $\wedge$, entonces $v(u) = v(u_1) \wedge v(u_2)$
 - u tiene etiqueta $\vee$, entonces $v(u) = v(u_1) \vee v(u_2)$

$C(x) = v(t)$, donde t es nodo de salida de C . El circuito C calcula la función $x \mapsto C(x)$.



Circuito C

Entrada: x_1, x_2

Salida: $C(x_1, x_2) =$

$x_1 \oplus x_2 =$

$(\neg x_1 \wedge x_2) \vee (x_1 \wedge \neg x_2)$ 11-4

La clase $\text{SIZE}(S(n))$

Definición

Sea $S : \mathbb{N} \rightarrow \mathbb{N}$. Una **familia de circuitos de tamaño** $S(n)$ es una secuencia $(C_n)_{n \in \mathbb{N}}$ tal que

- C_n es un circuito con n entradas
- $|C_n| \leq S(n)$

Definición

Una familia de circuitos $(C_n)_{n \in \mathbb{N}}$ [de tamaño $S(n)$] **decide** un lenguaje $\mathcal{L}$ si para todo $n \geq 1$ y todo $x \in \{0, 1\}^n$

$$x \in \mathcal{L} \quad \text{sii} \quad C_n(x) = 1$$

Identificamos la tupla $(x_1, \dots, x_n) \in \{0, 1\}^n$ con la cadena $x_1 \dots x_n$.

- Observar que alcanza con que exista la familia de circuitos $(C_n)_{n \in \mathbb{N}}$.
- No pedimos (por ahora) que sea **uniforme**, es decir que exista una máquina determinística M tal que $M(n) = \langle C_n \rangle$.

La clase $\mathbf{SIZE}(S(n))$

Clase de complejidad: $\mathbf{SIZE}(S(n))$

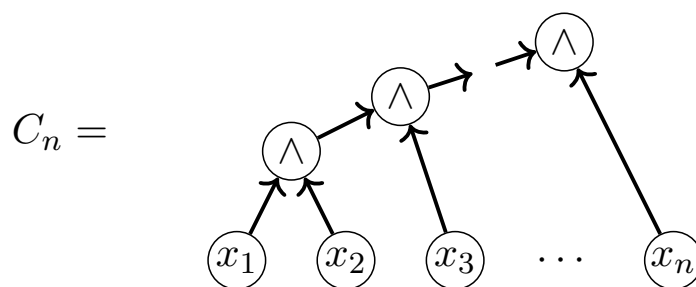
$\mathbf{SIZE}(S(n))$ es la clase de lenguajes $\mathcal{L}$ ($\epsilon \notin \mathcal{L}$) tal que existe un familia de circuitos $(C_n)_{n \in \mathbb{N}}$ de tamaño $S(n)$ que decide $\mathcal{L}$.

11-6

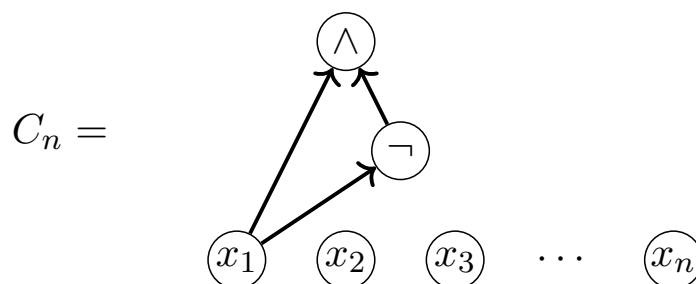
Ejemplo: cualquier lenguaje unario

$$\mathcal{L} \subseteq \{1^n : n \geq 1\}$$

Si $x = x_1 \dots x_n \in \mathcal{L}$ definimos



si no,



$\mathcal{L} \in \mathbf{SIZE}(S(n))$ para S lineal.

11-7

Ejemplo: suma bit a bit

$$\mathcal{L} = \{xyz : |x| = |y| = n, |z| = n + 1, x + y = z, n \geq 1\}$$

0000, 0101, 1001, 1110 $\in \mathcal{L}$

El resto de las palabras de largo 4
no están en $\mathcal{L}$

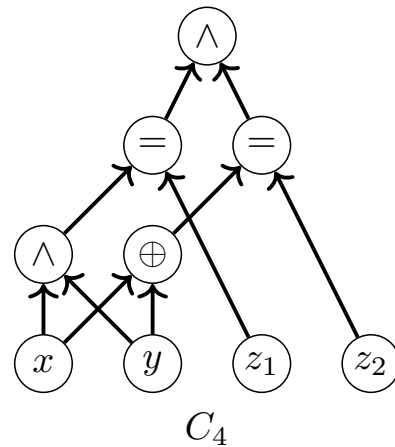
En general,

$$x_n \dots x_1 \ y_n \dots y_1 \ z_{n+1} \dots z_1 \in \mathcal{L}$$

sii

- $c_1 = 0$
- para $j = 1 \dots n$:
 $z_j = x_j \oplus y_j \oplus c_j$
 $c_{j+1} = (c_j \wedge (x_j \vee y_j)) \vee (x_j \wedge y_j)$
- $z_{n+1} = c_{n+1}$

$\mathcal{L} \in \mathbf{SIZE}(S(n))$ para S lineal.



- $\oplus$ representa el
circuito para xor
(o exclusivo)
- $=$ representa el
circuito para el sii

11-8

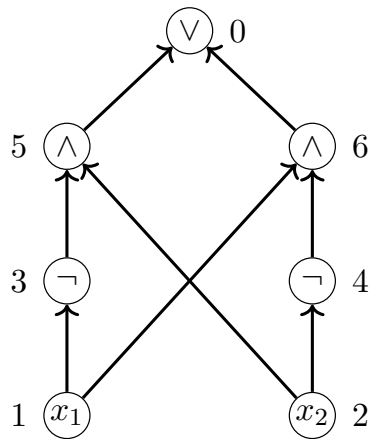
Codificación de un circuito de tamaño S

Representamos C en binario:

- Supongamos nodos de C : $u_0, \dots, u_{S-1}$ de modo tal que $u_1, \dots, u_n$ corresponden a la entrada y u_0 a la salida
- codificamos cada nodo u con $[u]$, es decir, con $\lceil \log S \rceil$ bits
- cada nodo de C tiene a lo sumo dos hijos (porque las operaciones son unarias o binarias)
- codificamos C con $\tau_0 \sigma_0 \dots \tau_{S-1} \sigma_{S-1}$, donde
 - si u_i es una entrada y por lo tanto no tiene hijos
 $\tau_i = 00$ y $\sigma_i = \varepsilon$
 (en realidad alcanza con codificar la cantidad de entradas)
 - si u_i tiene etiqueta $\neg$ (tiene solo 1 hijo v_i),
 $\tau_i = 01$ y $\sigma_i = [v_i]$
 - si u_i tiene etiqueta $\wedge$ y tiene hijos v_i, w_i ,
 $\tau_i = 10$ y $\sigma_i = [v_i][w_i]$
 - si u_i tiene etiqueta $\vee$ y tiene hijos v_i, w_i ,
 $\tau_i = 11$ y $\sigma_i = [v_i][w_i]$
- codificamos C con $\leq S \cdot (2 \cdot \lceil \log S \rceil + 2) \leq 4 \cdot S \cdot \log S$ bits
para $S > 1$.

11-9

Ejemplo: codificación de un circuito



nodo u_i	tipo τ_i	hijos σ_i	
0	$\vee$	5	6
1	entrada		
2	entrada		
3	$\neg$	1	
4	$\neg$	2	
5	$\wedge$	3	2
6	$\wedge$	1	4

tipo τ_i	hijos σ_i	
11	101	110
00		
00		
01	001	
01	010	
10	100	010
10	001	010

11 101 110 00 00 01 001 01 010 10 100 010 10 001 010

(los espacios no forman parte de la codificación)

11-10

La clase $\mathbf{P}_{/\text{poly}}$

Clase 11

Circuitos booleanos

La clase $\mathbf{P}_{/\text{poly}}$

Tamaño y profundidad de circuitos

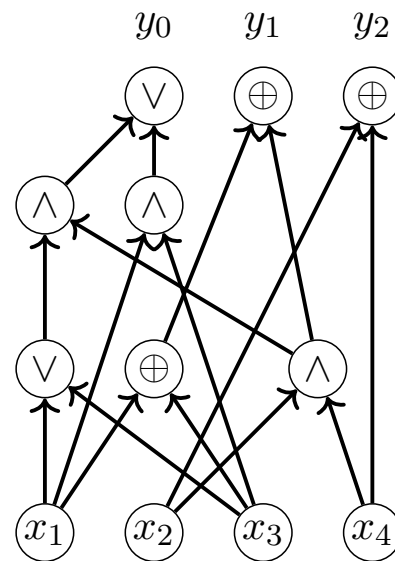
11-11

Circuitos con salidas m -arias

Lo mismo que ya vimos sirve para representar funciones $\{0, 1\}^n \rightarrow \{0, 1\}^m$.

- hay n nodos de entrada (con $\text{fan-in} = 0$)
- hay m nodos de salida (con $\text{fan-out} = 0$)
- las representaciones son análogas a las que vimos solo que representamos todos los nodos de salida

Ejemplo



Calcula $x_1x_2x_3x_4 \mapsto y_0y_1y_2$, donde $y_0y_1y_2 = x_1x_2 + x_3x_4$ (suma en binario)

11-12

Repaso de la idea del teorema de Cook-Levin (una vez más)

Supongamos una máquina determinística M tal que

- M corre en tiempo $t(n)$
- M es *oblivious* y sin cinta de salida
- $x \in \mathcal{L}$ si existe una secuencia de mini-configuraciones $z_0, \dots, z_{t(|x|)}$ de M con entrada x tal que
 - z_0 es inicial para x
 - z_i evoluciona en un paso en z_{i+1}
 - $z_{t(|x|)}$ es final
- cada mini-configuración tiene información sobre el estado, el bit leído de la cinta de trabajo y el bit leído de la cinta de entrada
- Recordar que, para $n = |x|$:
 - $|z_i| = k$, k depende solo de M
 - z_0 solo depende de x
 - z_{i+1} solo depende de 1 bit de x , z_i y $z_{\text{prev}(i,n)}$, donde $\text{prev}(i, n) = \max\{j < i : t(j, n) = t(i, n)\} \cup \{1\}$ ($t(n, j)$ es la posición de la cabeza de trabajo al paso j)

11-13

La clase $\mathbf{P}_{/\text{poly}}$

Clase de complejidad: $\mathbf{P}_{/\text{poly}}$

$$\mathbf{P}_{/\text{poly}} = \bigcup_c \text{SIZE}(c \cdot n^c)$$

11-14

Teorema

$$\mathbf{P} \subseteq \mathbf{P}_{/\text{poly}}.$$

Demostración

Sea $\mathcal{L} \in \mathbf{P}$ y sea M una máquina determinística tal que

- M corre en tiempo $t(n)$ con t un polinomio
- M es *oblivious*, con 1 cinta de trabajo, sin cinta de salida
- $x \in \mathcal{L}$ sii existe una secuencia de mini-configuraciones $z_0, \dots, z_{t(|x|)}$ de M con entrada x tal que
 - z_0 es inicial para x
 - z_i evoluciona en z_{i+1}
 - $z_{t(|x|)}$ es final

Representamos z_i con la cadena ets , donde:

- $e \in \{0, 1\}^2$ codifica el símbolo en la cinta de entrada,
- $t \in \{0, 1\}^2$ codifica el símbolo en la cinta de trabajo,
- $s \in \{0, 1\}^c$ codifica el estado de M (c es constante)

Entonces $|z_i| = k = 4 + c$ (constante).

11-15

Para n fijo, construimos un circuito C_n tal que $M(x) = C_n(x)$ para todo $x \in \{0, 1\}^n$.

Sea $x \in \{0, 1\}^n$ y $z_i = ets$, $|z_i| = 4 + c$.
 z_{i+1} solo depende de

- el símbolo leído por la cabeza de entrada (codificado con 2 bits),
- z_i
- $z_{prev(i,n)}$

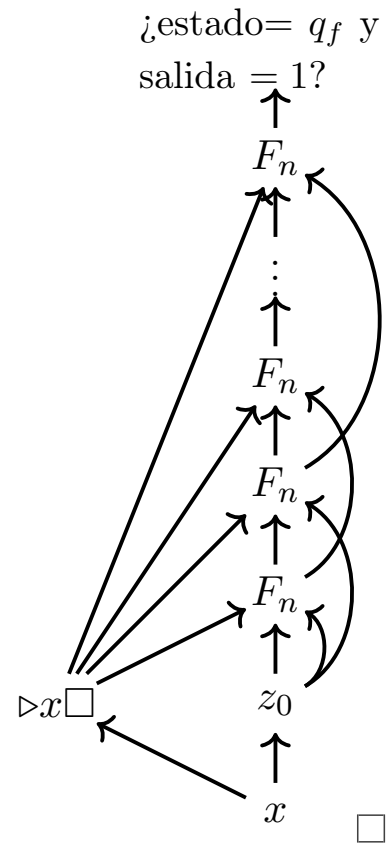
Sea $F_n : \{0, 1\}^{2k+2} \rightarrow \{0, 1\}^k$ tal que

$$F_n(e \ z_i \ z_{prev(i,n)}) = z_{i+1}$$

F_n se representa con un circuito de tamaño constante.

Entonces todo el circuito C_n cumple

$$|C_n| = O(t(n)) \quad y \quad C_n(x) = M(x)$$



11-16

Teorema

$\mathbf{P} \not\subseteq \mathbf{P}_{/poly}$.

Demostración.

Recordemos que si $\mathcal{L} \subseteq \{1^n : n \in \mathbb{N}\}$ entonces $\mathcal{L} \in \mathbf{P}_{/poly}$.
 Podemos tomar un $\mathcal{L} \subseteq \{1^n : n \in \mathbb{N}\}$ indecidible, por ejemplo

$$\mathcal{H} = \{1^n : x \text{ es la } n\text{-ésima cadena y } halt(x) = 1\}$$

(numeramos las cadenas $\epsilon, 0, 1, 00, 01, 10, 11, 000, \dots$)

$\mathcal{H} \in \mathbf{P}_{/poly} \setminus \mathbf{P}$. □

11-17

Notación para reemplazo de variables por constantes

Notación: Variables de una fórmula booleana

Si φ es una fórmula booleana con variables entre $x_1, \dots, x_n$ (pueden ser menos), la notamos

$$\varphi(x_1, \dots, x_n).$$

Notación: Reemplazo de variable por constante

Sea $\varphi(x_1, \dots, x_n)$ una fórmula booleana. Representamos

$$\varphi(x_1, \dots, x_{i-1}, b, x_{i+1}, \dots, x_n)$$

a la fórmula booleana que resulta de reemplazar todas las apariciones de x_i por $b \in \{0, 1\}$.

Notar que

$$\underbrace{b_1 \dots b_n}_{\substack{\text{valuación} \\ v \in \{0, 1\}^n}} \models \underbrace{\varphi(x_1, \dots, x_n)}_{\text{variables}} \quad \text{sii} \quad \models \underbrace{\varphi(b_1, \dots, b_n)}_{\substack{\text{constantes} \\ v \in \{0, 1\}^n}}$$

11-18

Ejemplo

$$\varphi(x_1, x_2, x_3) = (x_1 \wedge \neg x_2) \vee (x_3 \wedge x_2)$$

$$\varphi(x_1, 1, x_3) = (x_1 \wedge \neg 1) \vee (x_3 \wedge 1)$$

$$\varphi(x_1, 0, x_3) = (x_1 \wedge \neg 0) \vee (x_3 \wedge 0)$$

$$110 \models \varphi(x_1, x_2, x_3) \text{ sii } \models \varphi(1, 1, 0) \text{ sii } \models (1 \wedge \neg 1) \vee (0 \wedge 1)$$

Encontrar valuaciones booleanas

Supongamos $\varphi(x_1, \dots, x_n)$ satisfacible. Entonces

$$\varphi(1, x_2, \dots, x_n) \in \text{SAT} \implies \exists v \in \{0, 1\}^{n-1} \ v \models \varphi(1, x_2, \dots, x_n)$$

$$\varphi(1, x_2, \dots, x_n) \notin \text{SAT} \implies \exists v \in \{0, 1\}^{n-1} \ v \models \varphi(0, x_2, \dots, x_n)$$

Es decir,

$$\varphi(\overbrace{\chi_{\text{SAT}}(\varphi(1, x_2, \dots, x_n))}^{\in \{0,1\}}, x_2, \dots, x_n)$$

es satisfacible.

Podemos seguir adelante con el mismo razonamiento:

$$\varphi(\overbrace{\chi_{\text{SAT}}(\varphi(1, x_2, \dots, x_n))}^{b \in \{0,1\}}, \overbrace{\chi_{\text{SAT}}(\varphi(b, 1, x_3, \dots, x_n))}^{\in \{0,1\}}, x_3, \dots, x_n)$$

es satisfacible. Y así sucesivamente.

11-20

Circuito para encontrar valuaciones booleanas

- Podemos codificar fórmulas booleanas (con constantes 0 y 1) en binario.
- Cada fórmula de tamaño n tiene a lo sumo n variables y las suponemos numeradas $x_1, \dots, x_i, i \leq n$.
- Consideramos circuitos booleanos con n entradas y m salidas, que representan funciones $\{0, 1\}^n \rightarrow \{0, 1\}^m$.

Proposición

Sea $(C_n)_{n \in \mathbb{N}}$ una familia de circuitos de tamaño $S(n)$ tal que $C_n(\varphi) = \chi_{\text{SAT}}(\varphi)$ para toda fórmula booleana $\varphi = \varphi(x_1, \dots, x_n)$ con $|\varphi| = n$. Entonces existe una familia de circuitos $(C'_n)_{n \in \mathbb{N}}$ de tamaño polinomial en $S(n)$ tal que para todo n : C'_n tiene n salidas y para toda fórmula booleana $\varphi = \varphi(x_1, \dots, x_n)$ con $|\varphi| = n$, tenemos

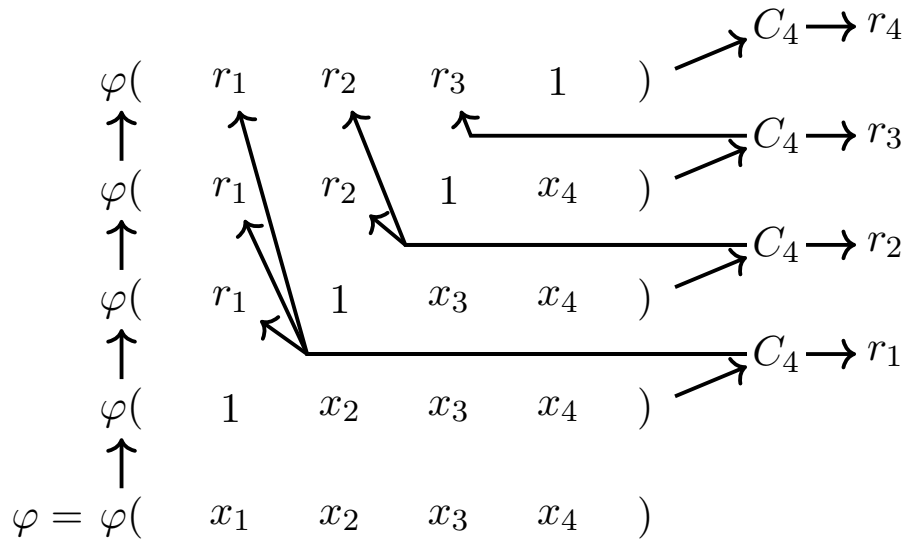
$$\varphi(C'_n(\varphi)) = \chi_{\text{SAT}}(\varphi).$$

(notamos $C_n(\varphi)$ en vez de $C_n(\langle \varphi \rangle)$ y lo mismo para C'_n)

11-21

Demostración.

Dada $\varphi = \varphi(x_1, \dots, x_n)$ de tamaño n y el circuito C_n tal que $C_n(\varphi) = \chi_{\text{SAT}}(\varphi)$ definimos el circuito C'_n con entrada φ y salidas $r_1, \dots, r_n$ de esta forma: (ejemplo para $n = 4$):



$\varphi \in \text{SAT}$ sii $r_1 \dots r_n \models \varphi$ sii $C'_n(\varphi) \models \varphi$ □

11-22

Problemas Π_i^P -completos

Ya probamos que para todo $i > 0$, $\Sigma_i \text{SAT} \in \Sigma_i^P$ -completo.
Análogamente,

Problema: Satisfacibilidad de QBF acotada

φ es una QBF de la forma
 $\forall \bar{y}_1 \exists \bar{y}_2 \dots Q_i \bar{y}_i \psi(\bar{y}_1, \dots, \bar{y}_i)$ donde las $\bar{y}_i$ son
 $\Pi_i \text{SAT} = \{ \langle \varphi \rangle : \text{tuplas de variables booleanas, } \psi \text{ es una fórmula booleana, los cuantificadores se alternan y } \models \varphi \}$

Proposición

Para todo $i > 0$, $\Pi_i \text{SAT} \in \Pi_i^P$ -completo.

11-23

Teorema de Karp-Lipton

Teorema (Karp-Lipton)

Si $\mathbf{NP} \subseteq \mathbf{P}_{/\text{poly}}$, entonces $\mathbf{PH} = \Sigma_2^{\mathbf{P}}$.

Como se cree que $\mathbf{PH}$ no colapsa, es un indicio de que $\mathbf{NP} \not\subseteq \mathbf{P}_{/\text{poly}}$.

Notación: Reemplazo de variables por constantes

Para la fórmula booleana

$$\varphi = \varphi(x_1, \dots, x_i, y_1, \dots, y_j)$$

con variables $x_1, \dots, x_i, y_1, \dots, y_j$, y $\bar{u} = u_1, \dots, u_i \in \{0, 1\}^*$ notamos

$$\varphi_{\bar{u}} = \varphi_{\bar{u}}(y_1, \dots, y_j) = \varphi(u_1, \dots, u_i, y_1, \dots, y_j)$$

11-24

Demostración

Alcanza con probar $\Pi_2^{\mathbf{P}} \subseteq \Sigma_2^{\mathbf{P}}$, o $\Pi_2\text{SAT} \in \Sigma_2^{\mathbf{P}}$ porque $\Pi_2\text{SAT}$ es $\Pi_2^{\mathbf{P}}$ -completo.

Supongamos $\mathbf{NP} \subseteq \mathbf{P}_{/\text{poly}}$. Existe un polinomio p y una familia de circuitos $(C_n)_{n \in \mathbb{N}}$ de tamaño $p(n)$ tal que para toda fórmula booleana φ de tamaño n

$$\begin{aligned} \varphi \in \text{SAT} \quad & \text{sii} \quad \exists \bar{v} \in \{0, 1\}^n \quad \bar{v} \models \varphi \\ & \text{sii} \quad \exists \bar{v} \in \{0, 1\}^n \quad \models \varphi(\bar{v}) \\ & \text{sii} \quad C_n(\varphi) = 1 \end{aligned}$$

Supongamos $\varphi = \varphi(\underbrace{x_1, \dots, x_i}_{\bar{x}}, \underbrace{y_1, \dots, y_j}_{\bar{y}})$ de tamaño n :

$$\forall \bar{u} \in \{0, 1\}^i \left(\underbrace{\exists \bar{v} \in \{0, 1\}^j \quad \models \varphi_{\bar{u}}(\bar{v})}_{\varphi_{\bar{u}} = \varphi_{\bar{u}}(\bar{y}) \in \text{SAT}} \iff C_n(\varphi_{\bar{u}}) = 1 \right)$$

11-25

$$\forall \bar{x} \exists \bar{y} \varphi(\bar{x}, \bar{y}) \in \Pi_2 \text{SAT} \quad \text{sii}$$

$$\forall \bar{u} \in \{0, 1\}^i \overbrace{\exists \bar{v} \in \{0, 1\}^j \models \varphi(\bar{u}, \bar{v})}^{\varphi_{\bar{u}} = \varphi_{\bar{u}}(y) \in \text{SAT}} \quad \text{sii}$$

$$\forall \bar{u} \in \{0, 1\}^i C_n(\varphi_{\bar{u}}) = 1 \quad \text{sii}$$

$$\forall \bar{u} \in \{0, 1\}^i \models \varphi_{\bar{u}}(C'_n(\varphi_{\bar{u}})) \quad \text{sii}$$

$$\underbrace{\exists c \in \{0, 1\}^{q(n)} \forall \bar{u} \in \{0, 1\}^i \models \varphi_{\bar{u}}(R_c(\varphi_{\bar{u}}))}_{\exists c \in \{0, 1\}^{q(n)} \forall \bar{u} \in \{0, 1\}^i [\text{Polinomial}] \in \Sigma_2^P}$$

- $|\varphi(\bar{x}, \bar{y})| = |\varphi_{\bar{u}}(\bar{y})| = n$
- C'_n es tiene una codificación de tamaño $q(n)$ con q un polinomio tal que $\psi(C'_n(\psi)) = \chi_{\text{SAT}}(\psi)$ para toda ψ de tamaño n
- R_c es el circuito representado por $c \in \{0, 1\}^*$
- Decidimos $\varphi_{\bar{u}}(R_c(\varphi_u))$ en tiempo polinomial □

11-26

Tamaño y profundidad de circuitos

Clase 11

Circuitos booleanos

La clase $\mathbf{P}_{/\text{poly}}$

Tamaño y profundidad de circuitos

11-27

Cotas al tamaño mínimo de circuitos

Definición

Sea $f : \{0, 1\}^n \rightarrow \{0, 1\}$. Definimos

$$\text{minSize}(f) = \min\{|C| : C \text{ tiene } n \text{ entradas y } \forall \bar{x} \ C(\bar{x}) = f(\bar{x})\}$$

Proposición

$\text{minSize}(f) = O(n2^n)$ para toda $f : \{0, 1\}^n \rightarrow \{0, 1\}$.

Demostración.

Ya lo vimos para CNF. Más directo es verlo para DNF (forma normal disyuntiva):

$$f(x_1, \dots, x_n) = \bigvee_{\substack{(y_1, \dots, y_n) \in \{0, 1\}^n : \\ f(y_1, \dots, y_n) = 1}} \bigwedge_{1 \leq i \leq n} x_i = y_i$$

La subfórmula $x_i = y_i$ es x_i si $y_i = 1$ y $\neg x_i$ si no.

La fórmula se representa con un circuito de tamaño $O(n2^n)$. $\square$

11-28

Teorema

Para todo $n > 1$, existe $f : \{0, 1\}^n \rightarrow \{0, 1\}$ tal que $\text{minSize}(f) > 2^n/4n$.

Demostración.

Recordemos que codificamos un circuito de tamaño S con $\leq 4 \cdot S \cdot \log S$ bits. Para una función $S : \mathbb{N} \rightarrow \mathbb{N}$ cualquiera, definimos

$$K_{n,S} = \{\text{circuitos con } n \text{ entradas de tamaño } \leq S(n)\}$$
$$\#K_{n,S} \leq 2^{4S(n) \log S(n)}$$

Tomemos $S(n) = 2^n/4n$. Codificamos $C \in K_{n,S}$ con a lo sumo esta cantidad de bits:

$$4S(n) \log S(n) = 4 \frac{2^n}{4n} \log \frac{2^n}{4n}$$
$$< \frac{2^n}{n} \log 2^n = 2^n$$

Entonces $\#K_{n,S} < 2^{2^n}$.

La cantidad de funciones $\{0, 1\}^n \rightarrow \{0, 1\}$ es 2^{2^n} . Por lo tanto existe una función $f : \{0, 1\}^n \rightarrow \{0, 1\}$ tal que $\text{minSize}(f) > 2^n/4n$. $\square$

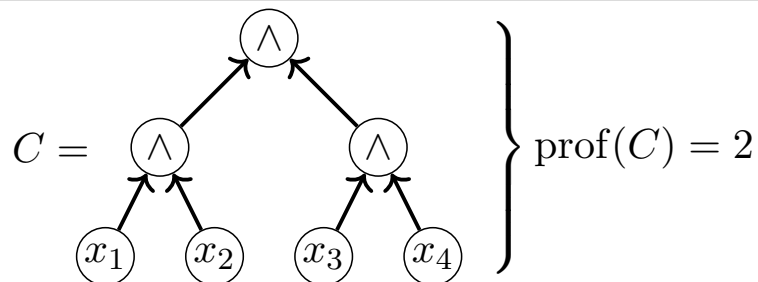
11-29

Profundidad de un circuito

Definición

La **profundidad** de un circuito C , notado $\text{prof}(C)$, es la longitud del camino más largo desde alguna entrada hasta alguna salida.

Profundidad de un circuito



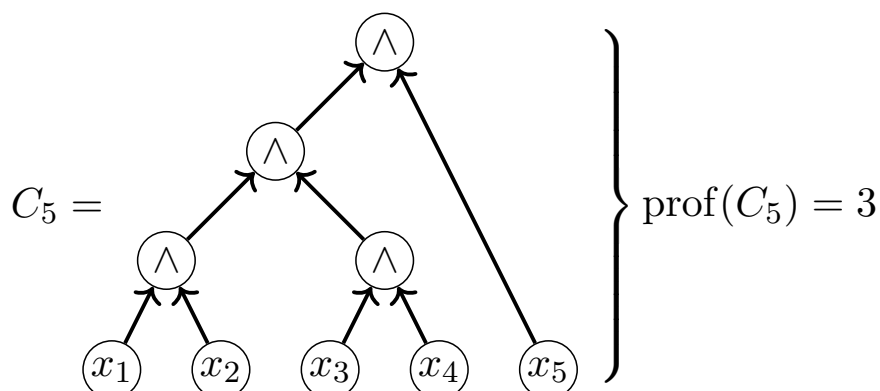
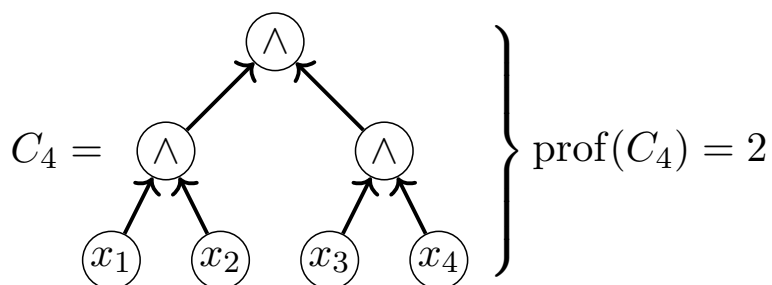
Definición

Una familia de circuitos $(C_n)_{n \in \mathbb{N}}$ tiene profundidad $d(n)$ si $\text{prof}(C_n) \leq d(n)$ para todo n .

11-30

Profundidad logarítmica

$\{1^n : n \geq 1\}$ es decidable por una familia de circuitos de profundidad $O(\log n)$



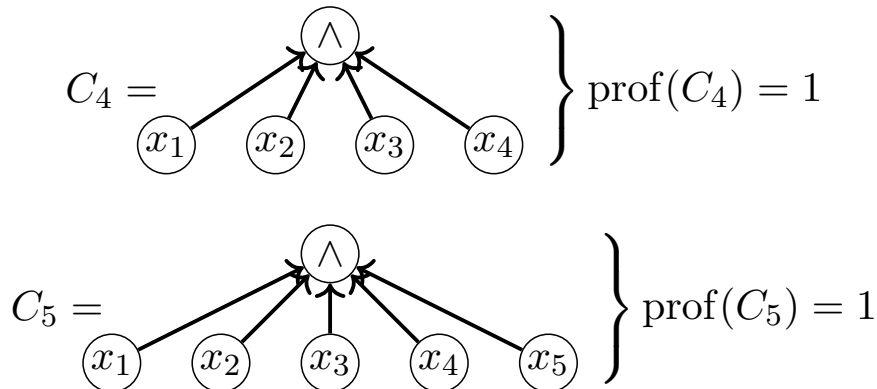
11-31

Conjunciones y disyunciones de aridad arbitraria

A veces vamos a considerar circuitos con $\wedge$ y $\vee$ con *fan-in* arbitrario. Esto permite profundidades más bajas.

Profundidad constante

Existe una familia de circuitos con $\wedge$ de *fan-in* arbitrario de profundidad $O(1)$ que acepta $\{1^n : n \geq 1\}$.



11-32

Clase 12

Clase 1

- Preliminares
- Representaciones en binario
- Problemas de decisión
- Máquinas de Turing
- Configuraciones y cálculos

Clase 2

- Funciones computables
- Tiempo de cómputo
- Codificación de máquinas
- Variantes de máquinas

Clase 3

- Halt y máquina universal
- La clase **P**
- La clase **NP**

Clase 4

- Máquinas no-determinísticas
- Reducibilidad polinomial
- Lógica proposicional

Clase 5

- Mini-configuraciones
- Teorema de Cook-Levin
- Problemas **NP-completos**
- La clase **coNP**
- Las clases **EXPTIME** y **NEXPTIME**

Clase 6

- La jerarquía de tiempos determinísticos
- La jerarquía de tiempos no-determinísticos
- Teorema de Ladner

Clase 7

- Espacio usado por un cómputo
- Espacio polinomial: **PSPACE** y **NPSPACE**
- Teorema de Savitch

Clase 8

- Espacio logarítmico: **L** y **NL**
- Teorema de Immerman-Szelepcsényi

Clase 9

- La jerarquía polinomial
- Problemas Σ_i^P -completos

Clase 10

- Máquinas con oráculo
- Teorema de Baker, Gill, Solovay
- La jerarquía polinomial y **NP** con oráculos

Clase 11

- Circuitos booleanos
- La clase **P/poly**
- Tamaño y profundidad de circuitos

Clase 12

- Las clases **NC_{nu}** y **AC_{nu}**
- Uniformidad: las clases **NC** y **AC**
- P-completitud**

Las clases $\mathbf{NC}_{\text{nu}}$ y $\mathbf{AC}_{\text{nu}}$

Clase 12

Las clases $\mathbf{NC}_{\text{nu}}$ y $\mathbf{AC}_{\text{nu}}$

Uniformidad: las clases $\mathbf{NC}$ y $\mathbf{AC}$

$\mathbf{P}$ -completitud

12-2

Las clases $\mathbf{NC}_{\text{nu}}$ y $\mathbf{AC}_{\text{nu}}$

Clase de complejidad: $\mathbf{NC}_{\text{nu}}^d$ y $\mathbf{NC}_{\text{nu}}$

- $\mathbf{NC}_{\text{nu}}^d$ es la clase de lenguajes decidibles por una familia de circuitos $(C_n)_{n \in \mathbb{N}}$ tal que
 - $|C_n|$ es polinomial en n y
 - $\text{prof}(C_n) = O(\log^d n)$
- $\mathbf{NC}_{\text{nu}} = \bigcup_{i \geq 0} \mathbf{NC}_{\text{nu}}^i$

Clase de complejidad: $\mathbf{AC}_{\text{nu}}^d$ y $\mathbf{AC}_{\text{nu}}$

- $\mathbf{AC}_{\text{nu}}^d$ se define igual que $\mathbf{NC}_{\text{nu}}^d$ pero permitimos nodos $\wedge$ y $\vee$ con *fan-in* arbitrario.
- $\mathbf{AC}_{\text{nu}} = \bigcup_{i \geq 0} \mathbf{AC}_{\text{nu}}^i$

(en el nombre, ‘nu’ es por ‘no uniforme’)

12-3

$\text{NC}_{\text{nu}}^0, \text{AC}_{\text{nu}}^0, \text{NC}_{\text{nu}}^1$

- NC_{nu}^0 es muy limitado porque solo depende de una cantidad finita de bits de entrada
- AC_{nu}^0 no tiene esa misma limitación, por ejemplo

$$\{1^n : n \geq 1\} \in \text{AC}_{\text{nu}}^0.$$

Problema: Cantidad impar de 1s

$$\text{PARITY} = \{x : x \text{ tiene una cantidad impar de 1s}\}$$

Proposición

$$\text{PARITY} \in \text{NC}_{\text{nu}}^1.$$

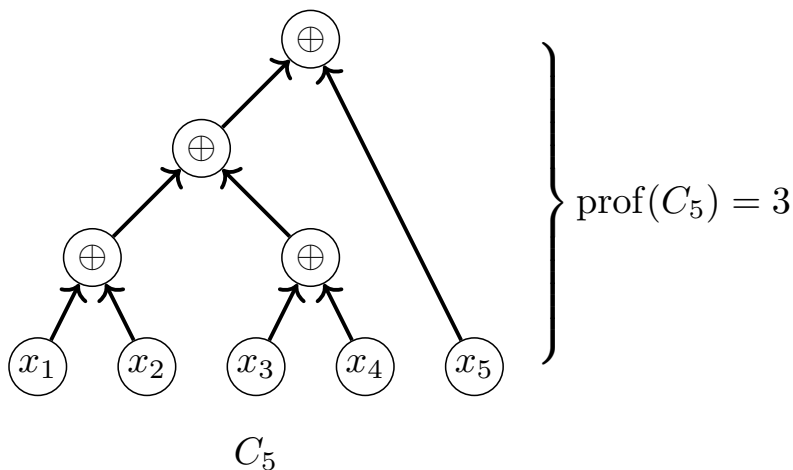
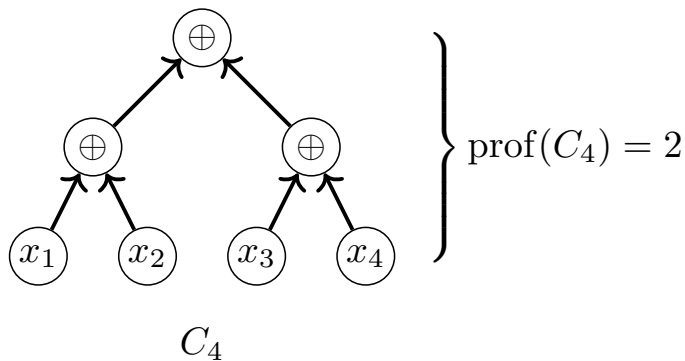
Proposición (no lo vamos a probar)

$$\text{PARITY} \notin \text{AC}_{\text{nu}}^0.$$

12-4

Demostración de $\text{PARITY} \in \text{NC}_{\text{nu}}^1$.

Sumamos (módulo 2) todos los bits de x .



En general,
 $\text{prof}(C_n) = O(\log n)$

□

12-5

Problema: Suma en binario

$$\text{SUMA} = \{xyz : |x| = |y| = i, |z| = i + 1, x + y = z, i \geq 1\}$$

Proposición

$$\text{SUMA} \in \text{AC}_{\text{nu}}^0.$$

12-6

Demostración.

Entrada: $x_n, \dots, x_1, y_n, \dots, y_1, z_{n+1}, z_n, \dots, z_1$.

$$\begin{array}{rcccccccc} \text{carry} \rightarrow & \dots & 1 & 1 & 1 & & & 1 \\ x = & \dots & & 1 & 0 & 1 & 1 & 0 & 1 \\ y = & \dots & & 0 & 1 & 1 & 0 & 0 & 1 \\ z = & \dots & 1 & 0 & 1 & 0 & 1 & 1 & 0 \\ & & \downarrow & & & \downarrow & & & \\ & & i+1 & & & j & & & \end{array}$$

Calculamos el *carry* c_i de esta forma:

$$c_1 = 0 \quad \text{y} \quad c_{i+1} = \bigvee_{i \geq j \geq 1} \left((x_j \wedge y_j) \wedge \bigwedge_{i \geq k > j} (x_k \vee y_k) \right)$$

Cada circuito para c_i tiene profundidad 3. Calculamos la suma de la forma usual: $s_i = x_i \oplus y_i \oplus c_i$ y $s_{n+1} = c_{n+1}$. Finalmente, la salida es

$$\bigwedge_{n+1 \geq j \geq 1} z_i = s_i$$

y lo calculamos con un circuito de profundidad constante. □

12-7

Proposición

$\mathbf{NC}_{\text{nu}}^d \subseteq \mathbf{AC}_{\text{nu}}^d \subseteq \mathbf{NC}_{\text{nu}}^{d+1}$. Por lo tanto, $\mathbf{AC}_{\text{nu}} = \mathbf{NC}_{\text{nu}}$.

Demostración.

$\mathbf{NC}_{\text{nu}}^d \subseteq \mathbf{AC}_{\text{nu}}^d$ es trivial. Veamos $\mathbf{AC}_{\text{nu}}^d \subseteq \mathbf{NC}_{\text{nu}}^{d+1}$.

Supongamos que $\mathcal{L} \in \mathbf{AC}_{\text{nu}}^d$. Existe una familia de circuitos con $\wedge, \vee$ de *fan-in* arbitrario $(C_i)_{i \in \mathbb{N}}$ que decide $\mathcal{L}$,

$$|C_n| = O(n^c) \quad \text{y} \quad \text{prof}(C_n) = O(\log^d n).$$

Simulamos el *fan-in* arbitrario con un circuito con *fan-in* binario de profundidad $O(\log n)$.

Definimos C'_n como el reemplazo en C_n de todo nodo de *fan-in* arbitrario con el subcircuito de profundidad $O(\log n^c) = O(\log n)$.

Es claro que $(C'_i)_{i \in \mathbb{N}}$ decide $\mathcal{L}$.

- C_n tiene profundidad $O(\log^d)$
- C'_n tiene profundidad $O(\log n \cdot \log^d n) = O(\log^{d+1} n)$
- $|C'_n|$ es polinomial

□

12-8

Uniformidad: las clases $\mathbf{NC}$ y $\mathbf{AC}$

Clase 12

Las clases $\mathbf{NC}_{\text{nu}}$ y $\mathbf{AC}_{\text{nu}}$

Uniformidad: las clases $\mathbf{NC}$ y $\mathbf{AC}$

$\mathbf{P}$ -completitud

12-9

Circuitos $\mathbf{P}$ -uniformes

Puede parecer extraño que lenguajes indecidibles estén en $\mathbf{P}_{/\text{poly}}$. El origen de este fenómeno es la no uniformidad del modelo de circuitos.

Definición

Una familia de circuitos $(C_n)_{n \in \mathbb{N}}$ es **$\mathbf{P}$ -uniforme** si existe una máquina determinística tal que

- M corre en tiempo polinomial
- $M(1^n) = \langle C_n \rangle$

12-10

Circuitos $\mathbf{P}$ -uniformes y $\mathbf{P}$

Teorema

$\mathcal{L}$ es decidable por una familia $\mathbf{P}$ -uniforme de circuitos sii $\mathcal{L} \in \mathbf{P}$.

Demostración.

$\subseteq$ Sea M una máquina determinística que corre en tiempo polinomial tal que para todo n , $M(1^n) = \langle C_n \rangle$ y para todo $x \in \{0, 1\}^n$:

$$x \in \mathcal{L} \quad \text{sii} \quad C_n(x) = 1.$$

Definimos la máquina determinística M' que con entrada x hace esto:

Simular $M(1^{|x|}) = \langle C \rangle$

Evaluar $C(x)$ y devolver su salida

La simulación y la evaluación lleva tiempo polinomial.

Como $M'(x) = C_{|x|}(x) = \chi_{\mathcal{L}}(x)$, concluimos que $\mathcal{L} \in \mathbf{P}$.

$\supseteq$ Seguir la demostración $\mathbf{P} \subseteq \mathbf{P}_{/\text{poly}}$. La familia que se construye es, de hecho, $\mathbf{P}$ -uniforme.

□

12-11

Circuitos **L**-uniformes

Definición

Una familia de circuitos $(C_n)_{n \in \mathbb{N}}$ es **L-uniforme** si existe una función $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ computable implícitamente en **L** tal que

$$f(1^n) = \langle C_n \rangle.$$

12-12

Circuitos **L**-uniformes y **P**

Teorema

$\mathcal{L}$ es decidable por una familia de circuitos **L**-uniforme sii $\mathcal{L} \in \mathbf{P}$.

Idea de la demostración.

- $\subseteq$ es consecuencia de que $\mathbf{L} \subseteq \mathbf{P}$ y de que si $\mathcal{L}$ es decidable por una familia de circuitos **P**-uniforme entonces $\mathcal{L} \in \mathbf{P}$.
- $\supseteq$ Ver que en la demostración de $\mathbf{P} \subseteq \mathbf{P}_{\text{poly}}$ podemos construir los circuitos usando espacio logarítmico, gracias a que la máquina es *oblivious*.

Ejercicio

Demostrarlo.

12-13

Las clases $\mathbf{NC}_{\text{nu}}$ y $\mathbf{AC}_{\text{nu}}$ generadas $\mathbf{L}$ -uniformemente

Clase de complejidad: $\mathbf{NC}$

- $\mathbf{NC}^d$ es la clase de lenguajes decidibles por una familia $\mathbf{L}$ -uniforme $(C_n)_{n \in \mathbb{N}}$ de circuitos tal que
 - $|C_n|$ es polinomial en n y
 - $\text{prof}(C_n) = O(\log^d n)$
- $\mathbf{NC} = \bigcup_{i \geq 0} \mathbf{NC}^i$

Clase de complejidad: $\mathbf{AC}$

- $\mathbf{AC}^d$ se define igual que $\mathbf{NC}^d$ pero permitimos nodos $\wedge$ y $\vee$ con *fan-in* arbitrario.
- $\mathbf{AC} = \bigcup_{i \geq 0} \mathbf{AC}^i$

12-14

Resultados ya vistos se adaptan a la uniformidad

Proposición

$\text{PARITY} \in \mathbf{NC}^1$.

Proposición

$\text{SUMA} \in \mathbf{AC}^0$.

Proposición

$\mathbf{NC}^d \subseteq \mathbf{AC}^d \subseteq \mathbf{NC}^{d+1}$. Por lo tanto, $\mathbf{AC} = \mathbf{NC}$.

Observación

$\mathbf{AC} = \mathbf{NC} \subseteq \mathbf{P}$

12-15

C  mputo paralelo

Un problema tiene una **soluci  n paralela eficiente** si puede ser resuelto para entradas de tama  o n usando una computadora paralela con una cantidad polinomial ($n^{O(1)}$) de procesadores en tiempo polilogar  tmico ($\log^{O(1)} n$).

Teorema

$\mathcal{L}$ tiene una soluci  n paralela eficiente sii $\mathcal{L} \in \mathbf{NC}$.

Ejemplos de problemas en $\mathbf{NC}$

- en enteros: suma, multiplicaci  n y divisi  n
- en matrices: suma y multiplicaci  n, determinante, inversa, rango
- en grafos: camino m  nimo, spanning tree m  nimo
- ...

12-16

Teorema

$\mathbf{NC}^1 \subseteq \mathbf{L}$.

Demostraci  n.

Sea $\mathcal{L} \in \mathbf{NC}^1$ y sean

- $(C_n)_{n \in \mathbb{N}}$ una familia de circuitos tal que
 - $|C_n|$ es polinomial en n ,
 - $\text{prof}(C_n) = O(\log n)$ y
 - $x \in \mathcal{L}$ sii $C_{|x|}(x) = 1$
- M una m  quina determin  stica que computa impl  citamente en $\mathbf{L}$ la funci  n $1^n \mapsto \langle C_n \rangle$

Definimos la siguiente funci  n $g(1^n, x, u)$ recursivamente:

- si u es la codificaci  n del k -  simo ($1 \leq k \leq n$) nodo de entrada de C_n , devolver $x(k-1)$
- si u es la codificaci  n de un nodo $*$ (con $*$ $\in \{\wedge, \vee\}$) de C_n , con hijos v_1, v_2 , devolver $g(1^n, x, v_1) * g(1^n, x, v_2)$
- si u es la codificaci  n de un nodo $\neg$ de C_n , con hijo v , devolver $\neg g(1^n, x, v)$

12-17

Tenemos

$$C_{|x|} = 1 \quad \text{sii} \quad g(1^{|x|}, x, u_0) = 1,$$

donde u_0 es la codificación de la salida (*sink*) de $C_{|x|}$.

Sea $n = |x|$.

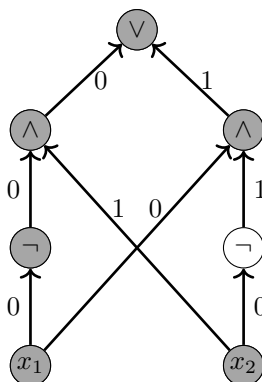
Podemos computar $g(1^n, x, u_0)$ en espacio $O(\log n)$: para resolver la recursión necesitamos

- llevar cuenta del camino desde la salida en C_n :
 $u_0 \rightarrow \dots \rightarrow u_i$, con $i \in O(\log n)$. Cada camino se codifica con una palabra en $\{0, 1\}^i$ (espacio $O(\log n)$):
 - 0 significa ‘hijo izquierdo’ o ‘único hijo’
 - 1 significa ‘hijo derecho’
- simular $M(1^n)$ para averiguar el tipo de nodo u_i : (espacio $O(\log n)$)

□

12-18

Ejemplo: evaluación de un circuito



- Es un recorrido DFS.
- Entre [] hay 1 bit que depende de los valores de x_1 y x_2 . Ahí va calculando operaciones intermedias.
- Cada estado usa espacio $O(h)$, donde h es la profundidad del circuito
- No entra todo el circuito en memoria: hay que ir pidiendo a M los nodos a medida que se necesitan para no usar más que espacio logarítmico

Estados:

ε
 0
 00
 000
 000[x_1]
 00[x_1]
 0[$\neg x_1$]
 0[$\neg x_1$]1
 0[$\neg x_1$]1[x_2]
 0[$\neg x_1$][x_2]
 [$\neg x_1 \wedge x_2$]
 [$\neg x_1 \wedge x_2$]1
 [$\neg x_1 \wedge x_2$]10
 [$\neg x_1 \wedge x_2$]10[x_1]
 [$\neg x_1 \wedge x_2$]1[x_1]
 ...

12-19

Teorema

Sea $\mathcal{L} \in \mathbf{NSPACE}(S(n))$. Existe una familia de circuitos $(C_n)_{n \in \mathbb{N}}$ y una máquina determinística M tal que para todo n y $x \in \{0, 1\}^n$

- $x \in \mathcal{L}$ sii $C_n(x) = 1$
- $M(1^n) = \langle C_n \rangle$
- C_n puede usar compuertas $\wedge$ y $\vee$ de *fan-in* arbitrario
- $|C_n| = 2^{O(S(n))}$
- $\text{prof}(C_n) = O(S(n))$

12-20

Demostración.

Sea N una máquina no-determinística que usa espacio $O(S(n))$ tal que $x \in \mathcal{L}$ sii existe un cómputo aceptador de $N(x)$.

Consideramos el grafo de configuraciones $G_{N,x}$, que tiene $m = 2^{c \cdot S(|x|)}$ nodos para alguna constante c .

Sea $A_x \in \{0, 1\}^{m \times m}$ la matriz de adyacencia de $G_{N,x}$

- $(A_x)_{ij} = 1$ sii i evoluciona en un paso en j .
- $B_x = A_x \vee I$: $(B_x)_{ij} = 1$ sii j es alcanzable desde i en ≤ 1 pasos.
- $(B_x^\ell)_{ij} = 1$ sii j es alcanzable desde i en $\leq \ell$ pasos.

Consideramos una multiplicación $\otimes$ de matrices donde $\vee$ juega de $+$ y $\wedge$ juega de $\cdot$.

En general para $C, D \in \{0, 1\}^{m \times m}$:

$$(C \otimes D)_{ij} = \bigvee_{1 \leq k \leq m} C_{ik} \wedge D_{kj}$$

(circuito de profundidad constante gracias al *fan-in* arbitrario)

12-21

Representamos matrices de dimensión $m \times m$ en circuitos: cada nodo es una posición de la matriz. Pensemos en una matriz como una tira de m^2 nodos.

Sea $n = |x|$. C_n hace esto:

- calcula A_x : profundidad constante.
- calcula B_x : ídem.
- calcula $D_x = B_x^m = B_x^{2^{c \cdot S(n)}}$: profundidad $O(\log m) = O(\log 2^{c \cdot S(n)}) = O(S(n))$.
Ejemplo: $B_x^{2^3} = ((B_x \cdot B_x) \cdot (B_x \cdot B_x)) \cdot ((B_x \cdot B_x) \cdot (B_x \cdot B_x))$
- la salida es $(D_x)_{1,2}$
(suponemos que la configuración inicial es 1 y la final es 2)

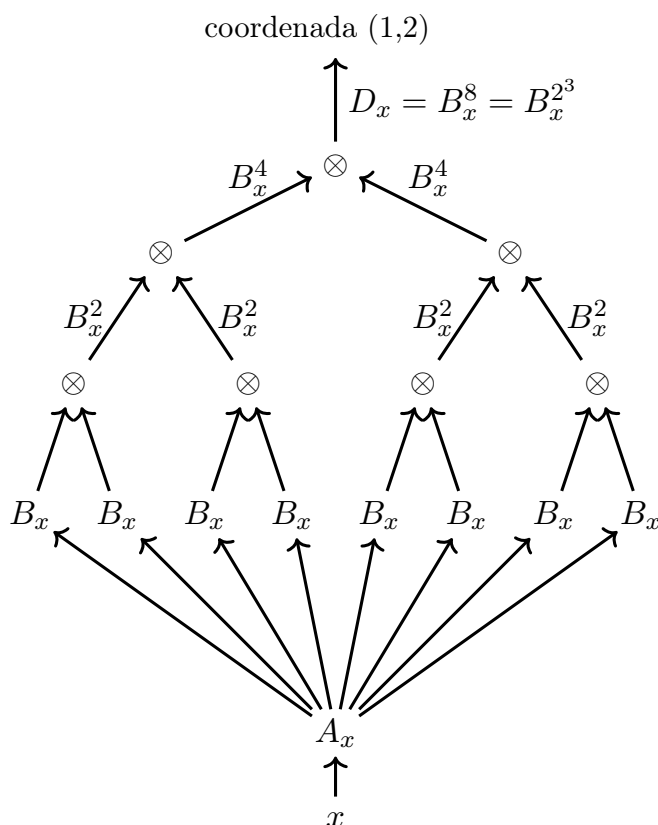
Tenemos

- $C_n(x) = 1$ sii N acepta x
- $\text{prof}(C_n) = O(S(n))$
- $|C_n| = 2^{O(S(n))}$
- La construcción es uniforme, es decir, existe una máquina determinística M tal que $M(1^n) = \langle C_n \rangle$

□

12-22

Ejemplo del circuito para $c \cdot S(n) = 3$



- $|x| = n$
- $|A_x| = |B_x| = |D_x| = (2^{c \cdot S(n)})^2 = 2^{O(S(n))}$
- En total, el circuito tiene $2^{O(S(n))}$ nodos y profundidad $O(S(n))$

12-23

Corolario

$\text{NL} \subseteq \text{AC}^1$.

Demostración.

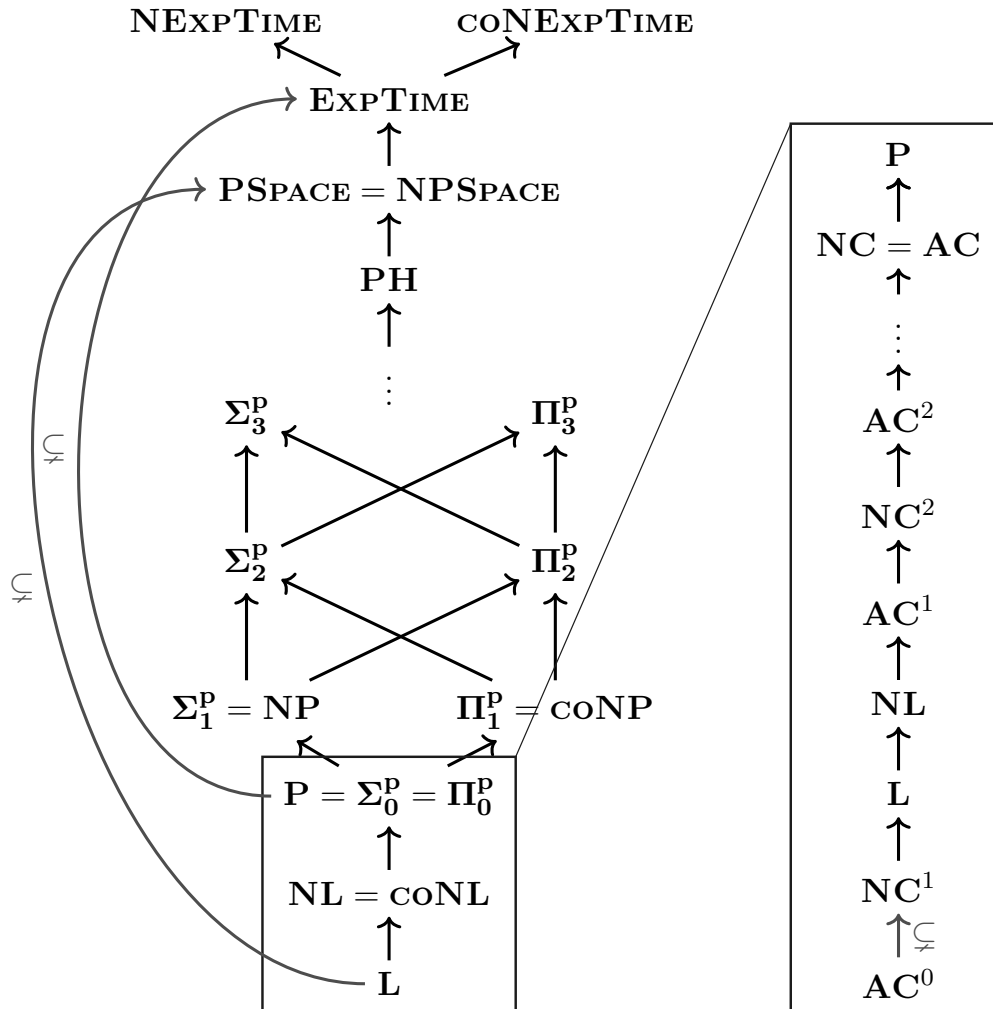
Sea $\mathcal{L} \in \text{NSPACE}(\log n)$. Existe una familia de circuitos $(C_n)_{n \in \mathbb{N}}$ y una máquina determinística M tal que para todo $n \geq 1$ y $x \in \{0, 1\}^n$

- $x \in \mathcal{L}$ sii $C_n(x) = 1$
- $M(1^n) = \langle C_n \rangle$
- C_n puede usar compuertas $\wedge$ y $\vee$ de *fan-in* arbitrario
- $|C_n| = 2^{O(\log n)} = n^{O(1)}$
- $\text{prof}(C_n) = O(\log n)$

Solo falta ver que la familia $(C_n)_{n \in \mathbb{N}}$ es **L**-uniforme.

Alcanza con ver que $1^n \mapsto \langle C_n \rangle$ es computable implícitamente en **L** por M . □

12-24



12-25

P-completitud

Clase 12

Las clases $\mathbf{NC}_{\text{nu}}$ y $\mathbf{AC}_{\text{nu}}$

Uniformidad: las clases $\mathbf{NC}$ y $\mathbf{AC}$

P-completitud

12-26

NC vs P

- ¿ $\mathbf{NC} = \mathbf{P}$ o $\mathbf{NC} \subsetneq \mathbf{P}$? Es una pregunta abierta.
- ¿Cualquier problema factible tiene solución paralela eficiente?
- Sabemos que $\mathbf{NC}^1 \subsetneq \mathbf{PSPACE}$ porque $\mathbf{NC}^1 \subseteq \mathbf{L}$ y $\mathbf{L} \subsetneq \mathbf{PSPACE}$.
- No sabemos $\mathbf{NC} \stackrel{?}{=} \mathbf{P}$.
- Ni siquiera sabemos $\mathbf{NC}^1 \stackrel{?}{=} \mathbf{PH}$.

Clase de complejidad: P-completo

$\mathcal{L}$ es **P-completo** si $\mathcal{L} \in \mathbf{P}$ y $\mathcal{L}' \leq_{\ell} \mathcal{L}$ para todo $\mathcal{L}' \in \mathbf{P}$.

Si apostamos a que $\mathbf{NC} \neq \mathbf{P}$, entonces los problemas **P-completos** son los que, a pesar de ser factibles, no tienen una solución paralela eficiente.

12-27

Un problema **P-completo**

Problema: Evaluación de un circuito

$\text{CIRC-EVAL} = \{ \langle C, x \rangle : \begin{array}{l} C \text{ es un circuito con } n \text{ entradas con } \text{única} \\ \text{salida, } x \in \{0, 1\}^n, \text{ y } C(x) = 1. \end{array} \}$

Proposición

CIRC-EVAL es P-completo.

Demostración.

Es claro que $\text{CIRC-EVAL} \in \mathbf{P}$.

Supongamos $\mathcal{L} \in \mathbf{P}$ y sea M una máquina determinística que corre en tiempo polinomial y $\mathcal{L}(M) = \mathcal{L}$. Ya vimos que $\mathcal{L}$ es decidible por una familia **L**-uniforme de circuitos.

Existe una función $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ computable implícitamente en **L** tal que $f(1^n) = \langle C_n \rangle$.

$$x \in \mathcal{L} \quad \text{sii} \quad C_{|x|}(x) = 1 \quad \text{sii} \quad \langle f(1^{|x|}), x \rangle \in \text{CIRC-EVAL}.$$

$x \mapsto \langle f(1^{|x|}), x \rangle$ es computable implícitamente en **L**.

Entonces $\mathcal{L} \leq_{\ell} \text{CIRC-EVAL}$. □

12-28

P-completitud

Ejercicio

Si $\mathcal{L} \in \mathbf{NC}$ y $\mathcal{L}' \leq_{\ell} \mathcal{L}$, entonces $\mathcal{L}' \in \mathbf{NC}$.

Teorema

Supongamos que $\mathcal{L}$ es **P-completo**.

1. $\mathcal{L} \in \mathbf{NC}$ sii $\mathbf{P} = \mathbf{NC}$
2. $\mathcal{L} \in \mathbf{L}$ sii $\mathbf{P} = \mathbf{L}$

Ejercicio

Probar el ítem 2.

Demostración del ítem 1.

$\Leftarrow$ es trivial.

Veamos que si $\mathcal{L} \in \mathbf{P-completo}$ y $\mathcal{L} \in \mathbf{NC}$ entonces $\mathbf{P} \subseteq \mathbf{NC}$ (ya sabemos que $\mathbf{NC} \subseteq \mathbf{P}$).

12-29

$\mathcal{L} \in \mathbf{P}$ -completo y $\mathcal{L} \in \mathbf{NC}$, entonces $\mathbf{P} \subseteq \mathbf{NC}$ (idea)

Como $\mathcal{L} \in \mathbf{NC}$, existe e y una familia $\mathbf{L}$ -uniforme $(C_n)_{n \in \mathbb{N}}$ de circuitos tal que $|C_n| = O(n^e)$, $\text{prof}(C_n) = O(\log^e n)$, y para todo $x \in \{0, 1\}^n$

$$C_n(x) = 1 \quad \text{sii} \quad x \in \mathcal{L}.$$

Sea $\mathcal{L}' \in \mathbf{P}$. Existe $f' : \{0, 1\}^* \rightarrow \{0, 1\}^*$ computable implícitamente en $\mathbf{L}$ tal que para todo $x \in \{0, 1\}^n$,

$$x \in \mathcal{L}' \quad \text{sii} \quad f'(x) \in \mathcal{L}.$$

Existe c tal que $|f'(x)| \leq c \cdot n^c$. Por simplicidad, supongamos que $|f'(x)| = c \cdot n^c$.

Como $\mathbf{L} \subseteq \mathbf{NC}$, existe d y una familia $\mathbf{L}$ -uniforme $(C'_n)_{n \in \mathbb{N}}$ de circuitos tal que $|C'_n| = O(n^d)$, $\text{prof}(C'_n) = O(\log^d n)$, y para todo $x \in \{0, 1\}^n$ $C'_n(x) = f'(x)$ (En realidad $\mathbf{L} \subseteq \mathbf{NC}$ lo vimos para lenguajes pero vale funciones computables implícitamente en $\mathbf{L}$, como f' .) Entonces

$$x \in \mathcal{L}' \quad \text{sii} \quad f'(x) \in \mathcal{L} \quad \text{sii} \quad C_{c \cdot n^c}(f'(x)) = 1 \quad \text{sii} \quad C_{c \cdot n^c}(C'_n(x)) = 1$$

La composición de los circuitos C y C' tiene tamaño polinomial, profundidad polilogarítmica, y se construyen uniformemente a partir de 1^n . Esto prueba que $\mathcal{L}' \in \mathbf{NC}$. □